



Contents lists available at ScienceDirect

Journal of Information and Intelligence

journal homepage: www.journals.elsevier.com/journal-of-information-and-intelligence

Automated data processing and feature engineering for deep learning and big data applications: A survey

Alhassan Mumuni^{a,*}, Fuseini Mumuni^b^a Department of Electrical and Electronic Engineering, Cape Coast Technical University, Cape Coast P. O. Box DL 50, Ghana^b Department of Electrical and Electronic Engineering, University of Mines and Technology, Tarkwa P. O. Box 237, Ghana

ARTICLE INFO

Keywords:

AutoML

Automated data preprocessing

Data processing

Automated feature engineering

Generative artificial intelligence

Big data

ABSTRACT

Modern approach to artificial intelligence (AI) aims to design algorithms that learn directly from data. This approach has achieved impressive results and has contributed significantly to the progress of AI, particularly in the sphere of supervised deep learning. It has also simplified the design of machine learning systems as the learning process is highly automated. However, not all data processing tasks in conventional deep learning pipelines have been automated. In most cases data has to be manually collected, preprocessed and further extended through data augmentation before they can be effective for training. Recently, special techniques for automating these tasks have emerged. The automation of data processing tasks is driven by the need to utilize large volumes of complex, heterogeneous data for machine learning and big data applications. Today, end-to-end automated data processing systems based on automated machine learning (AutoML) techniques are capable of taking raw data and transforming them into useful features for big data tasks by automating all intermediate processing stages. In this work, we present a thorough review of approaches for automating data processing tasks in deep learning pipelines, including automated data preprocessing – e.g., data cleaning, labeling, missing data imputation, and categorical data encoding – as well as data augmentation (including synthetic data generation using generative AI methods) and feature engineering – specifically, automated feature extraction, feature construction and feature selection. In addition to automating specific data processing tasks, we discuss the use of AutoML methods and tools to simultaneously optimize all stages of the machine learning pipeline.

1. Introduction

1.1. Background

Today, many problems are solved using machine learning (ML) methods. Deep learning approaches are generally preferred to traditional machine learning techniques for data-intensive tasks because of their ability to automatically extract useful features from data and perform low-level data processing. While deep learning methods have performed incredibly well, in the era of big data they have been shown to lack the power necessary to handle large volumes of complex heterogeneous data [1,2]. Moreover, there are many pertinent issues (e.g., bias [3], the presence of anomalies [4,5] and missing data points [6]) that require specific workarounds in deep learning pipelines. Consequently, additional steps are often needed to handle these complex data processing problems and to improve

* Corresponding author.

E-mail addresses: alhassan.mumuni@cctu.edu.gh (A. Mumuni), fmumuni@umat.edu.gh (F. Mumuni).<https://doi.org/10.1016/j.jiixd.2024.01.002>

Received 29 August 2023; Received in revised form 13 December 2023; Accepted 1 January 2024

Available online 8 January 2024

2949-7159/© 2024 The Author(s). Published by Elsevier B.V. on behalf of Xidian University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

the predictive performance and reliability of machine learning models in big data applications. In big data analytics, data processing is primarily aimed at simplifying the representation of data to reveal meaningful patterns, interrelationships and important trends. This is important in decision support systems, where human decision-making is enhanced by insights gained as a result of processing large quantities of relevant data. Fig. 1 shows a simplified workflow of a typical big data processing system.

The machine learning model design process itself is a time-consuming process requiring extensive domain knowledge. To train a model for image classification, for example, the developer would usually go through the following steps: (1) Collect and fine-tune image data, (2) choose a suitable ML algorithm, (3) define acceptable parameter and hyperparameter settings, (4) train the selected model on the training data, (5) assess the performance of the resulting model on test data, and (6) repeat the procedure from steps (3)–(6) until satisfactory performance is achieved. With this procedure, domain experts are needed to collect relevant data, carry out initial data preparation and perform additional processing and feature engineering to ensure that the resulting data is suitable for the specific machine learning task.

The basic workflow of these processing steps is shown in Fig. 2. In this paper, we explore various methods for automating data processing tasks for deep learning and big data applications. In the context of this survey, we broadly divide data processing tasks into three main subtasks: preprocessing, data augmentation and feature engineering (i.e., feature processing) functions.

1.2. Related works

Approaches for automating data acquisition and processing functions have received significant attention recently. Interest in these methods is largely driven by the need to leverage the enormous amount of data available in various forms from diverse sources for machine learning and big data applications. At the same time, the complexity of machine learning problems has increased drastically while requirements have also become more stringent. Additional factors such as data privacy and ethical issues limit access to data in application settings such as business, healthcare, security and law enforcement. Automated data processing systems provide a way for machine learning systems to process data in such situations where privacy issues may restrict access of the raw data to human actors. Indeed, healthcare [7,8] and business applications are already benefiting immensely from these techniques [9–11].

Despite the importance and resurgence of automated data processing methods, only a small number of survey works have been authored on these methods. With the exception of a few works such as He et al. [12] and Zöller and Huber [13], who dedicate a small part of their survey to automated data processing methods, most of the published works [13–16] have focused on ready-to-use, end-to-end automated machine learning (AutoML) tools designed for general purpose applications or they rather deal with the entire pipelines of custom AutoML implementations [7,16] without particular focus on data processing.

Strikingly, important aspects of low-level data processing tasks, including preprocessing and feature engineering methods have not been discussed sufficiently in the literature. While there exists a large number of extensive surveys on data augmentation methods [17–19], till date, relatively few surveys [6,20–23] have focused on data preprocessing and feature engineering methods. However, even for these works the techniques covered are mostly based on traditional methods. Our work is motivated by the dearth of literature specifically focused on current approaches to automating data processing functions in deep learning models, especially data preprocessing and feature engineering tasks.

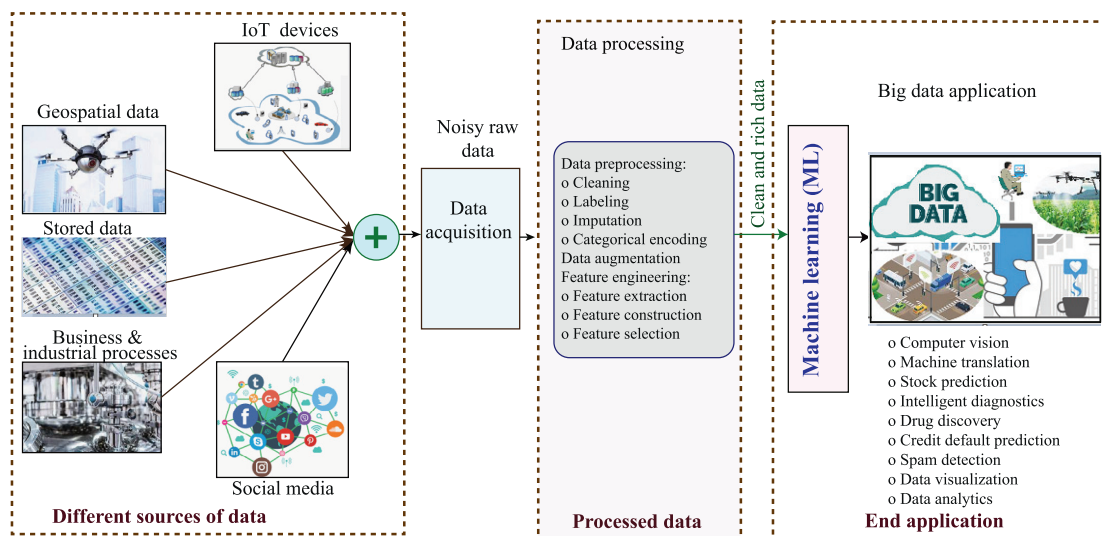


Fig. 1. Simplified representation of the basic components and workflow of a typical big data application. Typical big data applications rely on aggregating multimodal data from several different sources, and then applying suitable techniques to process these raw data to train deep learning models for downstream tasks.

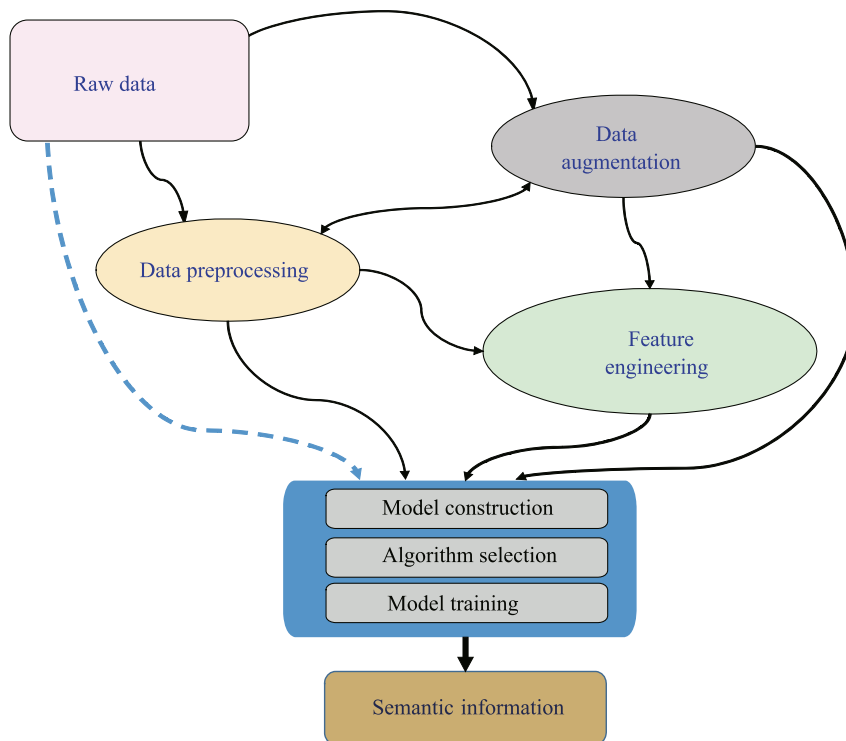


Fig. 2. Basic workflow of data preparation and feature engineering in machine learning and big data application development. There are a range of data preparation tasks (oval nodes) that may be performed before the data can be used for training. The arrows show the direction of progression of data processing steps. Note that, depending on the application requirements and the quality and quantity of the input data, some steps can be bypassed.

1.3. Scope and outline of survey

This work focuses on approaches for automating data preprocessing and feature engineering tasks in deep learning pipelines. While several recent surveys [7,12,24] have discussed issues on the entire AutoML pipeline – data preparation, feature engineering model generation, algorithm selection, performance evaluation and validation – our work specifically focuses on data preprocessing and feature engineering functions within these pipelines. For completeness, we also discuss data augmentation methods relevant to discussion in this work. In this survey, these functions are collectively referred to as data processing functions. The main data processing tasks discussed in this survey are presented in Fig. 3.

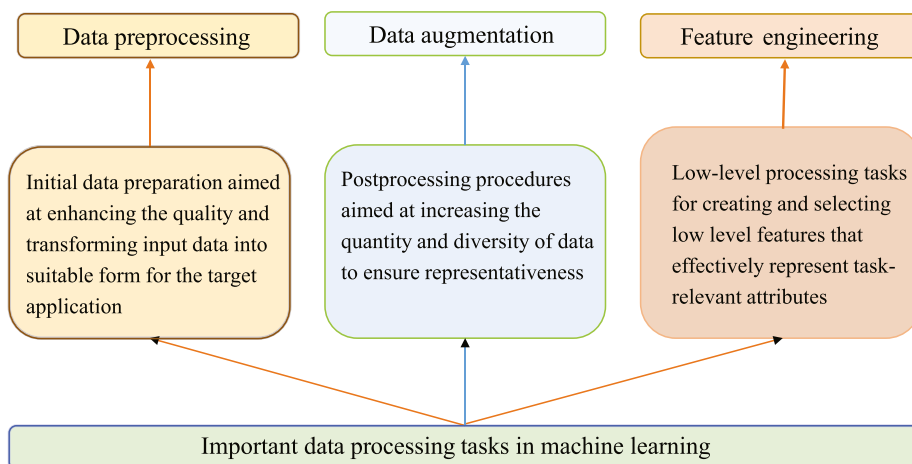


Fig. 3. Operational definition of the main data processing tasks discussed in this paper.

The rest of the survey is organized as follows. Section 2 provides an overview of the rationale, concepts and methods for automating data processing. In Section 3, automated data preprocessing techniques are covered. Data augmentation is treated in Section 4. Section 5 covers feature engineering methods. In Section 6, we discuss end-to-end workflows for data processing in deep learning pipelines where the focus is on approaches that perform all the different data processing tasks – including data preprocessing, augmentation and feature engineering – simultaneously in an end-to-end manner using a single machine learning framework. Section 7 summarizes the main features, application domains and categories of generic end-to-end AutoML tools used for data processing. In Section 8, we present an overview of some of the important applications of automated data processing techniques and their implications for industry and commerce. Furthermore, we provide an extensive discussion of pertinent issues and future prospects of automated data processing approaches in Section 9. We conclude in Section 10. Fig. 4 presents a broad outline of the this paper.

2. Automated data processing in machine learning pipelines

2.1. Data processing in traditional machine learning pipelines

The machine learning problem typically involves the following sequence of steps: data collection, data preparation and preprocessing, data augmentation, extraction of useful features from the data, construction of new features and pruning of the generated feature set (i.e., feature selection), selection of a suitable machine learning model, and optimization of model hyperparameters. Traditional machine learning approaches [25,26] require all of these stages of the machine learning process to be performed manually and as standalone processes (Fig. 5).

This approach presents a number of difficulties that make it extremely challenging to scale up machine learning models. For instance, traditional machine learning pipelines typically incorporate hand-crafted feature extractors to mine useful low-level features from training data for subsequent processing by the machine learning model. These feature extractors are basically case-specific algorithms aimed at finding the best set of features for the target task. Feature extractors, once designed, are typically fixed – i.e., in the process of learning, they cannot be modified automatically to work differently in response to new conditions. To adjust their operation, the algorithm may have to be redesigned and deployed anew.

2.2. Data processing in deep learning pipelines

With modern deep learning models based on neural network frameworks [27,28], data processing tasks are usually implemented within the machine learning framework and trained end-to-end (Fig. 6). Deep learning approaches employ neural networks that are

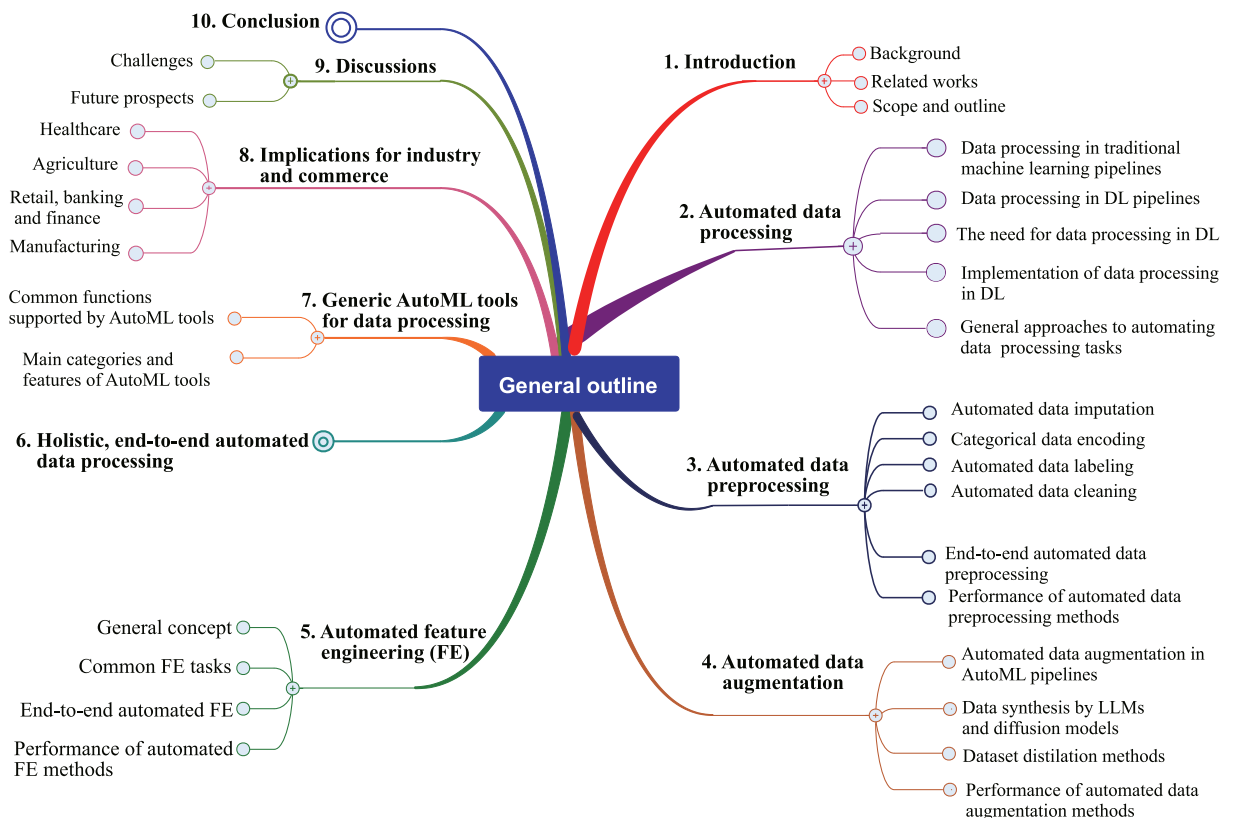


Fig. 4. Broad outline and structure of this paper.

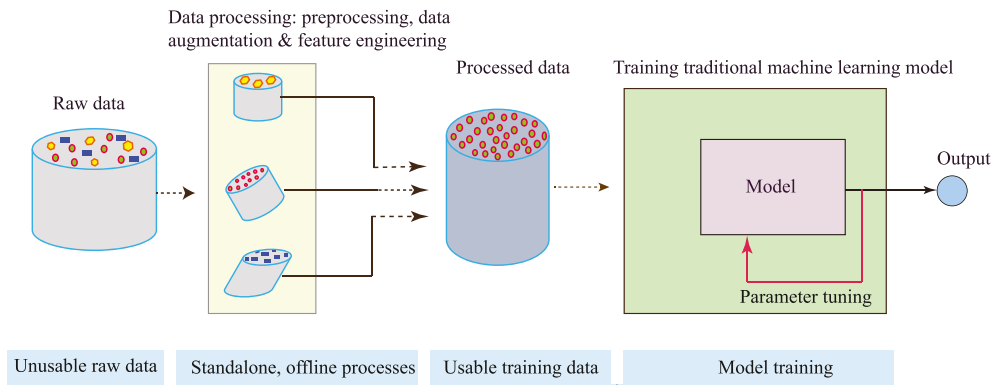


Fig. 5. Data processing in traditional machine learning. All processing steps are implemented manually as standalone tasks.

generally able to extract useful low-level information from raw data without the need for prior knowledge or additional processing. Information processing in deep learning pipelines roughly mimics the information processing properties of the brains of biological systems, where different semantic features are extracted using a large and functionally diverse network of complex neural structures. Several hierarchical layers of artificial neural networks extract different types of features at different semantic levels. In the case of computer vision, for example, shallower layers extract useful information relating to more basic, low-level visual concepts such as lines, object contours and shapes. Deeper layers sequentially increase the abstraction level of extracted information, mining higher and semantically meaningful information such as composite shapes and whole objects.

Unlike traditional machine learning pipelines where fixed feature extractors are responsible for information processing, in deep learning pipelines, during training, the synaptic elements that extract useful features use tunable parameters to adjust their operation on the fly based on feedback on the performance of the network on the target task. Thus, in the learning process the model automatically adjusts its way of feature extraction and low level data processing in response to performance results.

2.3. The need for additional data processing in deep learning models

2.3.1. The need for data preprocessing

Despite the high degree of automation of data processing in deep learning models, their performance has been shown to be highly dependent on the quality of training data [29–31]. However, in many practical scenarios, the qualitative properties of raw data are not often consistent with the requirements of the target application or model [2,32]. Consequently, data preprocessing has become an essential task in the deep learning application development process. Preprocessing includes initial data preparation tasks such as data normalization, cleaning (removal of outliers and inconsistent data points), encoding of categorical values, imputation of missing values and labeling.

2.3.2. The need for data augmentation

In many big data applications, the collected data may not be sufficient or representative enough for the target task. In such situations, after initial preprocessing, data augmentation is used to increase the quantity and variability of the training data by generating new

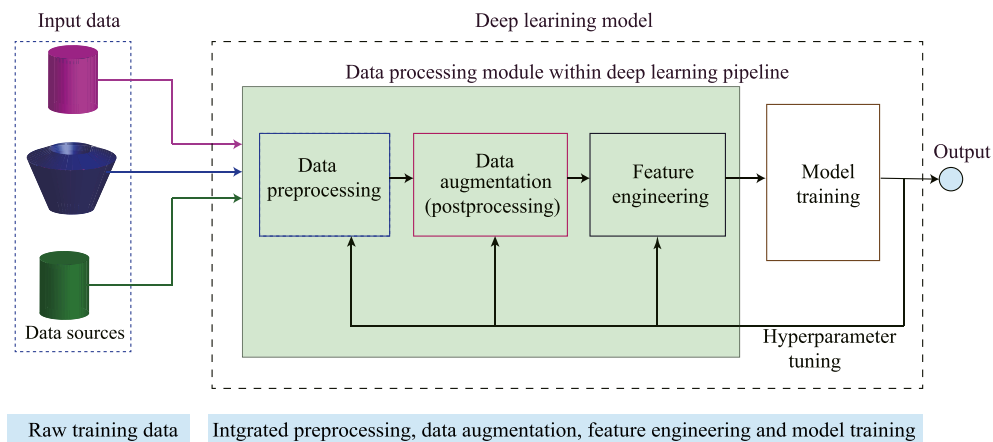


Fig. 6. Data processing in deep learning: All processing tasks – preprocessing, data augmentation and feature engineering functions – as well as model generation and training are realized as a unified process.

samples through manipulating the existing data. This is the most popular approach to enhancing the generalization power of deep learning models. Data augmentation has also been used to address problems such as bias [33], imbalanced data [34] and domain shift [35].

2.3.3. The need for feature engineering

While machine learning models constructed on the basis of deep neural architectures can automatically learn useful features [36], in some application settings these methods still require explicit feature processing to guarantee satisfactory performance. This is particularly so for complex and data-scarce applications such as fault diagnosis in machinery (e.g., in Ref. [37]), online fraud detection (e.g., Ref. [38]), credit risk evaluation [39] and some industrial applications [40]. Indeed, it has been shown in Ref. [41] that employing dedicated feature extractors in deep learning networks provides more useful feature representations of the input data that results in significant performance improvements.

Also, for big data applications, datasets are generally very large, heterogeneous, complex and noisy, leading to the need to handle very large volumes of features, often with redundant and duplicated samples. These intricacies may complicate the training process and lead to undesirable outcomes that can be harmful to performance [3,42]. This problem can be mitigated by feature engineering methods, which aim to find compact feature representations for the data. Another common shortcoming of features naturally learned in the usual process of training deep neural networks is that they are typically not interpretable. This can be a major limitation in mission-critical application settings such as medical domains [43], where the decisions of deep learning systems need to be understood and validated by human experts. Feature engineering is also useful for tasks where training data is insufficient to adequately encode the desired information [44].

2.4. Implementation of data processing functions in deep learning pipelines

Unlike in shallow learning where data processing is performed offline as a standalone process, modern deep learning favors an end-to-end learning paradigm that aims at realizing all functions in an integrated manner using a single model. This means in deep learning, additional data processing, when needed, is usually integrated into the machine learning process and implemented as a unified process (Fig. 6). The fact that this process is carried out online means that no additional data samples are stored. This has the advantage of increasing the volume and diversity of training data without a corresponding increase in the amount of storage requirements. Instead of applying explicitly-designed transformation operations to create additional data, many new works (e.g., Refs. [45–50]) have devised special neural network architectures to learn the relevant transformations directly from the training data. In this case, the data processing tasks become an integral part of the overall learning process.

2.5. General approaches to automating data processing and feature engineering tasks

Many different techniques are used to automate data processing tasks. These techniques have different strengths and limitations related to complexity, flexibility and predictive performance. While the crudest approaches to automating data processing tasks involve using hand-crafted routines, more advanced methods [51–53] utilize learned mechanisms to perform processing. The most advanced methods, however, employ automated machine learning (AutoML) techniques [12,54–56] to automate data processing functions. These methods can perform all the underlying data preprocessing tasks end-to-end without delegating intermediate tasks to human developers.

A typical AutoML pipeline for end-to-end data processing is shown in Fig. 7. The first part of the pipeline is concerned with data processing tasks. These include preprocessing, data augmentation and feature engineering. The second part of the AutoML pipeline performs functions such as model construction, hyperparameter setting, algorithm selection and result evaluation and validation. Additional functions such as result visualization (e.g., in Refs. [57,58]) may also be automated. In some cases (e.g., Ref. [59]), the automated machine learning problem involves composing an optimum model architecture to solve the machine learning task. Approaches for automating model structure construction belong to a class of automated machine learning methods called neural architecture search or NAS [60]. We describe the main features and functions of generic automated machine learning systems in Section 7.

3. Automated data preprocessing

3.1. The concept of data preprocessing

Data preprocessing functions consist of a set of basic operations that transform raw data into a form that is useful for the machine learning task. Important preprocessing subtasks include data cleaning [61,62], labeling or re-labeling [53,63,64], categorical encoding [65–67] and imputation of missing data [68–70]. Fig. 8 depicts the main categories of data preprocessing tasks and the set of common problems they commonly tackle. In general, as shown in Table 1, many works attempt to accomplish multiple data preprocessing tasks using a single deep learning framework.

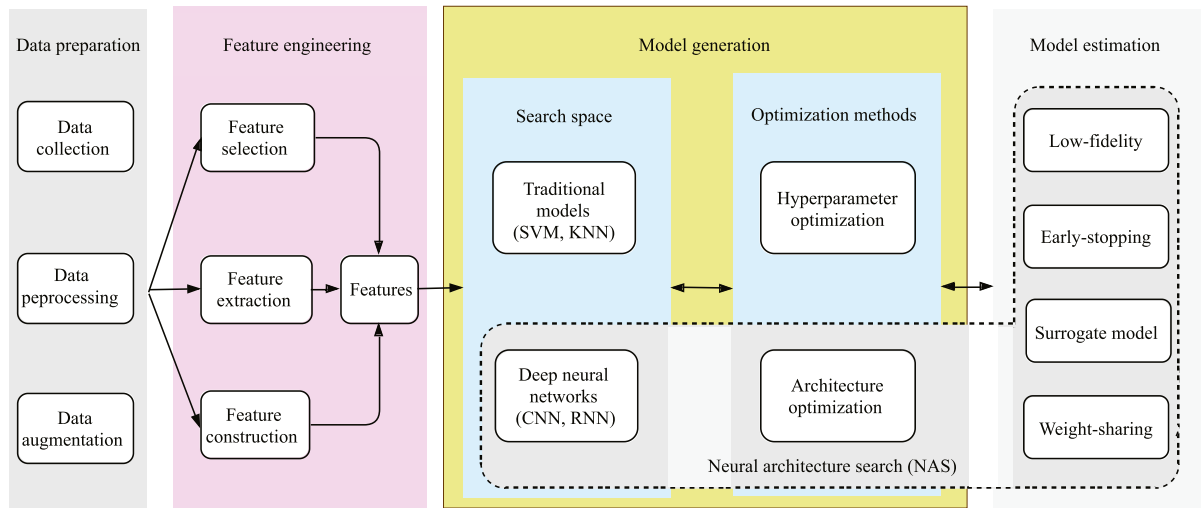


Fig. 7. General representation of AutoML pipeline. It incorporates deep learning and traditional machine learning models in its design. In the training process, optimization algorithms select the best model structure and hyperparameter setting for the specific task. The original illustration, from Ref. [12], has been modified slightly to reflect the context of the current paper.

3.2. Formalization of the automated preprocessing problem

Given a dataset D , the automated preprocessing problem can be defined as a task involving the automatic selection and application of a set of basic preprocessing operations $P_i \in S: i = 1, 2, 3, \dots, n$ on D such that the predictive performance on a target task is maximized. This is a challenging task because, besides determining which transformations are appropriate and to what extent to transform the data, the machine learning algorithm needs to also apply the operations in the correct order [71]. The most advanced automated preprocessing methods [72] typically take as input a dataset and a set of primitive transformation operations and their corresponding hyperparameters – parameters that define certain details about the model, including the strength of transformations and the order of their application. Optimization algorithms such as reinforcement learning (RL), gradient descent (GD), Bayesian optimization (BO) or evolutionary computational algorithms (ECA) are then employed to search for the best combination of these basic preprocessing operations to apply on the target data. The main features of some of the most important preprocessing methods are summarized in Table 1.

3.3. Approaches to automated preprocessing

Data preprocessing can be automated to varying degrees. The degree of automation depends on how preprocessing operations and the underlying models are implemented. The most basic automation still requires the use of traditional, hard-coded techniques while the

Data imputation	Categorical data encoding	Data cleaning	Data labeling & re-labeling
- Missing data points - Missing attributes - Incomplete data - Missing values - Missing keys - Missing meta data - Incomplete metadata - Insufficient data - Balance imbalanced data	- Need to convert ordinal data to numerical - Need to convert nominal data to numerical - Need to weight data or attributes - Need to ensure compatibility of data across different systems	- Invalid data - Ordering errors - Noisy or corrupted data - Redundant data - Irrelevant data - Wrong data types or categories - Inconsistent formats - Overall low quality data	- Unlabeled data - Tagging errors (for crowd-sourced data) - Incorrect labels - Incorrect category nomenclature - Typographical errors in labels - Domain shift

Fig. 8. Data preprocessing tasks and the set of common problems they typically tackle.

Table 1

Summary of important data preprocessing methods. We capture the main preprocessing functions (imputation, cleaning, labeling, and categorical encoding) each of the techniques performs and the supported data types. We also indicate the main machine learning tasks, the classification of the approaches and the datasets used.

Work	Data type/task	Main preprocessing functions				Approach to automation	Main datasets
		Data input.	Data clean.	Data label.	Categ. encod.		
scIGANs [51]	Tabular (genomics)	✓				Generative modeling	PMBC 10K, Synthetic
MIWAE [87]	Tabular (classification)	✓				Generative modeling	UCI Repository
Automunge [68]	Tabular, insurance claims, housing price (regression)	✓	✓	✓	✓	Mostly analytical	Ames Housing, Allstate Claims
HyperImpute [89]	Tabular (classification)	✓				AutoML	UCI
PLACL [63]	Image (keypoint localization)			✓		Deep learned	MPII, CUB-200-2011, ATRW, MS-COCO, AnimalPose
cleanTS [62]	Tabular (time series)	✓	✓	✓		AutoML	Power, Temperature, CO2 Emission
GP-VAE [88]	Image, digitized signals (binary format)	✓				Generative modeling (VAE)	Physionet, Healing MNIST, SPRITES
DataAssist [125]	Tabular	✓	✓	✓	✓	Deep learned	Various (Kaggle)
Gain [81]	Tabular (classification)	✓				Generative modeling	UCI
DiffPrep [126]	Tabular (regression)	✓	✓		✓	AutoML (gradient-based)	OpenML
BERT-Sort [99]	Tabular (regression, classification)				✓	Deep learned	Various (UCI, Kaggle)
BioAutoMATED [127]	Sequence-based data (genomics)		✓		✓	AutoML	Glycan, Peptide sequences, Ribosome binding
Rotom [108]	Text, tabular, entity matching, NLP		✓	✓		Deep learned (meta-learning)	TextCLS, EDT, EM
Learn2Clean [104]	Text, multimedia (web data)	✓	✓			Deep learned (RL)	House Prices, Google Play Store Apps, Users
AlphaClean [128]	Tabular		✓			AutoML (greedy search)	LAQ, Physician, Hospital
AutoData [53]	Image, text (from the web)		✓	✓		Deep learned (RL)	Custom (web data)
BoostClean [105]	Tabular (classification)	✓	✓			Deep learned	Various (UCI, Kaggle, USCensus, NFL, Titanic, etc).

most advanced methods utilize AutoML frameworks. The general approaches for automating data processing tasks are summarized in Table 2.

Generally, all automated preprocessing approaches use some form of machine learning to define and/or select an ensemble of data preprocessing operators or deep learning pipelines from a set of possible options that maximize performance. We discuss in detail specific approaches for automating some of the most important preprocessing tasks in the next subsections.

One major difficulty with data preprocessing is that not only are the processing methods dependent on data type, but they are also highly sensitive to the machine learning model [73]. This means for different models and tasks, the application of the same preprocessing operations on the same type of input data can often lead to vastly different outcomes. Applying all possible preprocessing operations irrespective of model or data type is a highly combinatorial problem and impractical to carry out in applications dealing with very large data. Consequently, many fully-automated preprocessing methods rely on first identifying the input data and model types and then applying context-specific preprocessing operations. Unfortunately, in some situations it is not often possible to determine beforehand what operations could be effective for particular models or data types. To solve this problem, approaches (e.g., Ref. [74]) have been proposed to jointly estimate the performance of primitive preprocessing operations so as to apply them in a data- and model-dependent way.

Large-scale AutoML frameworks such as AutoGluon-Tabular [75] incorporate a two-level preprocessing scheme, with the first level implementing model-agnostic preprocessing operations while the second one generally focuses on model-specific preprocessing. These AutoML models typically combine many different optimization algorithms and pipelines with different structures that define data preprocessing blocks and machine learning model blocks within a single framework. The composite nature of this class of models makes generic AutoML tools flexible and multipurpose, and allows them to solve a wide range of problems.

Table 2

General approaches for automating data preprocessing tasks. The approaches vary in sophistication, from basic to fully automated methods based on the level of automation of the implemented AutoML pipelines.

Method	Description and main characteristics	Example works
Basic	Standalone preprocessing, usually manual or with the aid of generic data processing tools (e.g., image processing libraries for manipulating images). This approach is largely manual.	[131,132]
Analytical	Automated processing within deep learning pipelines using explicitly formulated analytical relations.	[45,49,133]
DL	Deeply learned processing modules within machine learning pipelines.	[51,83,130]
AutoML	End-to-end processing, model selection and hyperparameter tuning using AutoML.	[53,54,61,68,126,134]

3.4. Automated data imputation

Handling missing data is a common task that is encountered when developing machine learning models [76]. The problem of missing data arises from various causes associated with data acquisition, including lost data entries, inability to collect certain data, or from situations where some data is simply not accessible for various reasons such as privacy concerns. Missing data can also result from incorrect measurements and human errors, especially in data systems relying on manual entry. Traditional data imputation methods (e.g., Refs. [77,78]) rely on analytically formulated statistical techniques. The methods first identify missing data using knowledge about the statistical properties of the observed data. Missing values are then computed and incorporated into the training set. In contrast, automated imputation techniques [79,80] discover and correct issues with missing data without encoding explicit mechanisms for doing so.

Generative modeling approaches have emerged as an important class of methods for automated missing data imputation (e.g., in Refs. [51,81–86]). These methods use learned mechanisms to encode the correct distribution of “normal” data. The learned knowledge about the underlying data distribution can then be used to help detect instances of missing data and perform imputation in an end-to-end manner. Yoon et al., for instance, propose a so-called generative adversarial imputation network (GAIN) [81] that utilizes a generative adversarial network (GAN) to model data distribution for identification and imputation of missing data. Gondara and Wang [82] perform data imputation with the help of denoising autoencoders. Generative techniques that utilize variational autoencoders are also common (e.g., Refs. [52,87,88]).

More recently, methods based on AutoML techniques [62,68,79,89] have emerged as the most important class of methods for automating data imputation. Teague [68], for example, develops a tool based on AutoML technique to automatically impute missing tabular data. HyperImpute [89] implements missing data imputation by utilizing an AutoML framework that optimizes multiple candidate machine learning models using different search algorithms. Its basic structure is shown in Fig. 9. The approach can realize a more generalized and adaptive imputation process than methods that rely on optimizing a single machine learning model using a single search technique. In addition to these dedicated data imputation models, most popular automated machine learning tools such as Auto-Sklearn [90], Scikit-Learn [91] and Azure Databricks AutoML [92] aim to, among other things, perform automatic imputation in an end-to-end manner.

It has been recognized that fully automated methods of data imputation can be detrimental to performance in some situations [93]. While machine learning techniques can generally detect situations of missing data points, determining the correct data input can sometimes require some form of high-level understanding of the underlying context which machine learning models often lack. To address this limitation, some authors adopt human-in-the-loop approaches (e.g., Refs. [80,94,95]). Bilal et al. in Ref. [80], for example, implement a mechanism to automatically detect missing data and prompt the user, through a data visualization interface, to accept or reject suggested candidate data modifications.

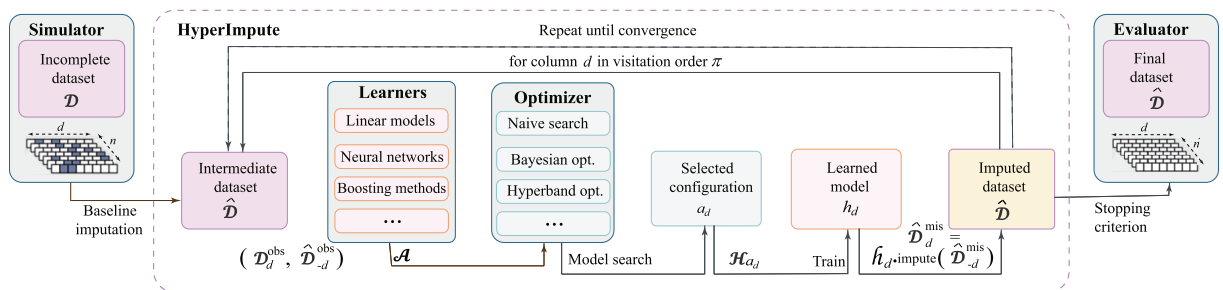


Fig. 9. Basic architecture and workflow of HyperImpute, an automated data imputation method proposed in Ref. [89]. The approach incorporates different ML models and optimization techniques which are used to find an optimal imputation pipeline.

3.5. Categorical data encoding

Categorical data encoding is aimed at converting categorical data into numerical information. This preprocessing step is often necessary because deep learning models typically work well with numerical data but are unable to handle categorical data. However, many real-world datasets often include categorical variables. Traditional approaches to converting categorical variables into numerical representations rely on techniques such as binary encoding, ordinal encoding, label encoding, one-hot encoding and target encoding [65].

Currently, it is challenging to completely automate the entire process of categorical data encoding. Unlike preprocessing functions such as data cleaning and imputation, it is difficult to define primitive operations that can be applied by machine learning models to convert between categorical and numerical data. Because of this limitation, many approaches, including state-of-the-art AutoML models such as TPOT [96] and Auto-Keras [97] currently implement encoding manually. Some AutoML-based categorical encoding methods [54,98,99] employ traditional encoding schemes within deep learning networks. The approach in Ref. [54] incorporates custom implementations of traditional categorical data encoding and other preprocessing functions within a generic AutoML framework. It employs a tree-based ensemble technique to evaluate and select the encoded elements. Similarly, H2O AutoML framework [100] implements categorical data encoding using OneHotEncoding library – an implementation of the traditional one-hot encoding method.

Modern AutoML tools that attempt to automate this preprocessing step (e.g., LightAutoML [101], AutoGluon-Tabular [75], Auto-Sklearn [90] and Pharm-AutoML) mostly require some form of human supervision. For example, Auto-Sklearn includes a so-called label encoder – a categorical data to integer conversion algorithm, but it requires user input to define the underlying categories before performing the necessary operations.

While many approaches rely on using traditional techniques within automated learning frameworks to perform categorical to numerical data conversion, some recent works suggest encoding categorical data with the help of learned word embeddings. The authors in Ref. [102], for instance, propose a word embedding method based on deep neural network model to learn useful features for encoding categorical data. BERT-Sort [99] employs a learned embedding approach that exploits semantic relationships among ordinal data elements to perform categorical encoding. The basic idea is to learn the context of textual information with the help of a pre-trained language model. This knowledge is then used to encode input data according to the high-level semantic context of the underlying text.

3.6. Automated data cleaning

Data cleaning [103] is aimed at correcting errors in a dataset or eliminating noisy data. The main sources of these errors include ordering and indexing mistakes, incorrect class assignments and inconsistent naming. In machine learning domains such as natural language processing, common errors include typographical, grammatical and syntax errors. Random errors and anomalies can also have significant impact on the performance of time-series prediction. Duplicate or redundant and irrelevant data may also be present in the training dataset and may need to be removed to preserve the fidelity of the data. Data cleaning may also involve removing incomplete – i.e., absence of some necessary attribute(s) required for the target task, incorrect or invalid (e.g., wrong data types) samples. Operations such as normalization, zero centering and scaling are additionally used to ensure that the statistical characteristics of the data are suitable for use in the deep learning model.

Basic data cleaning automation can be realized by implementing high-level wrapper functions around low-level data cleaning routines. CleanTS [62], for instance, develops a set of high-level abstractions and functions at a semantic level on top of a wide-ranging suit of low-level library functions to automate data cleaning tasks in time-series applications.

A recent trend in data cleaning is to use machine learning as a way to identify discrepancies and errors and apply appropriate corrections. Learning-based data cleaning methods (e.g., Refs. [95,104–106]) optimize parameters associated with data cleaning operations based on the performance of the underlying network on downstream tasks. Learn2Clean [104], for example, employs a Q-learning algorithm to learn the best data cleaning operations. BoostClean [105] relies on a learning approach that utilizes gradient boosting to automate the data cleaning process.

Meta-learning techniques (e.g., Refs. [107,108]) have been employed to extend the scope and enhance the adaptability of data cleaning systems. This allows them to be applied as generic data cleaning models to process a wide variety of datasets for many different machine learning tasks. End-to-end AutoML-based data cleaning pipelines [62,98] have been proposed as stand-alone tools for generic cleaning purposes. For instance, Shende et al. in Ref. [62] develop a generic time series data cleaning method and accompanying toolset that tackles problems with outliers, duplicates, as well as inconsistent data types and formats.

3.7. Automated data labeling

Labeling, or annotation, is an important task in data preparation and in supervised machine learning in general. It is arguably the most tedious job in many machine learning tasks. Unfortunately, in many domains, it is also one of the most difficult processes to automate. At present, most annotations are generated either manually or semi-automatically, that is, using algorithms to generate possible label proposals which are subsequently refined and validated by human agents (e.g., in Refs. [109–113]). Only a few works in computer vision [114,115] and medical image analysis [110,116] domains have attempted carrying out the labeling process in a fully automatic manner.

Automated labeling techniques in computer vision and medical image analysis applications [110,112,117] typically employ classification, object detection or semantic segmentation models to first identify instances before generating labels for them. For instance, Ince et al. [112] employ a standard object detector model as the basis for annotations but the approach requires user validation of the generated labels. Zhang et al. [117] use metric learning approach to match unlabeled images with existing category information and then associate the unlabeled images with labels of relevant image categories. The basic principle of the approach is illustrated in Fig. 10.

Another common method is to exploit the rich textual descriptions naturally associated with certain types of data to obtain annotations for unlabeled samples. This is particularly common in medical imaging domains [110,118], where images are often associated with pathology reports that provide relevant descriptions of the corresponding image content that can be leveraged through training. Zhang et al. [110], for instance, proposed an automatic annotation mechanism for thyroid nodules in ultrasound scans using a natural language processing (NLP) model in combination with fixed rules (ontologies) to extract nodule information from medical pathology reports.

Even though a variety of machine-guided automated labeling [12,53,119–121] techniques have been proposed, they are generally limited to simple labeling tasks targeting very narrow application settings. It is currently challenging to extend them to more complex or general tasks. The main difficulties include the need for context awareness [115,122], lexical complexity [118], semantic ambiguity [123] and label subjectivity [124].

3.8. End-to-end automated data preprocessing

While a number of preprocessing methods are designed to specifically perform a single preprocessing task (e.g., in Refs. [84,89]), many works (e.g., Refs. [61,80,126]) tend to simultaneously implement multiple preprocessing functions. For example, Auto-Prep [80] performs automatic missing data imputation, data type detection and deletion of duplicates, categorical data encoding as well as feature scaling. AutoDC [61] performs outlier detection and elimination, label correction, edge case selection as well as data augmentation.

As we have already mentioned, most of the preprocessing methods based on AutoML frameworks generally aim to automate the entire machine learning pipeline, including data processing as well as model structure construction and algorithm selection functions. Because of their generic focus, they are able to handle only basic preprocessing tasks automatically. More complex preprocessing steps often require human guidance to accomplish. Models such as TPOT and Auto-Keras, for example, perform only missing data detection. While human-in-the-machine-learning-loop has been suggested [80,94] as a viable solution, some researchers [129] suggest performing human and machine learning preprocessing independently and combining the results for better and reliable performance.

Some new approaches such as AutoData [53], DataAssist [125], BioAutoMATED [127], Atlantic [54] and DiffPrep [126] are designed as dedicated preprocessing plug-ins that can be integrated into standard AutoML frameworks. In this way, they can leverage the hyperparameter and pipeline optimization capabilities of large-scale AutoML models to optimize preprocessing functions. These models are generally aimed at addressing the limitations of generic methods by developing specialized modules that can be used to perform specific data preparation tasks within general-purpose AutoML pipelines. AutoData [53] (Fig. 11), for instance, implements a dedicated data processing module that can be used within standard AutoML frameworks as an end-to-end mechanism for performing many data preprocessing and augmentation tasks, including data acquisition, labeling, cleaning and augmentation. It uses reinforcement learning to search for suitable data from diverse external sources. It first takes a user-defined input specifying the required data type and

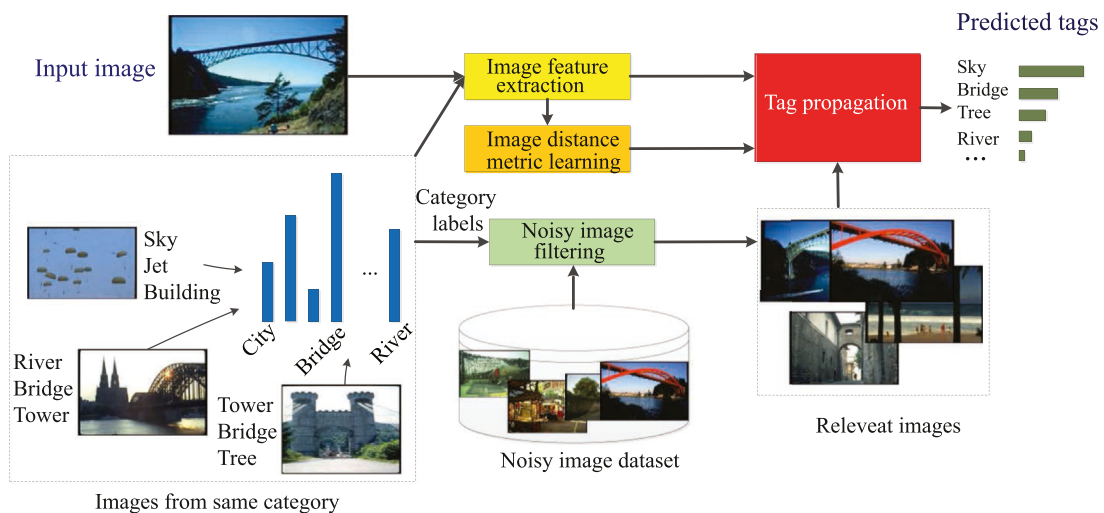


Fig. 10. Automatic image annotation by Zhang et al. [117]: The model first extracts and combines both hand-crafted and deeply learned visual features for two sets of images, labeled categories and unlabeled images. Metric learning is then used to estimate the similarity of unlabeled images and the known labeled categories. Next, the labels of relevant images from the labeled categories are transferred to the unlabeled instances based on their similarity scores. Information about the labeled categories is also used to filter noisy input samples to provide clean images for labeling.

task, as well as the overall performance objective. Given a particular input, the reinforcement learning algorithm searches for the appropriate data from external repositories and feeds the results to an AutoML framework (e.g., Auto-WEKA or Auto-Keras) which then constructs a model and evaluates the performance on the given task and provide feedback for refining the search results. Table 2 presents the main approaches for automating various preprocessing tasks. These reflect different degrees of automation – from the most basic (i.e., manual) to fully automated preprocessing based on AutoML.

3.9. Performance of automated data preprocessing methods

Despite the proven effectiveness and the proliferation of data preprocessing methods, there is a general lack of quantitative performance results for data preprocessing methods. The scarcity of comparative performance data in the literature can be attributed to several factors. Firstly, data preprocessing methods often vary widely in their implementation and application; hence, there are hardly any standard techniques for performing specific preprocessing tasks. In particular, there are no standard algorithms for common tasks such as data cleaning, labeling and imputation. Because of the highly varied training objectives and implementation details, it is generally difficult to compare performance across different studies.

The second major difficulty is that data preprocessing is rarely accomplished by a single algorithmic operation. Each of the preprocessing procedures (i.e., data imputation, labeling, cleaning, etc.) often involves a set of multiple sequential or parallel processing operations. Comparing different methods requires defining consistent pipelines, which can be very complex given the many possible combinations of preprocessing procedures.

Another important challenge is that most data preprocessing procedures involve subjective considerations. For example, determining outliers or selecting which data elements to retain can be influenced by domain knowledge and human bias. This subjectivity also makes it difficult to obtain fair and objective comparisons.

4. Automated data augmentation

Data augmentation involves creating variations of the original training data by applying context-appropriate transformation operations. For example, in computer vision, these transformations may involve image scaling, shearing, rotation, reflection and blurring operations. Text augmentation methods include synonym replacement, random insertion and deletion. The creation of such variations lead to a more representative data for the given task, and enhances the machine learning model's ability to generalize well on unseen data. Fig. 12 summarizes the common data augmentation operations for common machine learning tasks. Typically, for a particular task, data augmentation may involve applying many of these basic operations together with more advanced techniques.

As explained earlier, in most cases data augmentation is achieved by manipulating existing data in such a way as to create adequate variability. Most state-of-the-art methods for automated data augmentation rely on AutoML techniques. There are, however, practical situations where training data does not exist in any form [135], and augmentation methods are required to synthesize novel data from scratch. Generative artificial intelligence techniques have excelled in this area.

Presently, for data manipulation, automation has largely been successful in computer vision and image understanding domains. It has been particularly challenging to extend these methods to time series forecasting and natural language processing domains owing to the difficulty in achieving semantic-preserving transformations without human supervision. Only a very few works (e.g., Refs. [136,137]) have reported successful application of automated data augmentation techniques in natural language processing domains. Recently approaches for generative AI techniques have achieved remarkable results in text generation.

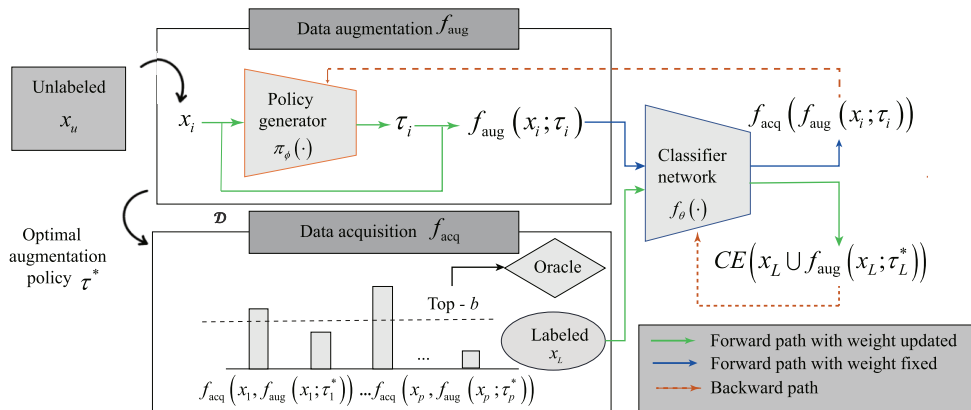


Fig. 11. Data labeling by LADA: The LADA framework proposed in Ref. [130] acquires unlabeled training data from the real world. An augmentation policy is then used to carry out initial preprocessing and generate additional data by learning relevant transformation operations. With the aid of a classification network, the optimum policy is determined using the classification loss. Finally, an oracle is used to label the unlabeled raw data.

Machine learning task	Basic data augmentation methods
Image processing and computer vision (image classification, object detection, visual tracking, etc.)	• Rotation: Rotating images by various angles to simulate different viewpoints • Flipping: Horizontally flipping images to create mirror images • Cropping: Randomly cropping images to focus on different parts of the scene • Zooming: Applying random zoom to create variations in image scale • Color jittering: Modifying the brightness, contrast, saturation, and hue of images • Gaussian noise: Adding random Gaussian noise to images to simulate real-world variations • Blurring: Applying Gaussian blur or other blurring techniques to images
Text recognition & language understanding (machine translation, text-to-voice conversion, etc.)	• Synonym replacement: Replacing words with their synonyms to create variations. • Random deletion: Randomly removing words to change sentence length and structure • Random insertion: Randomly inserting additional words into a sentence to simulate natural language variation • Text paraphrasing: Rephrasing sentences while preserving the original meaning • Character-level perturbations: Swapping, inserting, or deleting characters to simulate typographical errors or variations in spelling • Contextualized word embedding: Replacing words with alternatives that are contextually relevant
Time series forecasting (e.g., stock price prediction, economic performance forecasting, sales forecasting, etc.)	• Dynamic time warping: Stretching or compressing the time axis of a time series to induce variations in the temporal scale of the data • Window slicing: Dividing the original time series into overlapping or non-overlapping windows of varying lengths can provide the model with multiple perspectives of the data • Noise addition: Introducing random noise to the time series data • Data interpolation: Filling missing data points using interpolation techniques • Outlier injection: Introducing outliers or anomalies into the time series data can help the model learn how to identify and handle unexpected events. • Time shifting: Shifting the entire time series or specific portions of it forward or backward in time to create new instances with shifted patterns.

Fig. 12. Common data augmentation functions for various machine learning tasks. Note that this list is by no means complete.

4.1. Automated data augmentation in AutoML pipelines

AutoML has emerged as a powerful approach that has achieved significant success in data augmentation for large-scale machine learning and big data applications. AutoML-based data augmentation methods generally aimed to automate the process of creating and applying appropriate transformation operations on the training data. In addition to data augmentation, the AutoML models perform a number of additional tasks in line with machine learning model development (e.g., model selection, hyperparameter tuning, performance validation, etc.). In an AutoML framework, a large number of augmentation operations are usually created manually or in an automated way, and the data augmentation task reduces to a simple search for the best transformation operations and their associated hyperparameters. The basic concept of AutoML approach to data augmentation is shown in Fig. 13.

4.1.1. Generation of augmentation operations

The first step in the augmentation process is to generate diverse transformation operations to manipulate the input data. In image augmentation, for example, the basic transformations are typically standard geometric and photometric image processing functions like rotation, flipping, scaling, shearing, color jittering, solarizaion, noise addition and contrast adjustment. Transformation magnitudes may include scale factors, rotation angels, translation offsets, color intensity and brightness levels, etc. The various transformation operations and their corresponding magnitudes constitute a search space.

The effectiveness of automated augmentation strategies is based largely on the ability to compose a comprehensive set of operations that are representative of the tasks under consideration. The general approach to automating data augmentation tasks involves creating the set of required transformation operations and then applying the created operations on data and algorithmically determining the most useful augmentations using various optimization techniques. We briefly discuss common approaches to creating augmentations as well as techniques for optimizing the augmentation strategies.

4.1.1.1. Analytical methods. The most basic approach to automating data augmentation is the use of semi-automated or analytical methods to create augmented data. The analytical method involves defining a set of explicit mathematical relations or heuristics to perform required transformations on the training data. These relations can be domain-specific and depend on the characteristics of the given data type. This class of methods generally perform standard augmentation functions such as rotation, shearing, flipping, and color jittering. Many works (e.g. Refs. [138–142]) involve defining different types of basic transformations functions to apply, as well as specifying the degree or magnitudes of these operations within an applicable range. While simple, the approaches can be effective for certain tasks [143,144], although they may not be able to account for unknown transformations.

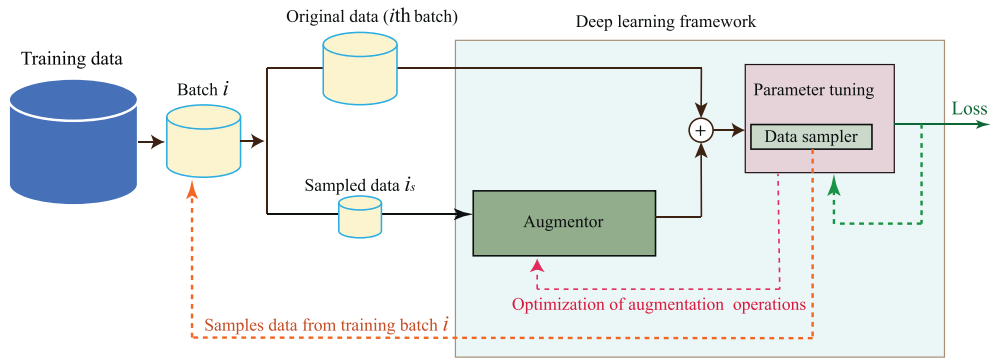


Fig. 13. General scheme and procedure for automated data augmentation. A portion of the training set is sampled by a specified algorithm, a so-called data sampler, for augmentation. The selected sample undergoes a series of transformations by an augmentor sub-module. The augmented sample is then added to the original data batch i for training. The training loss is fed to both the sampler and augmentor to fine-tune their performance. Over time, this results in better sampling and augmentations, leading to overall improved performance.

4.1.1.2. Deep-learned transformations. Some recent works [46,145–148] have suggested leveraging learned mechanisms in favor of predefined augmentation operations. The goal is to allow augmentation operations to be learned from input data rather than relying on explicitly formulated relations. The basic idea is to implement data transformation sub-modules within deep learning pipelines whose parameters and hyperparameters can be learned and applied on input data in the training process. This method offers a higher degree of automation than approaches based on explicit analytical formulations. It also allows arbitrary or unknown augmentation styles to be achieved. A common technique widely employed for learning transformations is the spatial transformation network (STN) [45] (Fig. 14(a)).

4.1.1.3. Generative machine learning techniques. Another common way to automate the construction of data augmentation operations is based on generative modeling [145,149–151]. This approach creates transformed samples without explicitly requiring analytical formulas, or even requiring implicit transformation parameters to be learned. The basic idea is to first learn the distribution of real data by training a discriminator-generator pair on the original dataset. Knowledge about the distribution can then help to generate new but slightly varied samples that simulate data diversity. Thus, in the automated data processing pipeline, the generative model is employed as an intermediate data transformation unit. Its parameters are learned jointly with the CNN model in a bi-level optimization scheme (see Fig. 14(b)).

Generative machine learning approaches based on GANs are particularly common in computer vision domains, where they enable realistic image samples to be generated with desired visual features. This family of methods is also useful in applications where it is necessary to transfer knowledge from an existing domain to a different domain (e.g., in Ref. [152]). Generative methods are also used to align the distributions of data from different sources, enabling deep learning models to perform better when applied to new data that may come from a slightly different distribution. However, despite the usefulness of the approach, generative models themselves require careful tuning, thereby greatly increasing the for human supervision and limiting the full benefits of automation.

4.1.2. Optimization of augmentation strategies

Optimization techniques are typically employed to search for the best augmentation strategy. The search task involves finding not only relevant transformations but also the optimum levels of transformations. For image augmentation these levels may be rotation angles, translation offsets, saturation values, etc.

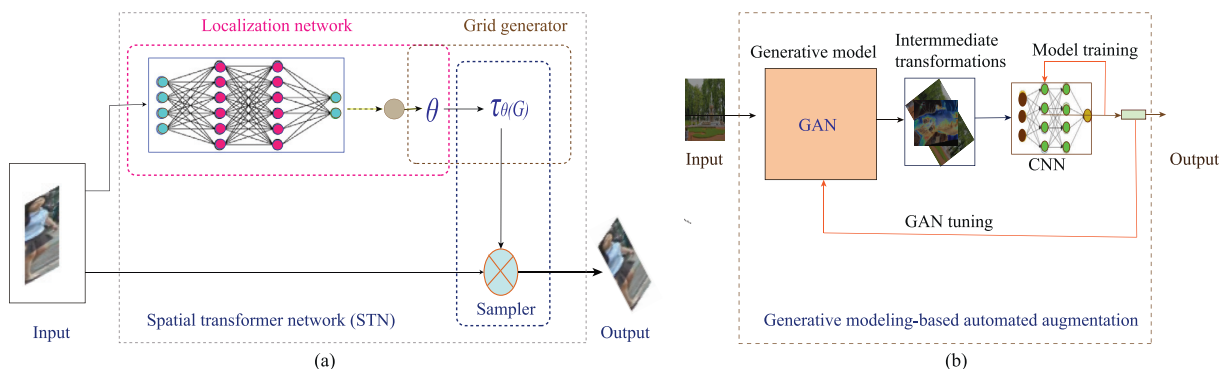


Fig. 14. Approaches to generating basic transformations within automated data augmentation frameworks: (a) Learned transformation using spatial transformation network (STN) [45], and (b) generative-modeling-based synthesis of transformed image data.

Because the search space is inherently complex and discontinuous, many works utilize black-box optimization techniques such as reinforcement learning (e.g., in Refs. [138,140,142,153,154]), Bayesian optimization (e.g., in Refs. [142,155,156]) and evolutionary computation algorithms (e.g., in Refs. [157–160]) to search for good augmentation strategies. These techniques have produced impressive results but are noticeably slow when dealing with very large data.

To overcome this limitation, approaches for optimizing the discrete augmentation search space using the concepts of approximation gradients [161] have also been proposed [149,162–164]. The use of these concepts makes it possible to develop methods for solving optimization problems for discontinuous functions present in the search space. Approaches based on approximate techniques are generally more efficient than black-box search techniques.

Alternative search spaces providing the ability to find effective augmentation policies without extensive search have also been considered [165,166]. In some cases (e.g., Ref. [167]) these approaches eliminate the need for the search stage altogether. For instance, researchers in Ref. [166] contend that it is not necessary to conduct exhaustive search for extensive range of possible augmentations and hyperparameters in a combinatorial manner and, instead, propose to reduce the search space to a linear search space of uniform probabilities and augmentation parameters that can be traversed by a simple grid search. The approaches, despite the significant reduction in compute time, have shown competitive performance. However, there seems to be little room to further extend their predictive performance.

4.2. Data synthesis by large language models (LLMs) and diffusion models

The automated data augmentation methods covered in Section 4.1.1 utilize generative modeling techniques to perform intermediate data processing in AutoML pipelines. While generative methods can be used to perform data transformation operations in that manner, approaches based on variational autoencoders [168], generative adversarial networks [169], diffusion models [170], autoregressive models [171,172] and large language models (LLMs) [173] are commonly designed to directly synthesize data by themselves, thereby bypassing all intermediate processing steps. Recent advances in deep generative AI-based data synthesis methods, especially diffusion models and large language models (LLMs), have enabled the possibility of generating clean data from scratch or from noisy data in an end-to-end manner. Data generated this way can be used to augment existing data (e.g., in Refs. [174–176]) or completely replace natural data in situations where datasets are inadequate or are inaccessible for training machine learning models (e.g., in Refs. [177,178]).

Large language models are a special type of deep learning systems based on the concept of the transformer [179] and are primarily designed to perform NLP tasks. On the other hand, diffusion models are a particularly versatile class of generative methods that synthesize data in a similar way to generative adversarial networks – a technique that allows deep neural networks to generate data using random noise. They are based on nonequilibrium thermodynamics [180]. Diffusion models are realized by first defining a Markov chain of diffusion steps that gradually increase the random noise component to input data, and then implement a reverse diffusion process to re-create the desired data samples from the input noise (Fig. 15).

Generative methods based on LLMs [174,181] and diffusion models [182–184] have become very useful in augmenting data for natural language processing and computer vision applications, respectively. In NLP applications, additional text data commonly generated by prompting LLMs such as ChatGPT [185], LLaMA [186] and BERT [187] to complete input sentences with missing words or phrases so that different variations of the original sentences could be created. For instance, Ubani et al. in Ref. [188] propose to manipulate text data with the help of ChatGPT to produce augmented data for training deep learning networks for NLP-based tasks by giving appropriate prompts to generate syntactically different variations of the original sentences. In computer vision applications, generative AI-based data augmentation typically involves manipulating existing images to generate novel styles, poses, background contexts and views. DiffusionCLIP [184] introduces an approach to perform various image transformations using text-guided prompts with a diffusion model developed on the basis of CLIP [189]. The approach achieved photorealistic stylizations of different images in a wide range of novel contexts. Fig. 16 shows the result of various image stylizations methods using DiffusionCLIP [184] and two other text-based generative data manipulation models – StyleCLIP [190] and StyleGAN-NADA [191].

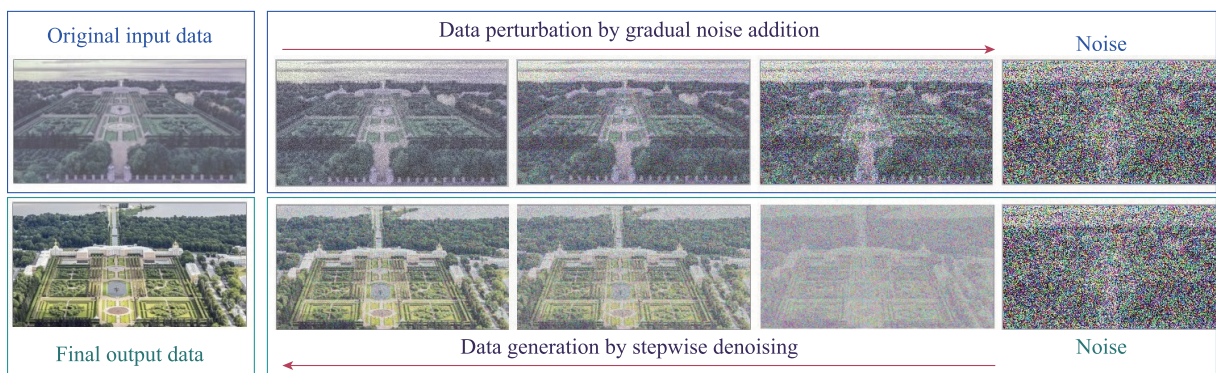


Fig. 15. The idea of diffusion models is to learn to generate data by gradually adding random perturbations to the input data, and then perform the reverse process of denoising the data until a clean data output is achieved.

While diffusion models and LLMs can both perform data processing – e.g., text manipulation [188] and image quality enhancement (super-resolution) [182,192], denoising [183] and styling [193,194], their most powerful use-case has been the task of automatically generating high quality data from “scratch” [195,196]. Modern generative AI techniques based on diffusion models can be used to synthesize many different types of data – including images [197,198], videos [199–201], text [188,202], audio [203,204], time series [205] and tabular data [206–208] – using only text prompts without additional input data.

Language models, when trained on pairs of data (e.g., images) and corresponding descriptive texts (e.g., captions), can learn the association between the textual descriptions and the underlying data. The trained model can then be used to generate desired images based on descriptive text inputs. In this regard, large language models and diffusion models play a complementary role in the data generation process. Indeed, the popularity of large-scale diffusion models has been driven largely by the development of powerful generative tools that generate data based on intuitive text prompts enabled by LLM techniques. Some of the most prominent data generation models in this category include Dalle-2 [209], stable diffusion [210], Glide [193] and Imagen [211]. Fig. 17 depicts photorealistic images generated by different diffusion models based on user’s supplied prompts.

4.3. Dataset distillation methods

While the approaches discussed in Section 4.1 rely on generating extra data to improve machine learning models, an alternative approach, known as dataset distillation [214,215], seeks to select an informative subset from a large-scale training dataset that retains the important properties and, importantly, the generalization performance of the original dataset. That is, the aim is to produce a leaner dataset that still performs satisfactorily on the target task. The approach can also be used to distill data labels from a large set of labels (e.g., in Refs. [216,217]) or from noisy labels (e.g., in Ref. [218]). While traditional approaches to dataset distillation rely on manually engineered procedures, some recent works [219,220] have proposed mechanisms for automating the process. Although these approaches are well motivated, and can potentially lead to high performing but leaner deep learning models, at present their level of automation is limited. Consequently, their popularity is relatively low.

4.4. Performance of automated data augmentation methods

Results from several studies show that models trained on data generated by automated techniques outperform those trained on data generated using manual approaches. In Table 3 we compare the performance of ten state-of-the-art automated data augmentation techniques and ten of the best traditional augmentation methods across various datasets (CIFAR-10 [221], CIFAR-100 [221] and ImageNet [222]). All results are obtained using wide ResNet (WRN-28-10) [223] backbone trained until convergence – roughly about the same number of epochs (300). This provides a fair ground for comparison of the different methods. In Fig. 18 we summarize the comparative performance by averaging the classification scores as well as the percentage improvement over the baseline (i.e., models trained without augmentation). These results convincingly demonstrate the effectiveness of automated data augmentation approaches (see Table 4).

While the performance of automated augmentation techniques is in no doubt, an important challenge is the enormous computational resources typically required for training these models. Also, since the augmentation process is not guided by human intuition, some augmentation operations may unduly alter the original semantic meaning of the data, which may not be observed at training time and can later lead to catastrophic failure. For instance, modifying the label of an image or applying overly aggressive transformations in such a way that they transform its semantic meaning can result in erroneous training signals with potentially harmful implications.

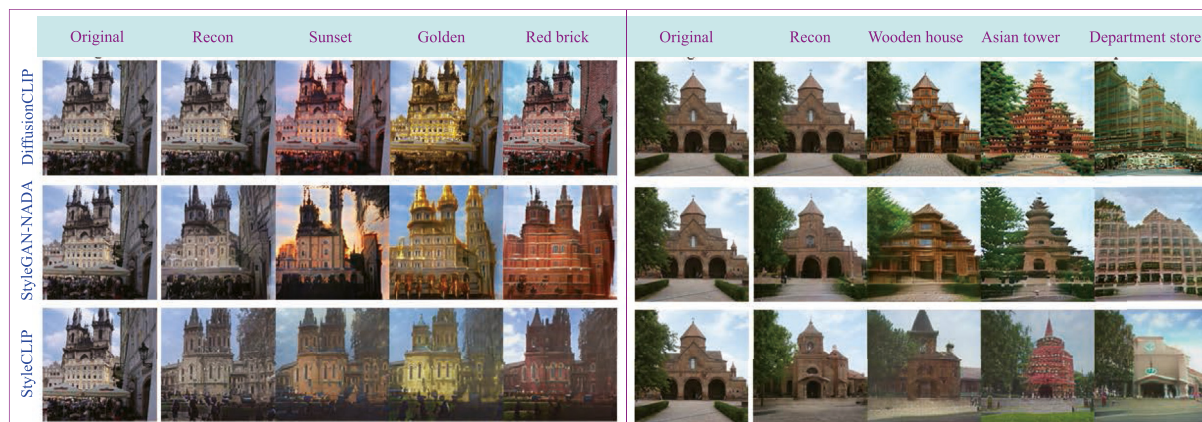


Fig. 16. Visual comparison of results of different generative AI-based image stylization methods: DiffusionCLIP [184], StyleGAN-NADA [191] and StyleCLIP [190]. The models attempt to style a church building with respect to different visual contexts. Image is from Ref. [184]. We have applied some minor changes in the text fonts to make it clearer.

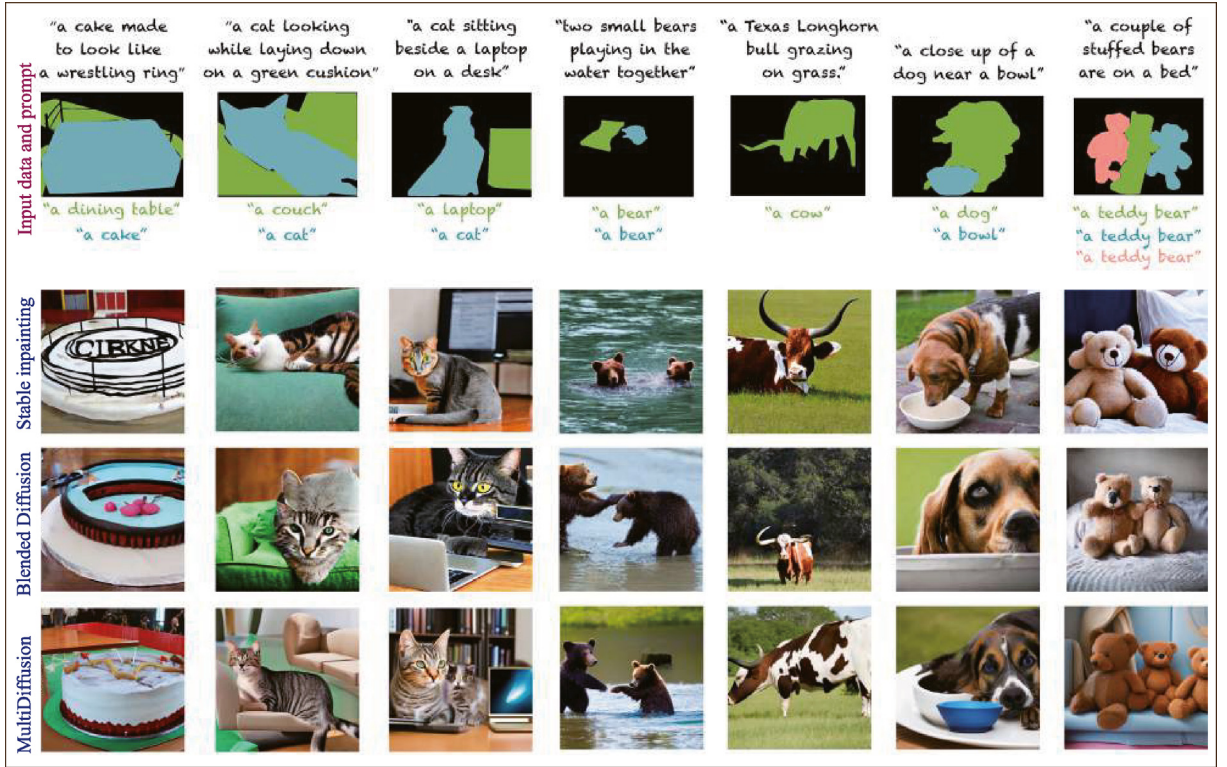


Fig. 17. Examples of images generated by state-of-the-art diffusion models: Stable inpainting [210], Blended Diffusion [212] and MultiDiffusion [213]. The images have been generated using text prompts together with simple foreground images as inputs. Image is from MultiDiffusion [213].

Table 3

Performance results of automated and traditional data augmentation methods on classification tasks. The values represent percentage accuracies on CIFAR-10, CIFAR-100 and ImageNet datasets. **Bold** is highest score, *italic* is second highest, and underline is third highest.

Method	Classification accuracy on various datasets			
	CIFAR-10	CIFAR-100	ImageNet (top 1)	ImageNet (top 5)
Baseline	96.1	81.2	76.3	92.1
Automated data augmentation methods				
TrivialAA [224]	97.5	<u>84.3</u>	78.1	93.9
DivAug [225]	98.1	84.2	78.0	–
AWS [153]	98.0	84.7	<u>79.4</u>	94.5
MetaAug [226]	97.7	83.8	<u>79.7</u>	94.6
PBA [159]	97.4	83.3	77.2	93.4
AdvAA [149]	98.1	84.5	79.9	94.5
A2-Aug [227]	98.0	85.2	79.2	–
FastAA [142]	97.3	82.8	77.6	93.7
KeepAA [228]	98.7	–	78.0	93.0
DeepAA [229]	97.4	83.7	78.3	–
Traditional data augmentation methods				
StochasticDepth [230]	–	–	77.5	93.7
RE [231]	96.9	82.3	77.3	93.3
RICAP [232]	97.2	82.0	78.6	–
SalencyMix [233]	96.0	80.5	78.7	–
GridMask [144]	97.2	83.4	77.9	–
SmoothMix [234]	–	–	77.7	93.7
Manifold Mixup [235]	–	–	77.5	93.8
CutOut [236]	96.9	81.6	77.1	–
Mixup [237]	97.3	82.1	77.9	93.9
FMix [238]	96.4	82.0	77.7	–

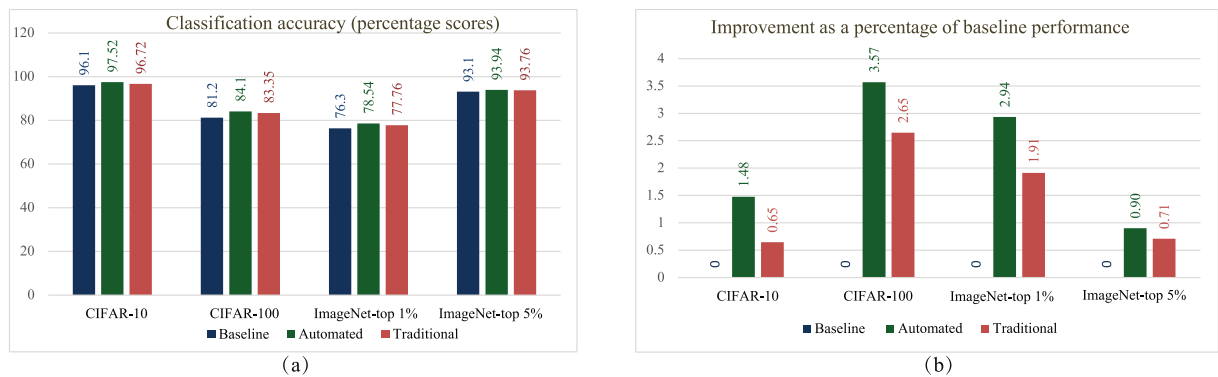


Fig. 18. Comparison of automated data augmentation and traditional methods. Results for each class of approaches are based on 10 state-of-the-art techniques.

5. Automated feature engineering

Feature engineering is a set of low-level data processing tasks that aim to increase the representativeness of input data by indirectly manipulating extracted feature vectors in deep learning layers. The goal is to obtain the minimum subset of features that can adequately encode all necessary information about the data without loss of predictive performance.

Traditional approaches to feature engineering [239] rely on assumptions about the underlying data and feature statistics to enrich feature representation. They typically employ analytical formulations of feature transformation operations to enhance the representativeness of extracted features. These techniques, apart from being enormously computationally expensive and heavily dependent on domain expertise, do not often lead to an optimal solution. For this reason, automated machine learning approach is considered a more viable option.

5.1. General concept of automated feature engineering

The basic idea of automated feature engineering is to create a large search space of all possible feature processing operations and then apply optimization techniques to search for an optimum feature engineering strategy. An optimum strategy is the best possible combination of the elementary operations (i.e., the set of operations that result in the best performance for the specific dataset and task).

Formally, for a target task T_{sk} , the feature engineering problem for a given dataset D consisting of a set of features $F \in f_1, f_2, f_3, \dots, f_n$ involves defining and applying a set of suitable transformations $T \in t_1, t_2, t_3, \dots, t_n$ on the original feature set F to generate additional feature set Φ and, with the help of a search mechanism, finding the best possible subset of features over the feature space $\Omega = F + \Phi$ that maximize performance on the target task T_{sk} .

Because of the need to run and evaluate an unusually large number of operations on the given data, and owing to complex interactions among these operations, it is often challenging to obtain a good solution in reasonable time, especially when dealing with large datasets. Instead of dealing with the entire search space at a go, it is often possible to create smaller search spaces which can be evaluated first before testing various combinations of the best performing strategies. A number of recent works, for example [240,241], have tended to favor such an approach. For example, VolcanoML, introduced by Li et al. in Ref. [240], aims to alleviate the challenge of computational burden of large search spaces by proposing a more scalable search space design that allows users to construct custom execution plans for their specific AutoML tasks.

Instead of attempting to search on the entire search space for the optimum solution, the authors suggest decomposing the search space into smaller components and then learn a strategy to select the most appropriate ones for a given task and dataset. To achieve this, they developed a high-level structural representation of the search space consisting of small atomic subspaces that can be used as

Table 4
Degrees of automation of feature engineering tasks.

Method	Description and main characteristics	Example works
Analytical	Basic and rigid automation based on fixed analytical formulas to separately extract, synthesize and select useful features.	[260–262]
DL-based	Deep learning-based approach to generate and select useful features given a set of raw data in an end-to-end manner.	[270,276,278]
Fully automated	AutoML and NAS-based approaches to holistically generate useful features directly from raw data, together with model selection and hyperparameter optimization.	[274,285,286]

building blocks for larger search spaces. The small search spaces form a tree structure and can be combined in different permutations. This provides a more flexible mechanism that allows selecting a final search space configuration based on computational budget and training time constraints. The authors demonstrated the ability of the approach to compose more efficient AutoML pipelines than with traditional search space configurations. NAS-based feature engineering approaches [59,60] differ from conventional AutoML methods in the sense that they do not only optimize parameters and hyperparameters for predefined model architectures but are also concerned with creating, training and validating new neural network topologies based on the input data. The goal is to find fundamentally new model structures which perform better than manually designed architectures. In this case, for the construction of the search space, dynamic feature processing operations are defined together with elementary building blocks for generating surrogate neural architectures. Search techniques are applied to jointly optimize the resulting neural architecture and the feature processing operations.

While a wide variety of AutoML tools (e.g., Refs. [97,242,243]) exist for generic data processing, dedicated AutoML-based feature engineering utilities and software packages [244–247] that perform automated feature extraction as their core functions have also been introduced. A large number of these tools are aimed at processing features from time series and tabular data. Tools for time series feature processing include TSFEL [248], tsfresh [246], tsfeaturex [247]. Featuretools [244] is aimed at extracting useful features from hierarchical and relational databases. This is achieved using a so-called deep feature synthesis technique that mines and combines features from multiple database tables at different hierarchical levels.

5.2. Common feature engineering tasks and approaches for automation

In the literature, three main feature engineering tasks can be distinguished [249]: feature extraction, feature synthesis (often referred to as feature construction in many literature sources (e.g., Refs. [250–253])), and feature selection. There is no consistent definition of these terms. Also, the scope and interrelationships among these three processes often vary widely. While many authors treat all three tasks as distinct processes, some authors (e.g., Ref. [254]) consider feature synthesis and selection to be aspects of the same process – feature extraction. And while authors such as Horn et al. [245] treat the extraction and synthesis subtasks as the same task, other literature sources (e.g., Ref. [255]) regard feature construction and extraction as a single process distinct from feature selection. In this work, we use the following designation:

- Feature extraction – relates to the process of manipulating input data or intermediate feature sets to obtain more generalizable, robust, representative and compact features for the particular task.
- Feature synthesis – also commonly termed feature construction [254], is the creation of new feature sets based on the available data or intermediate features to better characterize the dataset and enhance the performance of the resulting model.
- Feature selection – is the process of choosing and further processing only the best set of features that result in noticeable performance improvement.

Despite this clear delineation, the boundaries between these processes can be very blur, and in some cases, the terms can be used interchangeably. Moreover, they are often applied simultaneously in a given machine learning task.

5.2.1. Automated feature extraction

For many large-scale machine learning problems, feature extraction is the first and most important stage in the feature engineering process. It allows to obtain more compact and representative feature set from raw data. Since the input data in these machine learning application settings are usually characterized by a large number of variables, it is generally useful to simplify feature representation by maintaining only the important variables that account for performance. The process of feature extraction results in new features with reduced computational and memory footprint but with superior characteristics necessary to encode the underlying data. Feature extraction can also enhance interpretability since the reduced set generally contains only relevant features useful for the given task. The extracted features are generally obtained by applying various linear combinations of the original ones. This allows the dimensionality of identified set of features to be reduced while maintaining the information encoded by the original dataset. Traditional feature extraction techniques rely on algorithms such as principal component analysis (PCA) [256,257], independent component analysis (ICA) [258,259], linear discriminant analysis (LDA) [260] and locally linear embedding (LLE) [261,262]. In AutoML, feature extraction is usually achieved by using simplified analytical functions based on some variations of these standard algorithms to perform basic feature extraction operations (e.g., in Refs. [263,264]) within a bi-level optimization scheme, where the best performing operations and the corresponding hyperparameter values are automatically determined. Because of the potential for excessive explosion of features, most of these approaches fuse the extraction and selection stages as a single process. Yang et al. [265], for instance, employ simple mathematical operations to first extract data and then utilize Boruta algorithm proposed in Ref. [264] to select the most effective ones.

Some new approaches [266–268], instead of focusing solely on utilizing analytical functions to extract features from data, propose to rather learn effective network architectures for applying the necessary feature extraction operations. These approaches are based on the concept of neural architecture search (NAS). For instance, meta-learning for tabular data (MetaBu) [267] employs a NAS-based technique to automatically extract useful features by finding the best machine learning model and the corresponding hyperparameter values. Firstly, the authors manually construct a large number of meta-features (135 in all). They then employ optimal transport (OT) [269] technique to perform linear transformation on these manually designed meta-features in order to generate new features that are linear combinations of the basic meta-features. Finally, AutoML technique is used to find the best model configuration and hyperparameter values that results in the best features based on performance as measured on a validation set. Also in Ref. [266], Lopes et al. propose an automatic feature extraction method for sentiment analysis problem based on neural architecture search (see Fig. 19). The authors

construct diverse machine learning models, each of which independently performs feature extraction in different ways. They then use random search techniques to find the best performing model based on the performance scores on validation data. The approach uses a three-stage structure to accomplish the task of predicting sentiments based on enhanced features extracted from input data. The model takes as input images obtained from the internet and their corresponding textual descriptions. After an initial preprocessing stage, image and text data are classified separately by predefined models in the search space. The image and textual features are then fused together to obtain a composite feature representation before categorizing the underlying inputs into one of the following: positive, neutral and negative. The fusion process, in this formulation, is essentially a feature extraction mechanism that produces useful features from combined text and image data (see Fig. 20).

Transferable AutoML (Tr-AutoML) [268] utilizes a meta-feature extraction method that allows useful features learned from previously trained, multiple AutoML pipelines to be mined, aggregated and then transferred to new tasks. The approach combines meta-learning and architecture search for feature extraction from multiple but related datasets. It relies on transferring already-learned architectures that previously showed good results in performing feature extraction functions on new datasets and tasks. This significantly reduces the search space since a common architecture can serve for multiple feature extraction tasks.

5.2.2. Automated feature synthesis

The feature synthesis or construction stage creates additional features to further enhance the performance of the resulting model. The synthesis process generally exploits the statistical distribution of features extracted from the data to generate new, complementary

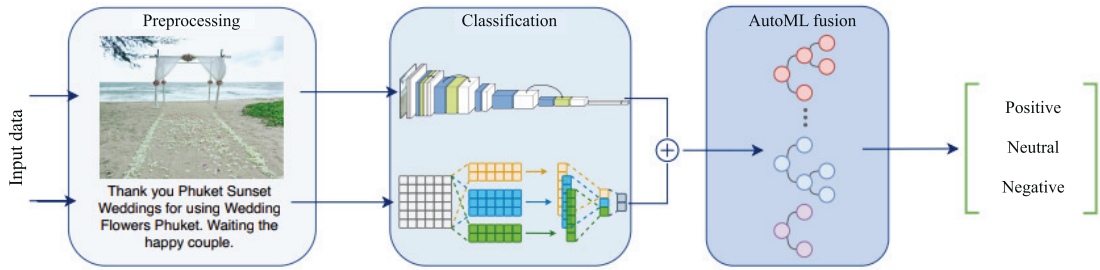


Fig. 19. Functional diagram of the automated feature extraction techniques proposed in Ref. [266]. The approach employs a three-stage neural network framework to perform sentiment analysis by first extracting and separating online images and their corresponding labels. Different models are then used to perform preprocessing before sentiment classification, also by independent sub-models. Different combinations of these models are then evaluated and the best settings selected according to the performance on the target task.

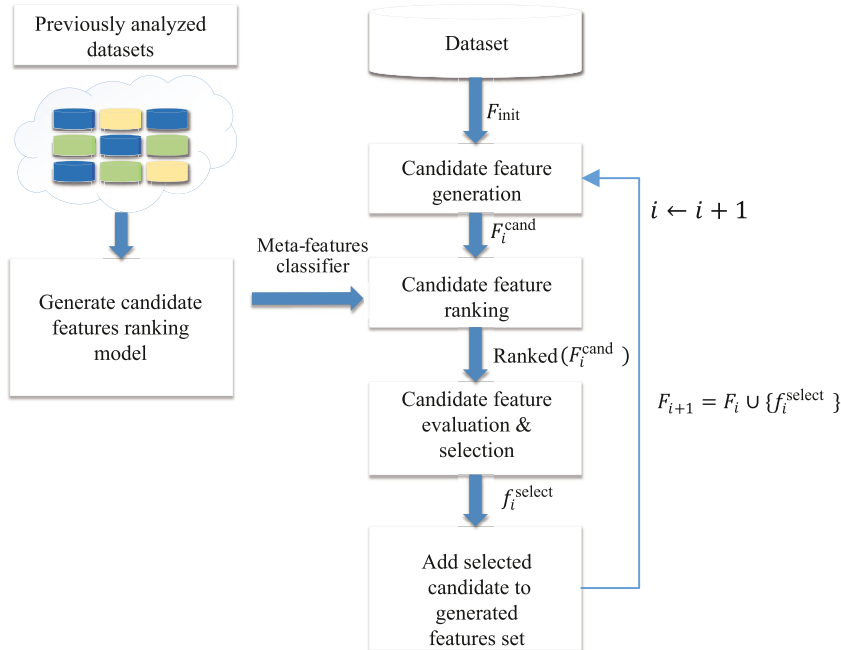


Fig. 20. Basic principle of feature synthesis and selection approach in ExploreKit [270]. This technique relies on drastically increasing the feature set by generating additional features and then searching for a small subset of features that yield the best performance.

features. This is useful in cases where extracted features are insufficient to provide an adequate representation of the input data. Typically, feature synthesis is achieved by performing manipulations on accessible features with the help of predefined transformation operations. Apart from generating new features, feature synthesis models [40,253,271] typically incorporate mechanisms to perform feature selection as not all artificially constructed features would be useful for the given task. However, these approaches, as discussed in this subsection, differ from feature selection methods as their main focus is in generating good features as oppose to merely selecting useful features from existing set extracted directly from input data.

Common techniques used to accomplish this task include feature transformation, interpolation, averaging and mixing. Traditional approaches to solving this problem requires repetitive trial and error using different combinations of operations with different hyperparameter settings. Automated feature synthesis [38,245,272–274] alleviates the laborious work involved in this procedure and is particularly valuable in situations where the scarcity of data restricts the adequacy of features that are mined through feature extraction process. For a given feature set derived from a training data, the automated feature synthesis task seeks to determine and optimize the best combination of basic feature-level processing operations to apply and their associated hyperparameters to use in order to generate new features that, when incorporated into the original feature set, maximize the performance of the model on the target data. Some automated feature synthesis methods (e.g., Refs. [274–276]) apply all operations at once to expand feature set and then search for the best features within the expanded set. Kanter and Veeramachaneni [274], for instance, simultaneously apply all transformation operations in the search space and then utilize different classification sub-models to select the most effective features. The main drawback of this approach is the high memory requirements that results from the need to store a large pool of features.

Another line of works (e.g., Refs. [270,273]), instead of aggressively expanding the search space, iteratively apply small number of operations to expand, test and select small subsets of features in batches. While this approach overcomes the high memory demand of techniques such as [274–276], testing for effective features at each iteration can lead to expensive computational overheads. Moreover, useful features that could form the basis to further generate more effective features can be lost in the selection process if they do not directly yield good performance. Thus, techniques based on this concept can fail to achieve optimum feature choice. In order to reduce the high computational cost of evaluating all candidate features, more recent approaches [271,277] suggest using meta-learning techniques to predict effective transformations that generate the best features and to prioritize them for the synthesis and evaluation stages.

Different from many of the feature synthesis paradigms [271,277] that employ a bi-level optimization mechanism to generate useful features, some approaches [270,276,278] utilize more conventional deep neural networks to accomplish feature engineering. These models learn to predict the effectiveness of different transformation operations through previous training on other datasets. Some works such as [59] have demonstrated the effectiveness of NAS techniques to synthesize useful features. These works construct the search space consisting of basic neural models and then employ neural architecture search approaches to find the best models capable of applying feature transformations to generate new features.

5.2.3. Automated feature selection

The overall goal of feature selection is to find a minimal set of features that provides the best predictive performance using the least possible computational resources. The process, thus, effectively simplifies the resulting model by eliminating irrelevant features. The overall performance can be drastically enhanced since irrelevant features often harm performance. In addition, feature selection helps to determine the effectiveness of different feature sets on the ML model and hence can provide additional insights about the important factors that influence performance [57]. Moreover, the reduction of features by selecting only the most useful ones leads to a more simplified and interpretable models. Feature selection, thus, performs five important functions:

- Model size reduction by the elimination of redundant and less useful features, leading to a reduction in computational and memory demand.
- Predictive performance improvement as a result of the elimination of potentially detrimental features.
- Model structure simplification as a result of reduced feature set.
- Knowledge discovery (i.e., provides insights into the internal representation) by virtue of the enhanced representativeness of features.
- Interpretability of data and resulting machine learning model owing to model simplicity and representative quality of features.

A major difficulty with feature selection is the fact that it is often not straightforward to determine the minimum subset of features that can provide optimum performance. A set of features that may not be useful when applied alone may turn out to be the optimal set when used in combination with another set of features. This makes the feature selection process an inherently combinatorial problem. Moreover, there may be multiple sets of features that result in optimal performance. Feature selection approaches aimed at knowledge discovery [57,279–282] often need to identify all such minimal feature sets. However, in some scenarios it may be necessary to identify all such sets of features. For example, feature selection for knowledge discovery applications generally aims to identify all useful sets of features. On the other hand, for problems such as regression, classification and time series forecasting, it is important to be able to quickly find only one out of the most useful sets of minimal features that can comprehensively encode the given data.

Typically, to carry out feature selection, a subset of features is drawn from the set of learned features based on a given criterion and then evaluated to assess the efficacy of the chosen subset on the end task. The process is repeated until a small subset of features is

obtained that can provide optimal performance. For automated feature selection, like in most automated machine learning tasks, optimization methods are usually employed to find the optimum feature sets. ExploreKit [270] uses a learned mechanism that employs a ranking classifier to identify useful features for further exploitation. In conventional deep learning models, regularization techniques based on feature dropout methods [283] are a common way to eliminate irrelevant features and retain more useful ones. But hyperparameter selection for these dropout mechanisms is generally accomplished manually, making it challenging to find the optimum set of features to drop in order to retain only the most effective set. Inspired by these feature-level information dropout regularization techniques, some recent feature engineering methods [283,284] are aimed at learning optimum dropout patterns based on automated hyperparameter tuning techniques. These approaches can be considered as an alternative variant of automated feature selection methods since, by eliminating irrelevant features, they practically allow only the most useful subset of features to be selected for the given task. AutoDropout [283], proposed by Pham and Le, learns effective feature-level information dropout using AutoML-based hyperparameter tuning. To accomplish this, the authors created primitive dropout patterns using adjustable hyperparameters that define important properties of the dropout pattern, including the dropout size, stride, and the number of times to repeat the particular pattern. In addition, hyperparameters describing geometric operations, viz. rotations and shearing, as well as additional flags (binary variables) that determine whether or not dropout patterns are shared across channels or are applied on the residual branches of ResNet, are also specified. In the optimization stage, optimum settings for variables describing these hyperparameters are then automatically learned through feedback of the loss signal.

5.3. End-to-end automated feature engineering

While individual feature processing tasks can be automated separately, many modern automated feature processing approaches [134,274,275,285–287] rather focus on performing the full range of feature engineering tasks (i.e., feature extraction, synthesis of additional features, and selection of useful features) in a holistic and end-to-end manner. In these works, the performance of the various subtasks may not be distinctive. Most approaches generally follow the so-called expansion-reduction paradigm, where various transformations are applied to the extracted features to create additional features that are then evaluated together with the original features and the best set of features selected. The idea is to utilize a composition of primitive transformation operations within a machine learning pipeline to generate diverse features which are jointly optimized with model hyperparameters in the training process using various search mechanisms. For instance, scalable automatic feature engineering (SAFE) [40] (Fig. 21) employs a search strategy based on XBoost [288] to find the best combinations of useful features. The search is conditioned on maximizing information gain. Evolutionary automated feature engineering (EAAFE) [285] and ring theory based harmony search (RTHS) [289] employ evolutionary computational algorithms to find useful features. The use of reinforcement learning approaches is also common [276].

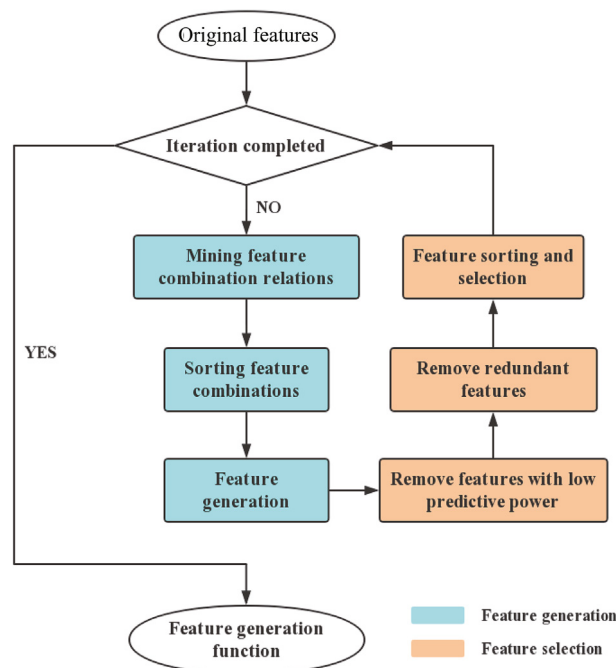


Fig. 21. Scalable automatic feature engineering (SAFE) [40] exploits the relationships among the initial feature set to obtain a reduced feature set which is then expanded and further filtered according to the performance of the various combinations of features generated.

Neural feature search (NFS) [59] uses a reinforcement learning-based RNN controllers to learn effective transformation policies for generating useful features. Differentiable automated feature engineering (DIFER) [287] formulates the automated feature engineering task as a differentiable problem by mapping discrete features into a continuous vector space with the help of a so-called encoder-predictor-decoder-based feature optimizer and then finds effective features by gradient ascent. The general architecture of their framework is shown in Fig. 22. The approach uses random sampling to generate initial population of features which are further transformed by applying a sequence of transformation operations.

5.4. Automated feature engineering based on NAS frameworks

Automated feature engineering methods [59,290] have also been proposed that employ neural architecture search (NAS) techniques for feature selection. For instance, Chen et al. in Ref. [290] employ a NAS mechanism to search for and select the most effective set of features for a multimodal person re-identification system based on RGB and infrared images. Their technique incorporates a two-level search space construction, where spatial and channel-wise features are independently selected. Neural feature search (NFS) [59] proposes a method that utilizes NAS technique to accomplish feature engineering (actually extraction and selection). The authors utilized reinforcement learning algorithm to train a recurrent neural network-based controller to generate useful transformations. The technique first aims to find a neural network model capable of performing the most effective feature transformations. The resulting features are selected and then enhanced by applying subsequent feature engineering operations. This process alleviates the problem of feature explosion where redundant and less useful features are spawn that adversely affect predictive performance while at the same time contributing to computational complexity and memory demands. Similarly, Rakotoarison et al. in Ref. [267] propose to learn the best values for model hyperparameters in order to optimize the selection of manually composed meta-features. The authors show that meta-features learned using AutoML techniques can outperform state-of-the-art feature extraction methods based on traditional machine learning approaches.

5.5. Performance of automated feature engineering methods

In this section, we compare performance results of automated and traditional feature engineering techniques commonly employed in practice and in scientific research. The first set of results (shown in Table 5) show classification accuracies (F-1 scores [291]) obtained from experimental evaluations by several authors – specifically, [59,278,287,292]. The feature engineering methods compared are the following. Raw – this entails using the original feature set without any further feature processing. Random (Ran) involves iteratively running through the feature set and applying a random set of transformation operations (or no transformation) on the features in each iteration. In the case of ME, the feature engineering algorithm selects only one transformation operation – the most effective operation – for each sample set. Brute-force (BF) applies all transformation operations on the entire feature set and then finds the best subset of features from the expanded set. In the literature, this approach is commonly referred to as Expansion-Reduction. The rest of the methods are LFE [278], NFS [59], autofeat [245] and DIFER [287]. We selected 23 datasets where performance results are widely available for many of the aforementioned automated feature engineering approaches. These datasets are from UCI [293,294] and OpenML [295] repositories. For more detailed information about their basic characteristics, the reader may refer to their repositories [293] and GitHub pages [294,296]. Random Forest model is employed as the base model for the classification tasks using ten-fold cross validation.

From the Table 5, it can be seen that the automated feature engineering techniques clearly outperform basic methods based on traditional feature manipulation paradigms. However, it is worth noting that the high performance of automated methods is not consistent for all datasets. Particularly, spambase, autos and convex show little or no improvements with automated feature engineering methods.

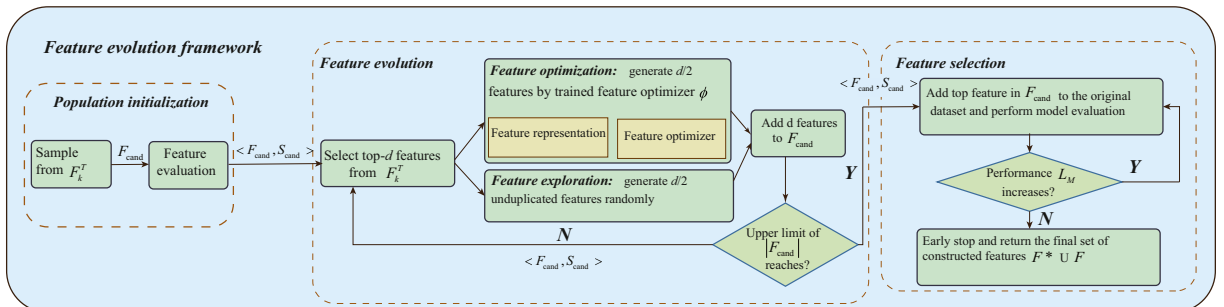


Fig. 22. The basic design of differentiable automated feature engineering (DIFER) [287] is made up of three main components: (1) population initialization module which generates feature by sampling randomly from the initial set, (2) feature evolution module for growing and introducing diversity into the feature set, and (3) feature selection which aims to reduce the overall feature count by selecting only the best set of features.

Table 5

Performance of state-of-the-art feature engineering methods. The results are classification accuracies (F-1 scores) on selected datasets from the UCI [293,294] and OpenML [295] repositories. **Bold** is highest score, *italic* is second highest, and underline is third highest. NB: Feat. – number of features, Inst. – number of instances or samples.

Dataset	Feat.	Inst.	Raw	ME	BF	Ran	NFS	Autofeat	LFE	DIFER
AP-omentum-lung	10,936	203	0.883	0.915	<u>0.925</u>	0.908	0.981	–	<i>0.929</i>	–
AP-omentum-ovary	10,936	275	0.724	0.775	0.801	0.745	0.873	–	<u>0.811</u>	<i>0.8724</i>
Autos	48	4562	<i>0.946</i>	0.95	<u>0.944</u>	0.929	–	–	0.96	–
Balance-scale	8	369	0.884	<i>0.916</i>	<u>0.892</u>	0.881	–	–	0.919	–
Convex	784	50,000	<i>0.82</i>	0.5	0.913	0.5	–	–	<u>0.819</u>	–
Credit-a	6	690	0.753	0.647	0.521	0.643	<u>0.803</u>	<i>0.8391</i>	0.771	0.8826
Dbworld-bodies	2	100	0.93	<i>0.939</i>	<u>0.927</u>	0.909	–	–	0.961	–
Diabetes	8	768	<u>0.745</u>	0.694	0.737	0.719	0.786	–	<i>0.762</i>	–
Fertility	9	100	0.854	0.872	0.861	0.832	0.913	0.7900	<u>0.873</u>	<i>0.9098</i>
Gisette	5000	2100	0.941	0.601	0.741	0.855	<i>0.959</i>	–	<u>0.942</u>	0.9635
Hepatitis	6	155	0.747	0.736	0.753	0.727	0.905	0.7677	<u>0.807</u>	<i>0.8339</i>
Higgs-Boson-subset	28	5000	<u>0.676</u>	0.584	0.661	0.663	0.827	–	0.827	–
Ionosphere	34	351	0.931	0.918	0.912	0.907	<i>0.972</i>	0.9117	<u>0.932</u>	0.9770
Labor	8	57	<u>0.856</u>	0.827	0.855	0.806	0.960	–	<i>0.896</i>	–
Lymph	10,936	138	<u>0.673</u>	0.664	0.534	0.666	0.987	–	<i>0.757</i>	–
Madelon	500	780	<u>0.612</u>	0.549	0.585	0.551	0.836	–	<i>0.617</i>	–
Megawatt1	37	253	0.873	0.874	0.882	0.869	0.933	<u>0.8893</u>	0.894	<i>0.9171</i>
Pima-Indians	8	768	0.74	0.687	0.751	0.726	0.790	<u>0.7631</u>	0.745	<i>0.7865</i>
Secom	590	470	0.917	0.917	0.913	0.915	0.934	–	<i>0.918</i>	–
Sonar	60	208	<i>0.808</i>	0.763	0.468	0.462	0.839	–	<u>0.801</u>	–
Spambase	57	4601	0.948	0.737	0.39	0.413	0.948	–	<u>0.947</u>	–
SPECTF-heart	43	80	0.941	0.955	0.881	<i>0.942</i>	–	–	0.955	–
Twitter-absolute	77	140,707	0.964	0.866	0.946	0.958	–	–	0.964	–

We present a composite table (Table 6) for the experimental performance evaluations in Ref. [292]. The results show the comparative performance of state-of-the-art traditional feature engineering methods (DCN-V2 [297] and FCTree [298]) and automated engineering methods (FETCH [299], AutoCross [300], SAFE [40], OpenFE [292] and autofeat [245]). Baseline results (i.e., task performance without feature engineering) are included for comparison. The evaluated models are implemented using LightGBM [301]. Details of the datasets are presented in Refs. [292,302,303]. The models are run ten times on each of the datasets. It can be observed that in most of the cases the traditional approaches fail to outperform the baseline. While traditional approaches have shown strong performance over baseline results on UCI and OpenML datasets in several works, including [59,278,287,292], they appear to perform very poor per [292] (shown in Table 6). This is due to the fact that feature engineering process is highly dataset-sensitive, and the approaches in DCN-V2 [297] and FCTree [298] were optimized for datasets, which are fundamentally different from the UCI and OpenML datasets. Indeed, for a number of the datasets in Table 6, the performance of the automated methods also dropped significantly, sometimes below the baseline. Li et al. [299] acknowledged this problem, and their approach, feature set data-driven search (FETCH), aims to specifically tackle this issue by developing a more generalizeable feature generation and selection mechanism.

6. Holistic, end-to-end workflow of data processing in machine learning

In Section 3 through Section 5, we describe approaches for automating specific low-level data processing tasks such as data cleaning, labeling and feature engineering. Many automated machine learning techniques incorporate manual processing in some stages of the pipeline. For example, feature engineering may be fully automated but appropriate transformations can be manually selected to preprocess input data according to the specific details of the task and the properties of the underlying data (e.g., in Refs. [40,285,287]). Recently, there is a trend toward full automation of the entire machine learning pipeline from data acquisition to model deployment, where not only data-centric tasks are automated, but also the realization of the overall big data solution (Fig. 23).

End-to-end frameworks aim to automatically collect and process data, perform feature engineering and also fix various errors inherent in the data (for example, missing metadata) or errors that may arise in the course of processing. In addition, these frameworks can simultaneously carry out hyperparameter optimization, model architecture construction, selecting of evaluation metrics, prediction, analysis of results, and performing other machine learning tasks. This enhances the ability to discover valuable semantic information from large volumes of unstructured data for big data applications.

To enable a single end-to-end AutoML framework to solve a wide range of big data problems, several components are often used to create AutoML systems. In particular, different blocks of processing units are typically used to handle different tasks such as data cleaning, augmentation and feature engineering. These functional blocks are often arranged sequentially in the form of a structured pipeline. To deal with the different processing requirements of diverse data formats and tasks, multiple pipelines with different structures incorporating several machine learning models are often used.

Since the types of computational blocks required for a particular problem depend on the nature of dataset and task, the selection of the structure and parameters of the specific pipeline to use is an important part of the automated data processing and machine learning tasks. Thus, the automation problem reduces to the task of combining and optimizing the various models in the pipeline in order to achieve the best possible performance. The pipeline is often represented as a computational graph, and its structure is determined using

Table 6

Performance of automated feature engineering methods (FETCH [299], AutoCross [300], SAFE [40], OpenFE [292] and autofeat [245]) against traditional and baseline methods (DCN-V2 [297] and FCTree [298]). **Bold** is highest score, *italic* is second highest, and underline is third highest. NB: No.S (k) – number of samples (in 1000 s); CL – number of classes; Cat, Ord and Num – numbers of categorical, ordinal and numerical features.

Dataset	No.S (k)	CL	Features			Metric	Raw	DCN-V2	FCTree	SAFE	Autofeat	AutoCross	OpenFE	FETCH
			Cat	Ord	Num									
Vehicle	98.5	2	0	0	100	AUC ↑	0.925	0.924	<u>0.926</u>	0.925	0.925	0.921	0.928	0.927
Diabetes	102	2	34	10	3	AUC ↑	0.731	0.717	0.731	0.730	<i>0.732</i>	<i>0.732</i>	0.888	0.731
Telecom	51	2	22	14	21	AUC ↑	0.671	0.661	0.670	<i>0.673</i>	0.672	0.651	0.680	<i>0.673</i>
California housing	20.6	–	0	1	7	RMSE ↓	<u>0.432</u>	0.479	0.432	–	0.444	–	0.421	0.430
Microsoft	1200	–	0	25	111	RMSE ↓	<i>0.744</i>	0.750	0.744	–	<i>0.744</i>	–	0.738	×
Nomao	34.4	2	29	55	34	AUC ↑	0.996	0.992	0.996	0.996	0.996	0.993	0.997	0.996
Broken machine	900	2	0	27	31	AUC ↑	<u>0.756</u>	0.748	0.750	0.750	0.750	0.765	0.786	×
Jannis	83.7	4	0	0	54	Acc. ↑	0.721	0.720	0.719	–	0.721	–	0.729	0.720
Covertypes	581	7	0	45	9	Acc. ↑	0.969	0.966	0.719	–	<u>0.968</u>	–	0.974	×
Medical	163	–	6	0	5	RMSE ↓	<u>1128</u>	1413.7	1089	–	1172	–	982.0	1130.4

various functions to numerically evaluate the performance of different configurations. While some end-to-end AutoML systems such as ATM [304], ML-Plan [305] and Hyperopt-Sklearn [306] utilize fixed pipelines, a large number of new approaches (e.g., AutoDES [307], FLAML [308], RECIPE [309] and H2O AutoML [100]) employ variable-length pipelines. Approaches employing variable pipelines typically use evolutionary computational algorithms to create a population of basic processing operations and iteratively adapt the simple algorithms to generate new and better algorithms through mutation and crossover. Recently, many large-scale generic AutoML tools have been developed to allow off-the-shelf application of machine learning models for various data-centric tasks. The main categories, characteristics and functions of these systems are presented in Section 8.

AutoSmart [311] (Fig. 24) is another example of a fully automated machine learning framework that performs several data-centric tasks in an end-to-end manner. These include data preprocessing, data integration (i.e., table merging), feature synthesis and selection, as well as ensemble learning and model hyperparameter tuning. The framework incorporates a computational time and memory controller for managing the usage of computational resources required for the task. There are many off-the-shelf tools built on the basis of end-to-end automated machine learning that are even more complex and functional more capable than the models described above. We describe these models in more detail in Section 8.

7. Generic AutoML tools for data processing and feature engineering

Generic AutoML tools are aimed at enabling fully automated workflows in model development. That is, they allow nonprofessional users to use AutoML system to take raw input data, perform all necessary processing functions and then generate suitable models for an end application. Thus, they provide non-expert users the opportunity to use state-of-the-art machine level techniques to solve complex

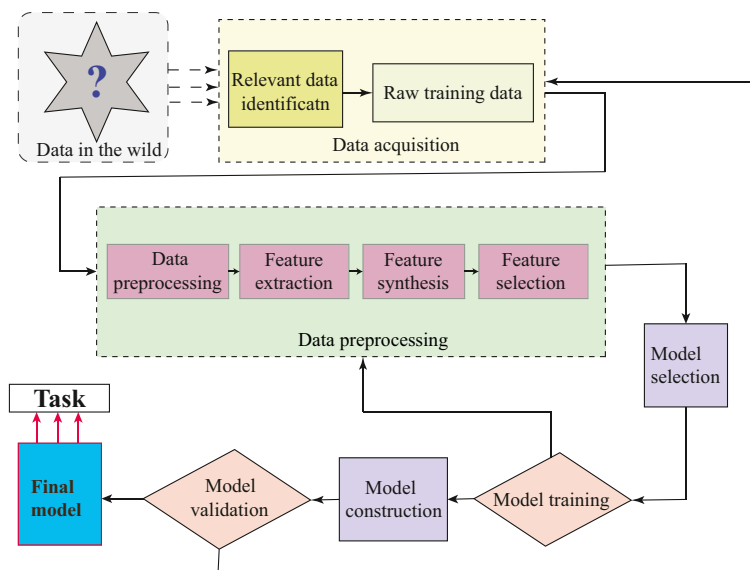


Fig. 23. Simplified architecture of an end-to-end AutoML framework.

problems without explicitly processing data and building models. These tools are generally characterized by a high degree of flexibility. They allow to construct ML pipelines using various types of input data. The most commonly supported input formats are tabular, text, time series and image data types. Many of the tools offer a way to customize and manage model performance and complexity. Some of the most widely used generic AutoML frameworks include TPOT [96], Auto-WEKA [243], Amazon SageMaker Autopilot [312], DataRobot [313], Auto-Keras [97], AutoGluon [314] and H2O AutoML [100]. We summarize the main features of these frameworks in Table 7.

While most of the most popular tools are designed to tackle general problems related to big data, some AutoML tools – e.g., Google AutoML Vision [242], AutoPrognosis [310], JADBio [57], GAMA [315] and MedicMind [27], Microsoft Azure Custom Vision [316] – are domain-specific. AutoPrognosis [310] and MedicMind [317], for instance, are specifically designed to deal with healthcare related data. AutoPrognosis [310] (shown in Fig. 25) is concerned with using electronic health records to aid diagnosis while MedicMind [317] deals with medical image analysis. Google AutoML Vision [242] and Microsoft Azure Custom Vision to [316] are aimed at solving computer vision problems such as image classification and object recognition.

7.1. Common functions supported by AutoML tools

In the course of training, the AutoML framework creates several alternative pipelines that test, evaluate and validate different sets of machine learning algorithms and hyperparameter settings for solving specific problems. To accomplish this in big data applications, the frameworks are usually sub-divided into specialized components dealing with specific tasks and data types: image, text or tabular data. There may also be special modules to support feature generation and preprocessing. AutoML tools typically handle multiple data processing tasks, including data collection and preliminary preparation, processing and post-processing: They can automatically find missing data, identify incorrect labels, and select the desired subsets of data required for a given task.

The tools generally come with user-friendly graphical user interfaces (GUIs) with production-ready tools. The main functions include task analysis to understand the needs of the particular user; problem recognition – to discover the specific problem the user intends to solve and generate a set of potential machine learning models capable of solving the problem; model evaluation and validation are carried out to arrive at an optimal solution. The GUI allows users to easily accomplish all these steps by performing intuitive actions to achieve desired results in an almost automated way; given a particular dataset, with minimal user input, the system is able to specify plausible problems that can be exploited to automatically generate models that produce end solutions. AutoML systems also provide intuitive information about the generated models and the data manipulation processes in a way that provides additional insights to guide non-expert users. Based on this information and their specific goals and preferences, users are able to refine the automatically defined problems and solution sets. Many AutoML platforms are readily compatible and interoperable with standard third-party tools, allowing the results of the processing stage to be seamlessly integrated with external applications [18]. In addition to data processing

Table 7

A summary of the main features of popular generic AutoML tools indicating the main tasks they perform and their domains of operation. CV – computer vision, NLP – natural language processing, Class. – classification, and Regres. – regression.

Tool	Type	Data types supported	Main domains and tasks				
			CV	NLP	Time series	Class.	Regres.
Auto-Keras [97]	Open source	Text, image, tabular	✓	✓	✓	✓	✓
Amazon SageMaker Autopilot [312]	Proprietary	Tabular, time series, image, texts	×	×	✓	✓	✓
Google Vertex AI [318]	Proprietary	Tabular, image, text, and video data	✓	✓	✓	✓	✓
Google Cloud AutoML [242]	Proprietary	Tabular, image, text, and video data	✓	✓	✓	✓	✓
Auto-WEKA [243]	Open source	Tabular	×	×	×	✓	✓
BigML OptiML [319]	Open source	Tabular, image, text, time series	✓	✓	✓	✓	✓
PyCaret [320]	Open source	Tabular, text, time series	×	✓	✓	✓	✓
DataBricks [321]	Proprietary	Tabular, text, time series	×	×	✓	✓	✓
AutoGluon (Amazon) [314]	Proprietary	Image, text, tabular data	✓	✓	×	✓	✓
Auto-Sklearn [322]	Open source	Tabular	×	×	×	✓	✓
MS Azure AutoML [323]	Proprietary	Time series, image, text	×	✓	✓	✓	✓
DataRobot [313]	Proprietary	Time series, image, text	✓	✓	✓	✓	✓
H2O AutoML [100]	Proprietary	Tabular, text	×	✓	×	✓	✓
H2O Driverless AI [324]	Proprietary	Time series, tabular, image, text	✓	✓	✓	✓	✓
TPOT [96]	Open source	Tabular, text, image	✓	×	×	✓	✓
Darwin [325]	Proprietary	Tabular	×	×	✓	×	✓
IBM Watson AutoAI [326]	Proprietary	Tabular, text, time series	×	×	✓	✓	✓
FEDOT [327]	Open source	Tabular, time series, images, text	✓	✓	✓	✓	✓
MLBox [328]	Open source	Tabular	×	×	×	✓	✓
NNI (Microsoft) [329]	Open source	Tabular, image	✓	×	×	✓	✓
Ludwig [330]	Open source	Tabular, text, time series	✓	✓	✓	✓	✓
FLAML (Microsoft) [308]	Proprietary	Tabular, text, time series	×	×	✓	✓	✓
ATM [304]	Open source	Tabular, image, text	×	×	×	✓	✓

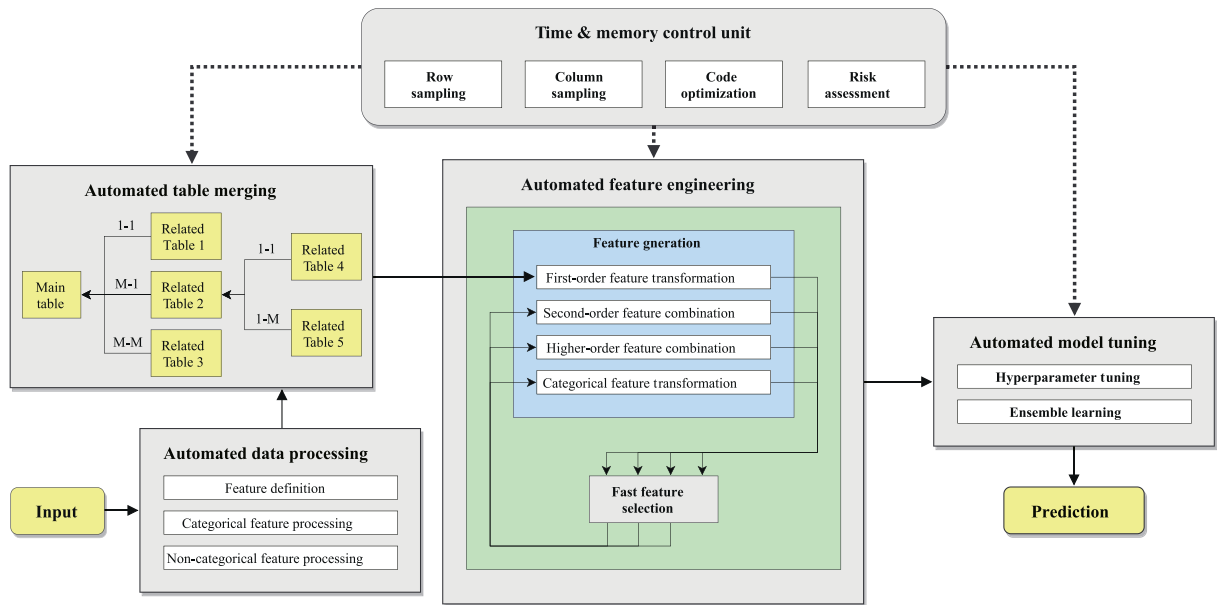


Fig. 24. The general architecture of AutoSmart [311]. The approach performs several data preprocessing and feature engineering tasks in an end-to-end manner.

functions, commercial AutoML tools provide additional functionalities such as data visualization. Tools like Pecan AI [331] allow users to seamlessly interact with their models and data without any coding, whatsoever.

Another important function commonly supported by advanced AutoML tools is visualization. Visualization utilities typically provide all necessary information needed to understand and make useful decisions about the task in relation to the available data. Such capabilities facilitate interpretability and provides useful insights into complex machine learning mechanisms. This is useful for knowledge discovery applications. In medicine and healthcare, these applications include the discovery of novel drug targets, or the design of better assays with minimal field measuring and testing requirements. Visualization capabilities can also allow developers to better assess the robustness and overall quality of a model before deploying in practical applications. For instance, one can examine the effect of missing specific features by deliberately removing features of interest from a given model (e.g., in Ref. [57]). Moreover, visualization of the AutoML process can help to enrich the knowledge of data scientists or machine learning researchers by providing new insights [58]. It also enhances trust in AutoML solutions [332,333] since developers can visually probe and validate pertinent issues related to trust and reliability in an intuitive manner.

Since many large-scale AutoML tools integrate several technologies, and multiple data pipelines into a single platform, they significantly increase synergy and effectiveness of solutions. The use of highly integrated platforms also speeds up development time by eliminating additional stages of development, testing and deployment of big data solutions.

7.2. Main categories and features of AutoML tools

AutoML tools can broadly be divided into two categories: open source and proprietary tools.

7.2.1. Open source AutoML tools

Common open source AutoML systems include Auto-WEKA [243], Auto-Keras [97], PyCaret [320], TPOT [96], FEDOT [327] and Auto-Sklearn [322]. The development of open source AutoML tools is characterized by the involvement of very large community of developers who contribute and update different functionalities independently. They are generally built on open source machine learning libraries and frameworks such as PyTorch [334], Scikit-learn [335] and TensorFlow [336]. Because the codebases for such systems are accessible to users and developers, custom modifications can easily be implemented, unlike proprietary systems. They may also offer seamless interoperability or integration options with minimal or no additional requirements for working with other open source tools. Another important feature of open source AutoML tools is the wide community involvement in the development and extension of functionalities. Generally, open source AutoML tools are more restricted in terms of the level of automation. They typically involve some level of manual work in the implementation of data processing tasks and model development. In particular, users are required to have some competence in the underlying programming languages (often, C++, Python or R) used to build these tools. It also requires a level of understanding of the modeling process – correct formulation of big data problem and an intuitive choice of possible models, as well as how to assess the suitability of resulting models. These tools are designed with flexibility and ease of adaptation and extension in mind. Compared to proprietary tools, they provide less intuitive graphical user interfaces (GUIs), and are generally more difficult for non-expert users to utilize in processing data and building models for various tasks.

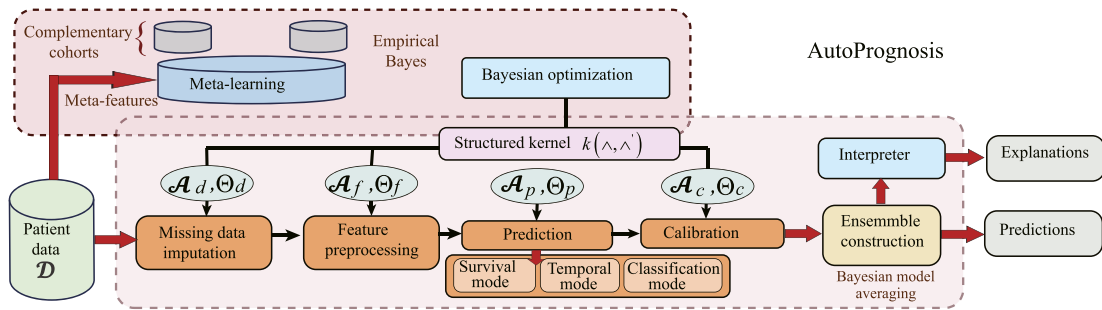


Fig. 25. AutoPrognosis [310] uses patient data to perform end-to-end automated data processing. The system is aimed at enhancing clinicians' decision making by making accurate prognosis of health outcomes and also providing explanations for predictions.

7.2.2. Proprietary AutoML tools

Some of the largest and most popular AutoML tools are proprietary solutions. Prominent among these include MS Azure AutoML [323], FLAML [308], H2O AutoML [100], Google Cloud AutoML [242], AutoGluon [314], IBM Watson AutoAI [326] and DataBricks [321]. Among these tools, one can also distinguish systems developed by technology start-ups (e.g., Darwin [325] by SparkCognition, OptiML [319] by BigML) and those owned by very big companies, so-called tech giants (e.g., AutoGluon [314] by Amazon and Google Cloud AutoML [242] by Alphabet). Solutions provided by start-ups are generally stand-alone products aimed at relatively narrow scope of applications. Those provided by technology giants tend to be more generic and focus on providing a broad range of tools to meet diverse business needs. The tools developed by tech giants are typically cloud-based platforms capable of handling large amounts of data for businesses as well as individual users. Prominent among these category of AutoML tools include Amazon SageMaker Autopilot [312], Google Cloud AutoML [242], Google Vertex AI [318], Microsoft Azure ML [323], IBM Watson AutoAI [326]. The tools provide easy-to-use graphical user interfaces and a rich set of tools within advanced integrated development environments that support complex tasks and processes. The cloud-based platforms are generally highly scalable and easily allow integration of other cloud-based utilities. Microsoft Azure ML [323], for instance, provides a suit of cloud-based AutoML tools such as Machine Learning API Service, Microsoft Azure Custom Vision and the Machine Learning Studio. In addition to providing utilities and APIs on the cloud that can be leveraged by non-expert users to process training data and build machine learning models for various tasks, it allows one to seamlessly work directly with other Microsoft tools such as Microsoft Azure HDInsight and MS SQL Server. Similarly, Google Cloud AutoML provides modules such as AutoML Vision, AutoML Data Science and AutoML Natural Language that perform computer vision, big data processing and analytics, and NLP tasks, respectively. These toolsets are cloud-enabled, and can further integrate and interact with other Google products through the cloud.

Proprietary frameworks are designed to particularly target business users with subject matter knowledge about their technical job areas or business operations but without data science expertise. Since such users cannot easily understand what data processing tasks need to be performed, or create appropriate queries to explore and analyze data, this type of AutoML tools provide typically intuitive graphical user interfaces and visualization utilities to allow users to perform these data processing, exploration and analysis functions at a higher level.

8. Implications for industry and commerce

The techniques discussed in this survey seek to automate mundane machine learning tasks and reduce errors associated with manual data processing procedures. Specifically, they eliminate all stages of data preparation and manual model selection, tuning and evaluation. This has led to greater productivity and improved accuracy of results in many areas. The automation of low-level data processing and model development steps allows business users to move straight to generating results from their raw input data. This reduces the time required for products and services into move from development to market. The enormous power of automated AI technologies also helps industries to process extremely large volumes of heterogeneous data. This allows them to better discover and exploit new insights to accelerate innovation, leading to the development of new products, services and even entirely new business models.

The transformative role of AI in business and industry is undisputed [337]. However, implementing practical AI solutions for industry and business requires highly qualified professionals with strong expertise in statistics, mathematics and computer science. Unfortunately, there is a severe shortage of qualified personnel [338,339]. Luckily, state-of-the-art AutoML and generative AI tools have simplified the development and application of AI solutions by encapsulating all complex algorithmic implementations behind intuitive user interfaces which support high-level interaction. These technologies have enabled the development of powerful AI tools and platforms that eliminate the need for AI specialists and allow non-expert users to be able to effortlessly perform otherwise complex data processing and entire machine learning tasks. They effectively eliminate the need for firms to hire expensive and scarce personnel. In

practice, this means lowering the entry barrier for businesses, and thus empowering small businesses and industries with powerful AI capabilities. The emergence of cloud-based tools further expands the potential of businesses without the needed physical infrastructure to leverage AI capabilities for productivity. The simplification and significant reduction of cost of new models will undoubtedly expand the scope of AI and machine learning technologies in business and industry. We discuss the implications of these technologies for specific industries, namely: healthcare, agriculture, manufacturing, and retail, banking and finance.

8.1. Healthcare

The implications of automated data processing and the emergence of predictive analytical tools in healthcare are enormous. With the ability to process large volumes of medical data with minimal human intervention and provide useful insights and recommendations, these technologies enable medical professionals to make accurate diagnoses and provide appropriate treatment plans. Specifically, automated AI tools with access to large volumes of medical data – e.g., from wearable medical devices and IoT sensors, electronic medical records, scans, clinical tests and genetic profiles – together with advanced scientific knowledge bases (e.g., in the form of generative AI-based models like LLMs) can quickly analyze any new medical case, provide high-level, human-understandable information about the diagnosis and treatment options. Indeed, specialized LLM-based medical tools such as Med-PaLM 2 [340] and MedAlpaca [341] are already capable of expert-level medical question answering, and in some cases outperforming medical professionals in medical licensing examination [342].

Additionally, the use of advanced data processing tools greatly helps to eliminate human factors from medical care, minimizing the risk of costly errors that humans sometimes commit as a result of distraction and fatigue.

Automated AI solutions also serve as a catalyst for innovation in medical treatment. For instance, they are able to analyze existing treatment options and reconfigure them to work better or to fight new medical conditions. They are able to provide more suitable treatment plans for individuals with peculiar underlying physiological issues. These data-driven tools are also important for tasks like drug discovery as they help in the identification of new drug targets, analysis of their effectiveness and prediction of potential drug reactions and side effects. The use of these modern technologies significantly increases the precision and efficiency of medical care while at the same time significantly reducing the costs.

In addition to extending the capabilities and service quality of medical professionals, these technologies, through online conversational AI tools (e.g., Chatbots), also offer patients the means to access personalized healthcare anytime and anywhere; individuals who experience symptoms can consult chatbots for personalized medical advice and direction. This capability is especially useful as supportive care system for elderly people. This also frees precious time for medical professionals, allowing them to focus on important medical problems instead of taking patients' medical complaints and providing consultancy services in non-critical situations.

8.2. Agriculture

In agriculture, these techniques will simplify the complex task of obtaining and processing heterogeneous data for various field variables and conditions. Additionally, farmers will be able to utilize advanced digital tools to automate complex agricultural tasks that would have otherwise required technical expertise in AI and machine learning. Specifically, they will be able to develop advanced predictive analytics models using AutoML and generative AI tools to predict and manage risks such as disease and pest infestation, changes in weather and climate patterns, and other factors for crop failure. Using these tools, lay farmers will be able to forecast market demand for their produce and predict the expected crop yield, as well as provide better service to their suppliers, distributors and downstream customers.

8.3. Retail, banking and finance

Another important area of significant impact of automated data processing techniques is retail, banking and finance. Here, AI-enabled technologies have simplified and automated important business processes (e.g., processing and analysis of financial data, management related tasks, analysis of documents to aid compliance with regulatory requirements). They offer a cheap and an effective means to discover important business patterns, trends and opportunities while simultaneously providing useful and actionable directions to responsible personnel to respond appropriately. With the help of automated data processing techniques companies are able to focus on business problems instead of on data processing and analysis, thereby increasing their productivity and profitability. By processing customer data in advanced analytics engines, companies are also able to predict customer behavior and preference, and thus provide tailored products and services to better address their needs. Generative AI tools developed on the basis of business-specific data also enhance customer experience by automating customer engagement tasks in the form of advanced virtual assistants that can provide context-specific information and personalized financial recommendations to clients in real-time.

8.4. Manufacturing

In manufacturing, automated AI tools can also help to streamline overall factory operations as well as automate, optimize and manage energy consumption, logistics, and production. Data gathered from IoT devices and sensors can be analyzed by automated machine learning models to provide useful information about potential hazards and recommend preventive actions. Generative AI techniques allow manufacturing companies to design new products simply by specifying a set of desired attributes. This particular capability has a huge potential in manufacturing as it can lead to innovative product concepts and products with qualitatively better

characteristics than their counterparts designed by traditional methods. For instance, such products may be more efficient, cheaper and environmentally-friendly.

9. Discussions

In the era of big data, the volume and complexity of data that machine learning systems typically handle have increased substantially. Consequently, collecting and processing the data into a form that is suitable for machine learning tasks is a challenging undertaking. Approaches based on conventional machine learning concepts are extremely laborious and require enormous development time. Automated processing methods, especially approaches based on AutoML, have greatly automated these laborious processes and, thus, have simplified and accelerated the development cycle of deep learning models. As the size of data and the complexity of machine learning problems increase, such approaches are expected to become a general practice, especially for generic machine learning tasks such as big data analytics and data visualization.

9.1. Main challenges

We discuss some of the most important challenges modern automated data processing systems have to deal with.

9.1.1. Growing volume and complexity of data

While more and more data processing tasks are becoming increasingly automated, today's AutoML methods are still limited in terms of the complexity of data and tasks that can be handled. Handling very complex data still requires the intervention of human developers at some stages in the development process. While many of the recent works have focused primarily on reducing the amount of computational resources needed to implement automated data processing solutions, the techniques continue to evolve steadily and many new workarounds are expected to be introduced in the foreseeable future to extend the scope of tasks that can be automated. Future AutoML systems will have the ability to create end solutions for complex problems by automating the full range of tasks from input data acquisition to model construction and validation.

Since many data processing tools are intended for users with different levels of expertise and the resulting models typically need to meet several performance objectives simultaneously, albeit within an acceptable cost, it is often challenging to ensure that they achieve the right balance of performance and complexity. To address this issue, it is possible to develop a flexible data processing scheme which can be modified per the specific user or application requirements. For instance, Tsamardinos et al. in Ref. [57] devised different model configuration options for an AutoML model, where each configuration prioritizes a specific machine learning objective: interpretability, predictive performance or minimization of data size. These configuration settings allow users to customize model performance according to their priorities. This approach is used by data processing engines in many general-purpose automated machine learning systems that handle large-scale data. However, these state-of-the-art approaches still require users to have an understanding of the underlying requirements and to manually select the best settings for the particular task. Future work could employ context knowledge to automate this process, too.

9.1.2. Complexity of data processing problems

The application of machine learning in more diverse application areas, coupled with the rapidly increasing complexity of data pipelines has made it challenging to solve many modern problems using automated machine learning. Preprocessing tasks such as labeling and categorical data annotation are particularly difficult to automate. Because of these challenges, in practical applications, AutoML systems usually automate only some data processing tasks in the machine learning pipeline and provide baseline results that can reveal further insights into possible avenues for improvement by manual means. In the near future, the introduction of a wide variety of new data processing techniques and algorithms for their optimization will provide a means for solving complex problems whose solutions are unattainable at present.

9.1.3. Context-awareness

Approaches based on state-of-the-art AutoML often generate a large pool of intermediate features and attributes in the search space. The performance of the end model ultimately, to a large extent, depends on the search space. Because the generation process is inherently “blind”, generated features might not be informative or relevant to the target task or could introduce redundant information. This could, even with good optimization methods, lead to poor performance.

9.1.4. Overly conservative results

To avoid making bizarre errors, generative models are typically designed to be very conservative and tend to produce outputs that align well within the distribution of the encoded training data, thus avoiding extreme cases and outliers. In many real-world domains, however, data is often not clean. Consequently, the output data of generative models often lack the wide variability of real-world data. For instance, large language models generally fail to produce context-specific responses and resort to overly generic answers. Training with such data may result in less-than-acceptable outcomes in some cases. Moreover, because of the enormous sizes, complexity and opacity of generative AI-based automated data processing and synthesis systems such as diffusion models and LLMs, they can sometimes produce unreliable and potentially harmful data that may be difficult to detect. The approaches are also prone to the so-called hallucination effect [343], a phenomenon that can cause generative models to generate wrong captions or image descriptions based on non-existent objects. These problems severely limit the utility of generative AI models in high-stakes tasks, especially in application

domains such as finance, security and healthcare.

9.1.5. *Adaptability and transferability*

Because the processing procedures are not based on intuitive and well-grounded mechanisms, data generated by automated techniques might be specifically optimized for a single task or a narrow set of tasks. This may produce good results for the specific models and datasets they have been tuned for but fail to generalize well in unseen data and new problem domains. The problem of transferability and adaptability of automatically generated data has not been investigated in the literature. Also, as data in many application settings (e.g., data about pandemics, economic or market conditions) evolve over time, automated processing methods need to be able to adapt accordingly. Moreover, it is often useful to have a single solution that can address a set of related problems in a broad application domain. Current approaches are not able to meet this need.

9.1.6. *Balancing multiple requirements and trade-offs*

Modern automated data processing problems often involve very large datasets with complex relationships and interactions. At the same time, models that are trained on them are required to meet multiple performance requirements. In particular, there is the need to navigate delicate trade-offs among different aspects of performance, such as reducing data size versus retaining important information, achieving higher predictive accuracy versus interpretability, robustness and algorithmic fairness. Methods that enhance performance in one aspect might harm performance in another. These complex trade-offs make it difficult for automated machine learning algorithms to strike the right balance independent of human input, as users may prioritize different aspects of performance according to their specific goals.

9.1.7. *Reliability and trust*

With the widespread adoption of AI in many mission-critical areas, their reliability – and consequently trust in their solutions – has become an important issue. This is particularly more serious with automated learning methods since they rely on black-box approaches automated hyperparameter optimization and there is very little human supervision of the overall learning process. For instance, automated hyperparameter optimization might result in complex settings and potentially harmful interactions that are difficult to predict or diagnose at design time but may manifest later under deployment. Moreover, with these black-box methods, it is challenging to guarantee that the end applications will perform as intended and align solutions with broader user objectives while respecting societal values and expectations – i.e., make accurate predictions while at the same time being constrained by wider issues such as safety, privacy, norms and ethics. It is currently not possible to incorporate these high-level concepts in the automated learning process.

9.1.8. *Lack of comprehensive valuation metrics for some processing tasks*

Unlike many machine learning tasks where standard model configurations, datasets and evaluation metrics have been created to test different methods, for many data processing tasks, especially preprocessing, there is a general lack of standardized settings and metrics that can be universally applied across different tasks and datasets. This leads to researchers using varying settings and metrics, thereby making direct comparison of methods very challenging.

9.1.9. *Scalability*

Scalability is another major limitation of automated data processing methods. While these techniques work well for large-scale problems involving huge datasets, they typically perform very poorly on small data. In situations where data is scarce, traditional processing techniques often outperform automated methods. Furthermore, automated processing methods are inherently computationally expensive. Therefore, while automated data processing approaches are promising, the choice between automated methods and traditional approaches ultimately depends on factors such as dataset size, problem domain, and available computational resources.

9.1.10. *Limited scope of application*

Existing AutoML tools are limited in terms of the range of tasks and data types they support. Presently, most generic solutions work best for tabular data. Also, most automated data processing techniques support classification and regression problems, while offering relatively little in problem domains such as natural language processing and time series forecasting.

9.2. *Future prospects*

Since automated data processing methods are fairly new, one would assume that we are just at the initial stage of realizing the vast potential these approaches present. At the same time, it is important to exercise caution when making assessments about its future possibilities as undue expectations can ultimately lead to disappointment, which can potentially result in “automated data processing winter” [344]. Also, because automated data acquisition and processing methods are characteristically resource intensive, their potential will depend on the progress in other areas such as computer hardware technology and the development and extensive use of cloud computing infrastructure. The main prospects for the foreseeable future include the following.

9.2.1. *Support for higher-level functions*

New data preprocessing algorithms that also perform quality control and budget management functions taking into consideration the specific application requirements as well as the objectives and priorities of the end user will be developed. This will involve more

advanced user interface (UI) and user experience (UX) solutions with intuitive features that support high-level semantic interaction.

9.2.2. *Extension of generic AutoML methods to handle more complex problems and diverse data types*

While specialized automated data processing methods have partially tackled a variety of data types and machine learning problems, presently, generic AutoML solutions are designed primarily to work with tabular data; their support for other data types such as images, audio, video and point clouds is very limited. Also, they mostly solve general classification and regression tasks, and poorly handle data processing for applications in time series and natural language processing domains. In the near future, new approaches that extend the scope of automated data processing methods, especially those based on AutoML frameworks, to a wider problem domain, including time series and natural language processing, are envisaged.

9.2.3. *Advanced human-in-the-loop (HITL) automated data processing*

Future research will concentrate on developing methods to provide user-friendly interfaces and more intuitive explanations for human actors based on techniques such as feature importance visualization, counterfactual explanations, and attention mechanisms. In turn, researchers will develop machine learning algorithms that can learn from human feedback through, for example, explicit labeling and reinforcement signals. Together, these two sets of approaches can be leveraged to provide synergy between humans and machines. This will ultimately lead to improved performance, reliability, robustness, and the ability to tackle more challenging problems that are beyond the capability of either humans or machines when working independently. It is conceivable that more advanced HITL systems will enable support for interaction of multiple users or user-expert groups with automated machine learning systems. Businesses, for example, will be able to model high-level information such as broad organizational goals and priorities, available budgetary resources, and the overall bottom line. Systems based on this approach will also align better with broader human needs, values, and expectations.

Humans in the machine learning loop will be particularly useful for more complex data processing tasks such as categorical encoding and data annotation. Humans will also play a vital role in model validation and refinement, as well as the incorporation of high-level concepts such as.

9.2.4. *New and dedicated infrastructure for automated data processing*

Dedicated tools that specifically cater to the needs of big and complex data problems will be developed. These will be specialized platforms (e.g., cloud-based infrastructure), tools (e.g., libraries and open-source software), and frameworks (e.g., generic data processing models) designed to support and streamline different aspects of automated data processing. This infrastructure will simplify various processing tasks and make it easier for both data scientists and non-experts to effectively use automated machine learning models for data processing. This type of infrastructure will seamlessly provide needed functions while taking care of contemporary challenges such as privacy, transparency, and data security.

9.2.5. *Progressive self-refinement of synthetic data*

It is important to note that while automated data processing models can perform complex data processing and can even synthesize relevant data automatically, paradoxically, their training requires large volumes of high-quality data that correctly captures the distribution of data in the target domains. For many domains data are not often available naturally. However, it is conceivable that in the future, in cases where domain-relevant data may be inaccessible, more generic generative AI models could be used to synthesize rudimentary data and then perform progressive self-refinement by continually generating, filtering and reusing the generated data for subsequent retraining and fine-tuning until an acceptable quality of data is attained. This approach is already being studied [345], and preliminary results are promising.

9.2.6. *Explainable data synthesis by generative AI techniques*

In the near future, researchers will focus on developing models that not only deliver high performance but also provide insights for their actions. Explainable AutoML frameworks designed for data processing will be able to perform data processing tasks in an end-to-end machine learning pipeline while providing explanations about the intermediate processes and resulting outcomes. This approach will be particularly useful for complex data processing tasks such as categorical encoding and data annotation. Explainability will also play a vital role in allowing researchers to better evaluate and validate results, as well as to perform further refinements.

Generative AI techniques will be particularly useful in solving the problem of opacity associated with the data generation process of traditional AutoML methods. For instance, the massive volume of knowledge embedded in LLMs and diffusion models can be leveraged to explain the underlying AutoML model's decisions regarding generated data. Specifically, these models will be able to create additional metadata about the generated data, including information about the relationships among different data elements and attributes. In addition, intuitive user interfaces and high-level interaction mechanisms will allow developers to incorporate human-understandable information about the target tasks for which the dataset is to be generated. This can help to mitigate inadequacies and potential flaws in the data generated by today's generative AI methods.

10. Conclusion

The importance of automated data processing has increased remarkably in the last few years. This is largely driven by the increasing demand for machine learning solutions in many areas, coupled with the large volumes of data that need to be processed for these machine learning tasks.

In this work, we survey state-of-the-art approaches for automating data processing for deep learning and big data tasks. We first present methods for realizing individual data processing solutions. These include data preprocessing (e.g., data cleaning, imputation, labeling, categorical encoding, etc.), data augmentation and feature engineering (specifically, feature extraction, construction and selection). We also discuss approaches to implementing all processing steps holistically within a single end-to-end deep learning framework. We summarized the main characteristics and functions of generic AutoML frameworks designed for big data applications. Furthermore, we discuss future developments that are likely to have a significant impact on the success of automated data processing.

The survey shows that while many data processing tasks can already be seamlessly automated, challenges still remain regarding the full automation certain tasks, especially data preprocessing. The need to address a wide scope of problems makes it particularly difficult for machine learning systems to incorporate effective search mechanisms that allow relevant data to be collected and exploited in a context-dependent manner.

Availability of data and materials

The work does not involve primary data. We report on works that have been published.

Code availability

Not applicable.

Authors' contributions

The authors' contribution to the manuscript is detailed below.

- (1) Both **Alhassan Mumuni** and **Fuseini Mumuni** conceived the original idea, carried out a preliminary survey of related studies, and designed the initial structure of this survey.
- (2) **Both authors** contributed significantly to the subsequent refinements of the study structure.
- (3) **Both authors** jointly devised the criteria to classify data preprocessing and augmentation methods.
- (4) **Alhassan Mumuni** primarily reviewed data augmentation techniques. He also essentially contributed the section on feature engineering approaches.
- (5) **Fuseini Mumuni** surveyed data preprocessing methods. In addition, he contributed the section on data integration techniques. He also investigated and presented summarized results on the general AutoML tools.
- (6) The initial drafting as well as graphics and artworks were done by both **Alhassan Mumuni** and **Fuseini Mumuni**.
- (7) **Both authors** participated in subsequent revisions and refinements of the paper.

Acknowledgments

The work was solely financed by the researchers. No external agency provided funding, whatsoever. The review of this article was coordinated by Dr. Jungong Han.

Declaration of competing interest

There is no conflict of interests or competing interests in this study.

References

- [1] A. L'Heureux, K. Grolinger, H.F. Elyamany, M.A. Capretz, Machine learning with big data: Challenges and approaches, *IEEE Access* 5 (2017) 7776–7797.
- [2] M.M. Najafabadi, F. Villanustre, T.M. Khoshgoftaar, N. Seliya, R. Wald, E. Muharemagic, Deep learning applications and challenges in big data analytics, *Journal of Big Data* 2 (1) (2015) 1–21.
- [3] P. Rouzrokh, B. Khosravi, S. Faghani, M. Moassefi, D.V. Vera Garcia, Y. Singh, K. Zhang, G.M. Conte, B.J. Erickson, Mitigating bias in radiology machine learning: 1. data handling, *Radiology: Artificial Intelligence* 4 (5) (2022) e210290.
- [4] S. Carta, A.S. Podda, D.R. Recupero, R. Saia, A local feature engineering strategy to improve network anomaly detection, *Future Internet* 12 (10) (2020) 177.
- [5] S. Lu, X. Wei, Y. Li, L. Wang, Detecting anomaly in big data system logs using convolutional neural network, in: *Proceedings of the 2018 IEEE 16th International Conference on Dependable, Autonomic and Secure Computing, 16th International Conference on Pervasive Intelligence and Computing, 4th International Conference on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech)*, IEEE, Piscataway, 2018, pp. 151–158.
- [6] S. Garcia, S. Ramirez-Gallego, J. Luengo, J.M. Benitez, F. Herrera, Big data preprocessing: Methods and prospects, *Big Data Analytics* 1 (1) (2016) 1–22.
- [7] J. Waring, C. Lindvall, R. Umeton, Automated machine learning: Review of the state-of-the-art and opportunities for healthcare, *Artificial Intelligence in Medicine* 104 (2020) 101822.
- [8] G. Liu, D. Lu, J. Lu, Pharm-AutoML: An open-source, end-to-end automated machine learning package for clinical outcome prediction, *CPT: Pharmacometrics and Systems Pharmacology* 10 (5) (2021) 478–488.
- [9] J. Drodal, J. Weisz, D. Wang, G. Dass, B. Yao, C. Zhao, M. Muller, L. Ju, H. Su, Trust in AutoML: Exploring information needs for establishing trust in automated machine learning systems, in: *Proceedings of the 25th International Conference on Intelligent User Interfaces*, ACM, New York, 2020, pp. 297–307.
- [10] Z. Li, H. Guo, W.M. Wang, Y. Guan, A.V. Barenji, G.Q. Huang, K.S. McFall, X. Chen, A blockchain and AutoML approach for open and automated customer service, *IEEE Transactions on Industrial Informatics* 15 (6) (2019) 3642–3651.
- [11] A. Crisan, B. Fiore-Gartland, Fits and starts: Enterprise use of AutoML and the role of humans in the loop, in: *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, ACM, New York, 2021, pp. 1–15.

- [12] X. He, K. Zhao, X. Chu, AutoML: A survey of the state-of-the-art, *Knowledge-Based Systems* 212 (2021) 106622.
- [13] M.-A. Zöller, M.F. Huber, Benchmark and survey of automated machine learning frameworks, *Journal of Artificial Intelligence Research* 70 (2021) 409–472.
- [14] K. van der Blom, A. Serban, H. H. Hoos, J. Visser, AutoML adoption in ML software, in: *Proceedings of the 8th ICML Workshop on Automated Machine Learning*, 2021, pp. 1–13.
- [15] U. Gain, V. Hotti, Low-code AutoML-augmented data pipeline - A review and experiments, *Journal of Physics: Conference Series* 1828 (2021) 012015.
- [16] A. Alsharaf, K. Aggarwal, M. Kumar, A. Mishra, Review of ML and AutoML solutions to forecast time-series data, *Archives of Computational Methods in Engineering* 29 (7) (2022) 5297–5311.
- [17] C. Shorten, T.M. Khoshgoftaar, A survey on image data augmentation for deep learning, *Journal of Big Data* 6 (1) (2019) 1–48.
- [18] S.Y. Feng, V. Gangal, J. Wei, S. Chandar, S. Vosoughi, T. Mitamura, E. Hovy, A survey of data augmentation approaches for NLP, 2021. <https://arxiv.org/pdf/2105.03075v5.pdf>.
- [19] Q. Wen, L. Sun, F. Yang, X. Song, J. Gao, X. Wang, H. Xu, Time series data augmentation for deep learning: A survey, 2020. <https://arxiv.org/pdf/2002.12478v4.pdf>.
- [20] V. Jane, Survey on IoT data preprocessing, *Turkish Journal of Computer and Mathematics Education (TURCOMAT)* 12 (9) (2021) 238–244.
- [21] S. Ramirez-Gallego, B. Krawczyk, S. Garcia, M. Woźniak, F. Herrera, A survey on data preprocessing for data stream mining, *Neurocomputing* 239 (C) (2017) 39–57.
- [22] S. Khalid, T. Khalil, S. Nasreen, A survey of feature selection and feature extraction techniques in machine learning, in: *Proceedings of the 2014 Science and Information Conference*, IEEE, Piscataway, 2014, pp. 372–378.
- [23] N. Pudjihartono, T. Fadason, A.W. Kempa-Liehr, J.M. O'Sullivan, A review of feature selection methods for machine learning-based disease risk prediction, *Frontiers in Bioinformatics* 2 (2022) 927312.
- [24] S. Meisenbacher, M. Turowski, K. Phipps, M. Rätz, D. Müller, V. Hagenmeyer, R. Mikut, Review of automated time series forecasting pipelines, *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 12 (6) (2022) e1475.
- [25] N.K. Chauhan, K. Singh, A review on conventional machine learning vs deep learning, in: *Proceedings of the 2018 International Conference on Computing, Power and Communication Technologies (GUCon)*, IEEE, Piscataway, 2018, pp. 347–352.
- [26] S. Marsland, *Machine Learning: An Algorithmic Perspective*, Chapman and Hall/CRC, 2011.
- [27] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, *Nature* 521 (7553) (2015) 436–444.
- [28] L. Deng, D. Yu, Deep learning: Methods and applications, *Now Foundations and Trends® in Signal Processing* 7 (3-4) (2014) 197–387.
- [29] L.R. Koetzier, D. Mastrodicasa, T.P. Szczukotowicz, N.R. van der Werf, A.S. Wang, V. Sandfort, A.J. van der Molen, D. Fleischmann, M.J. Willemsink, Deep learning image reconstruction for CT: Technical principles and clinical prospects, *Radiology* 306 (3) (2023) e221257.
- [30] L. Budach, M. Feuerpeil, N. Ihde, A. Nathansen, N. Noack, H. Patzlaff, F. Naumann, H. Harmouch, The effects of data quality on machine learning performance, 2022. <https://arxiv.org/pdf/2207.14529v4.pdf>.
- [31] M.O. Ojo, A. Zahid, Improving deep learning classifiers performance via preprocessing and class imbalance approaches in a plant disease detection pipeline, *Agronomy* 13 (3) (2023) 887.
- [32] K.J. Kubota, J.A. Chen, M.A. Little, Machine learning for large-scale wearable sensor data in Parkinson's disease: Concepts, promises, pitfalls, and futures, *Movement Disorders* 31 (9) (2016) 1314–1326.
- [33] V. Iosifidis, E. Ntoutsi, Dealing with bias via data augmentation in supervised learning scenarios, in: *Proceedings of the International Workshop on Bias in Information, Algorithms, and Systems co-located with 13th International Conference on Transforming Digital Worlds (iConference 2018)*, CEUR Workshop Proceedings, 2018, pp. 24–29.
- [34] Y. Shi, T. ValizadehAslani, J. Wang, P. Ren, Y. Zhang, M. Hu, L. Zhao, H. Liang, Improving imbalanced learning by pre-finetuning with data augmentation, in: *Proceedings of the Fourth International Workshop on Learning with Imbalanced Domains: Theory and Applications*, PMLR, 2022, pp. 68–82.
- [35] Q. Wang, F. Meng, T.P. Breckon, Data augmentation with norm-AE and selective pseudo-labelling for unsupervised domain adaptation, *Neural Networks* 161 (2023) 614–625.
- [36] Y. Bengio, A. Courville, P. Vincent, Representation learning: A review and new perspectives, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35 (8) (2013) 1798–1828.
- [37] M. Cerrada, L. Trujillo, D.E. Hernández, H.A. Correa Zevallos, J.C. Macancela, D. Cabrera, R. Vinicio Sanchez, AutoML for feature selection and model tuning applied to fault severity diagnosis in spur gearboxes, *Mathematical and Computational Applications* 27 (1) (2022) 6.
- [38] S. Chang, C. Wang, C. Wang, Automated feature engineering for fraud prediction in online credit loan services, in: *Proceedings of the 2022 13th Asian Control Conference (ASCC)*, IEEE, Piscataway, 2022, pp. 738–743.
- [39] Q. Liu, Z. Liu, H. Zhang, Y. Chen, J. Zhu, Mining cross features for financial credit risk assessment, in: *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, ACM, New York, 2021, pp. 1069–1078.
- [40] Q. Shi, Y.-L. Zhang, L. Li, X. Yang, M. Li, J. Zhou, SAFE: Scalable automatic feature engineering framework for industrial tasks, in: *Proceedings of the 2020 IEEE 36th International Conference on Data Engineering (ICDE)*, IEEE, Piscataway, 2020, pp. 1645–1656.
- [41] A. Sharif Razavian, H. Azizpour, J. Sullivan, S. Carlsson, CNN features off-the-shelf: An astounding baseline for recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, IEEE, Piscataway, 2014, pp. 806–813.
- [42] M.A. Hall, *Correlation-based Feature Selection for Machine Learning*, Ph.D. Thesis, The University of Waikato, 1999. <https://www.cs.waikato.ac.nz/~mhall/thesis.pdf>.
- [43] Z. Che, S. Purushotham, R. Khemani, Y. Liu, Distilling knowledge from deep networks with applications to healthcare domain, 2015. <https://arxiv.org/pdf/1512.03542v1.pdf>.
- [44] D. Dai, T. Xu, X. Wei, G. Ding, Y. Xu, J. Zhang, H. Zhang, Using machine learning and feature engineering to characterize limited material datasets of high-entropy alloys, *Computational Materials Science* 175 (2020) 109618.
- [45] M. Jaderberg, K. Simonyan, A. Zisserman, K. Kavukcuoglu, Spatial transformer networks, in: *Proceedings of the 28th International Conference on Neural Information Processing Systems*, MIT Press, Processing Systems, MIT Press, 2015, pp. 2017–2025.
- [46] S. Mounsaveng, I. Laradji, I. Ben Ayed, D. Vazquez, M. Pedersoli, Learning data augmentation with online bilevel optimization for image classification, in: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, IEEE, Piscataway, 2021, pp. 1691–1700.
- [47] H. Luo, W. Jiang, X. Fan, C. Zhang, STNReID: Deep convolutional networks with pairwise spatial transformer networks for partial person re-identification, *IEEE Transactions on Multimedia* 22 (11) (2020) 2905–2913.
- [48] H.T. Vu, C.-C. Huang, A multi-task convolutional neural network with spatial transform for parking space detection, in: *Proceedings of the 2017 IEEE International Conference on Image Processing (ICIP)*, IEEE, Piscataway, 2017, pp. 1762–1766.
- [49] A. Karagrysis, *Color space transformation network*, 2015. <https://arxiv.org/pdf/1511.01064.pdf>.
- [50] P. Tarasiuk, M. Pryczek, Geometric transformations embedded into convolutional neural networks, *Journal of Applied Computer Science* 24 (3) (2016) 33–48.
- [51] Y. Xu, Z. Zhang, L. You, J. Liu, Z. Fan, X. Zhou, scIGANs: Single-cell RNA-seq imputation using generative adversarial networks, *Nucleic Acids Research* 48 (15) (2020) e85.
- [52] A. Nazabal, P.M. Olmos, Z. Ghahramani, I. Valera, Handling incomplete heterogeneous data using VAEs, *Pattern Recognition* 107 (2020) 107501.
- [53] J. Liu, F. Zhu, C. Chai, Y. Luo, N. Tang, Automatic data acquisition for deep learning, *Proceedings of the VLDB Endowment* 14 (12) (2021) 2739–2742.
- [54] L. Santos, L. Ferreira, Atlantic - Automated data preprocessing framework for supervised machine learning, *Software Impacts* 17 (2023) 100532.
- [55] A. Karras, C. Karras, S. Schizas, M. Avlonitis, S. Sioutas, AutoML with Bayesian optimizations for big data management, *Information* 14 (4) (2023) 223.
- [56] H. Jin, F. Chollet, Q. Song, X. Hu, AutoKeras: An AutoML library for deep learning, *Journal of Machine Learning Research* 24 (6) (2023) 1–6.
- [57] I. Tsamardinos, P. Charonyktakis, G. Papoutsoglou, G. Borboudakis, K. Lakiotaki, J.C. Zenklusen, H. Juhl, E. Chatzaki, V. Lagani, Just add data: Automated predictive modeling for knowledge discovery and feature selection, *NPJ Precision Oncology* 6 (1) (2022) 38.

- [58] M. Francia, J. Giovanelli, G. Pisano, HAMLET: A framework for human-centered AutoML via structured argumentation, *Future Generation Computer Systems* 142 (2023) 182–194.
- [59] X. Chen, Q. Lin, C. Luo, X. Li, H. Zhang, Y. Xu, Y. Dang, K. Sui, X. Zhang, B. Qiao, et al., Neural feature search: A neural architecture for automated feature engineering, in: *Proceedings of the 2019 IEEE International Conference on Data Mining (ICDM)*, IEEE, Piscataway, 2019, pp. 71–80.
- [60] T. Elsken, J.H. Metzen, F. Hutter, Neural architecture search: A survey, *The Journal of Machine Learning Research* 20 (1) (2019) 1997–2017.
- [61] Z.Y.-C. Liu, S. Roychowdhury, S. Tarlow, A. Nair, S. Badhe, T. Shah, AutoDC: Automated data-centric processing, 2021. <https://arxiv.org/pdf/2111.12548v1.pdf>.
- [62] M.K. Shende, A.E. Feijóo-Lorenzo, N.D. Bokde, cleanTS: Automated (AutoML) tool to clean univariate time series at microscale, *Neurocomputing* 500 (2022) 155–176.
- [63] C. Wang, S. Jin, Y. Guan, W. Liu, C. Qian, P. Luo, W. Ouyang, Pseudo-labeled auto-curriculum learning for semi-supervised keypoint localization, 2022. <https://arxiv.org/pdf/2201.08613v2.pdf>.
- [64] Y. Liu, X. Zhang, J. Kauttonen, G. Zhao, Uncertain label correction via auxiliary action unit graphs for facial expression recognition, 2022. <https://arxiv.org/abs/2204.11053>.
- [65] J.T. Hancock, T.M. Khoshgoftaar, Survey on categorical data for neural networks, *Journal of Big Data* 7 (1) (2020) 1–41.
- [66] K.M. Rao, G. Saikrishna, K. Supriya, Data preprocessing techniques: Emergence and selection towards machine learning models - A practical review using HPA dataset, *Multimedia Tools and Applications* 82 (2023) 1–20.
- [67] P. Cerda, G. Varoquaux, Encoding high-cardinality string categorical variables, *IEEE Transactions on Knowledge and Data Engineering* 34 (3) (2020) 1164–1176.
- [68] N.J. Teague, Missing data infill with automunge, 2022. <https://arxiv.org/pdf/2202.09484v1.pdf>.
- [69] S.-H. Jung, H.-S. Lee, J.-Y. Kim, C.-B. Sim, A novel on auto imputation and analysis prediction model of data missing scope based on machine learning, *Journal of Korea Multimedia Society* 25 (2) (2022) 257–268.
- [70] N.J. Teague, Tabular engineering with automunge, 2021. https://datacentralai.org/neurips21/papers/15_CameraReady_TabularEngineering_102621_Final.pdf.
- [71] X. Chu, I.F. Ilyas, S. Krishnan, J. Wang, Data cleaning: Overview and emerging challenges, in: *Proceedings of the 2016 International Conference on Management of Data*, ACM, New York, 2016, pp. 2201–2206.
- [72] T.N. Minh, M. Sinn, H.T. Lam, M. Wistuba, Automated image data preprocessing with deep reinforcement learning, 2021. <https://arxiv.org/pdf/1806.05886v2.pdf>.
- [73] W. Liang, G.A. Tadesse, D. Ho, L. Fei-Fei, M. Zaharia, C. Zhang, J. Zou, Advances, challenges and opportunities in creating data for trustworthy AI, *Nature Machine Intelligence* 4 (8) (2022) 669–677.
- [74] S. Krishnan, J. Wang, E. Wu, M.J. Franklin, K. Goldberg, ActiveClean: Interactive data cleaning for statistical modeling, *Proceedings of the VLDB Endowment* 9 (12) (2016) 948–959.
- [75] N. Erickson, J. Mueller, A. Shirkov, H. Zhang, P. Larroy, M. Li, A. Smola, AutoGluon-Tabular: Robust and accurate AutoML for structured data, 2020. <https://arxiv.org/pdf/2003.06505v1.pdf>.
- [76] M. Lam, S. Awasthi, H.J. Watson, J. Goldstein, G. Panagiotaropoulou, V. Trubetskoy, R. Karlsson, O. Frei, C.-C. Fan, W. De Witte, et al., RICOPILI: Rapid imputation for ConSortias PipelIne, *Bioinformatics* 36 (3) (2020) 930–933.
- [77] J. Barnard, X.-L. Meng, Applications of multiple imputation in medical studies: From AIDS to NHANES, *Statistical Methods in Medical Research* 8 (1) (1999) 17–36.
- [78] M. Puma, R.B. Olsen, S.H. Bell, C.S. Price, What to do when data are missing in group randomized controlled trials. NCEE 2009-0049, National Center for Education Evaluation and Regional Assistance, 2009. <https://api.semanticscholar.org/CorpusID:65471313>.
- [79] M. Lovrić, T. Đuričić, H. Hussain, B. Lucić, R. Kern, PyChemFlow: An automated pre-processing pipeline in Python for reproducible machine learning on chemical data, 2023. <https://chemrxiv.org/engage/chemrxiv/article-details/64b9082aae3d1a7b0d0a53dc>.
- [80] M. Bilal, G. Ali, M.W. Iqbal, M. Anwar, M.S.A. Malik, R.A. Kadir, Auto-Prep: Efficient and automated data preprocessing pipeline, *IEEE Access* 10 (2022) 107764–107784.
- [81] J. Yoon, J. Jordon, M. Schaar, GAIN: Missing data imputation using generative adversarial nets, in: *Proceedings of the International Conference on Machine Learning*, PMLR, 2018, pp. 5689–5698.
- [82] L. Gondara, K. Wang, MIDA: Multiple Imputation Using Denoising Autoencoders, In: D. Phung, V. Tseng, G. Webb, B. Ho, M. Ganji, L. Rashidi (Eds). *Advances. Advances in Knowledge Discovery and Data Mining*, Springer, 2018, pp. 260–272.
- [83] Z. Dai, Z. Bu, Q. Long, Multiple imputation via generative adversarial network for high-dimensional blockwise missing value problems, in: *Proceedings of the 2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA)*, IEEE, Piscataway, 2021, pp. 791–798.
- [84] S. Zhang, J. Chen, J. Chen, X. Chen, H. Huang, Data imputation in IoT using spatio-temporal variational auto-encoder, *Neurocomputing* 529 (2023) 23–32.
- [85] Q. Ma, X. Li, M. Bai, X. Wang, B. Ning, G. Li, MIVAE: Multiple imputation based on variational auto-encoder, *Engineering Applications of Artificial Intelligence* 123 (2023) 106270.
- [86] D.-H. Shin, S.-E. Lee, B.-U. Jeon, K. Chung, Missing value imputation model based on adversarial autoencoder using spatiotemporal feature extraction, *Intelligent Automation & Soft Computing* 37 (2) (2023) 1925–1940.
- [87] P.-A. Mattei, J. Frellsen, MIWAE: Deep generative modelling and imputation of incomplete data sets, in: *Proceedings of the International Conference on Machine Learning*, PMLR, 2019, pp. 4413–4423.
- [88] V. Fortuin, D. Baranchuk, G. Ratsch, S. Mandt, GP-VAE: Deep probabilistic time series imputation, in: *Proceedings of the International Conference on Artificial Intelligence and Statistics*, PMLR, 2020, pp. 1651–1661.
- [89] D. Jarrett, B.C. Cebere, T. Liu, A. Curth, M. van der Schaar, HyperImpute: Generalized iterative imputation with automatic model selection, in: *Proceedings of the 39th International Conference on Machine Learning*, PMLR, 2022, pp. 9916–9937.
- [90] M. Feurer, K. Eggensperger, S. Falkner, M. Lindauer, F. Hutter, Auto-Sklearn 2.0: Hands-free AutoML via meta-learning, 2020. <https://arxiv.org/pdf/2007.04074v3.pdf>.
- [91] O. Kramer, *Machine Learning for Evolution Strategies*, Scikit-Learn, Springer, 2016, pp. 45–53.
- [92] R. I. Esteve, *The Azure Data Lakehouse Toolkit*, Databricks, Springer, 2022, pp. 83–139.
- [93] A. Elangovan, J. He, K. Verspoor, Memorization vs. generalization: Quantifying data leakage in NLP performance evaluation, 2021. <https://arxiv.org/pdf/2102.01818v1.pdf>.
- [94] T.T. Khuat, D.J. Kedziora, B. Gabrys, The roles and modes of human interactions with automated machine learning systems: A critical review and perspectives, *Foundations and Trends® in Human-Computer Interaction* 17 (3–4) (2023) 195–387.
- [95] B. Wu, C.A. Knoblock, Maximizing correctness with minimal user effort to learn data transformations, in: *Proceedings of the 21st International Conference on Intelligent User Interfaces*, ACM, New York, 2016, pp. 375–384.
- [96] R.S. Olson, J.H. Moore, TPOT: A tree-based pipeline optimization tool for automating machine learning, in: *Proceedings of the Workshop on Automatic Machine Learning*, PMLR, 2016, pp. 66–74.
- [97] H. Jin, Q. Song, X. Hu, Auto-Keras: An efficient neural architecture search system, in: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ACM, New York, 2019, pp. 1946–1956.
- [98] F. Neutatz, B. Chen, Y. Alkhatib, J. Ye, Z. Abedjan, Data cleaning and AutoML: Would an optimizer choose to clean? *Datenbank-Spektrum* 22 (2) (2022) 121–130.
- [99] M. Bahrami, W.-P. Chen, L. Liu, M. Prasad, BERT-Sort, A Zero-shot MLM semantic encoder on ordinal features for AutoML, in: *Proceedings of the International Conference on Automated Machine Learning*, PMLR, 2022, pp. 1–26.
- [100] E. LeDell, S. Poirier, H2O AutoML: Scalable automatic machine learning, in: *Proceedings of the AutoML Workshop at ICML, ICML*, 2020.

- [101] A. Vakhrushev, A. Ryzhkov, M. Savchenko, D. Simakov, R. Damdinov, A. Tuzhilin, LightAutoML: AutoML solution for a large financial services ecosystem, 2021. <https://arxiv.org/pdf/2109.01528v1.pdf>.
- [102] M.K. Dahouda, I. Joe, A deep-learned embedding technique for categorical features encoding, *IEEE Access* 9 (2021) 114381–114391.
- [103] P. Li, X. Rao, J. Blase, Y. Zhang, X. Chu, C. Zhang, CleanML: A study for evaluating the impact of data cleaning on ML classification tasks, in: *Proceedings of the 2021 IEEE 37th International Conference on Data Engineering (ICDE)*, IEEE, Piscataway, 2021, pp. 13–24.
- [104] L. Berti-Equille, Learn2Clean: Optimizing the sequence of tasks for web data preparation, in: *Proceedings of the The World Wide Web Conference*, ACM, New York, 2019, pp. 2580–2586.
- [105] S. Krishnan, M.J. Franklin, K. Goldberg, E. Wu, BoostClean: Automated error detection and repair for machine learning, 2017. <https://arxiv.org/pdf/1711.01299.pdf>.
- [106] F. Neutatz, B. Chen, Z. Abedjan, E. Wu, From cleaning before ML to cleaning for ML, *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering* 44 (1) (2021) 24–41.
- [107] I. Gemp, G. Theodorou, M. Ghavamzadeh, Automated data cleansing through meta-learning, in: *Proceedings of the Twenty-Ninth Innovative Applications of Artificial Intelligence Conference*, AAAI, Melon Park, 2017, pp. 4760–4761.
- [108] Z. Miao, Y. Li, X. Wang, Rotom: A meta-learned data augmentation framework for entity matching, data cleaning, text classification, and beyond, in: *Proceedings of the 2021 International Conference on Management of Data*, ACM, New York, 2021, pp. 1303–1316.
- [109] M.M. Morsali, H. Mohammadzade, S.B. Shouraki, Face: Fast, accurate and context-aware audio annotation and classification, 2023. <https://arxiv.org/pdf/2303.03666v1.pdf>.
- [110] J. Zhang, M.A. Mazurowski, B.C. Allen, B. Wildman-Tobriner, Multistep automated data labelling procedure (MADLaP) for thyroid nodules on ultrasound: An artificial intelligence approach for automating image annotation, *Artificial Intelligence in Medicine* 141 (2023) 102553.
- [111] A. Kaushik, D. Dunham, Z. He, M. Manohar, M. Desai, K.C. Nadeau, S. Andorf, CyAnno: A semi-automated approach for cell type annotation of mass cytometry datasets, *Bioinformatics* 37 (22) (2021) 4164–4171.
- [112] K.G. Ince, A. Koksai, A. Fazla, A.A. Alatan, Semi-automatic annotation for visual object tracking, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, IEEE, Piscataway, 2021, pp. 1233–1239.
- [113] A. Chakravarty, J. Sivaswamy, An assistive annotation system for retinal images, in: *Proceedings of the 2015 IEEE 12th International Symposium on Biomedical Imaging (ISBI)*, IEEE, Piscataway, 2015, pp. 1506–1509.
- [114] X. Ke, J. Zou, Y. Niu, End-to-end automatic image annotation based on deep CNN and multi-label data augmentation, *IEEE Transactions on Multimedia* 21 (8) (2019) 2093–2106.
- [115] X. Qian, C. Liu, X. Qi, S.-C. Tan, E. Lam, N. Wong, Context-aware transformer for 3D point cloud automatic annotation, 2023. <https://arxiv.org/pdf/2303.14893v1.pdf>.
- [116] A. Ligocki, A. Jelinek, L. Zalud, E. Rahtu, Fully automated DCNN-based thermal images annotation using neural network pretrained on RGB data, *Sensors* 21 (4) (2021) 1552.
- [117] W. Zhang, H. Hu, H. Hu, J. Yu, Automatic image annotation via category labels, *Multimedia Tools and Applications* 79 (2020) 11421–11435.
- [118] D.A. Wood, J. Lynch, S. Kafiabadi, E. Guilhem, A. Al Busaidi, A. Montvila, T. Varsavsky, J. Siddiqui, N. Gadapa, M. Townend, et al., Automated labelling using an attention model for radiology reports of MRI scans (ALARM), in: *Proceedings of the Medical Imaging with Deep Learning*, PMLR, 2020, pp. 811–826.
- [119] X. Chen, B. Mersch, L. Nunes, R. Marcuzzi, I. Vizzo, J. Behley, C. Stachniss, Automatic labeling to generate training data for online lidar-based moving object segmentation, *IEEE Robotics and Automation Letters* 7 (3) (2022) 6107–6114.
- [120] T. Anwar, COVID19 diagnosis using AutoML from 3D CT scans, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, IEEE, Piscataway, 2021, pp. 503–507.
- [121] M. Guillerme, R.K. Billones, A. Bandala, R.R. Vicerra, E. Sybingco, E.P. Dadios, A. Fillone, Implementation of automated annotation through mask RCNN object detection model in CVAT using AWS EC2 instance, in: *Proceedings of the 2020 IEEE Region 10 Conference (TENCON)*, IEEE, Piscataway, 2020, pp. 708–713.
- [122] M. Jiu, H. Sahbi, Context-aware deep kernel networks for image annotation, *Neurocomputing* 474 (2022) 154–167.
- [123] G. Yang, J. Zhang, Y. Zhang, B. Wu, Y. Yang, Probabilistic modeling of semantic ambiguity for scene graph generation, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, IEEE, Piscataway, 2021, pp. 12527–12536.
- [124] B. Martinez, M.F. Valstar, Advances, challenges, and opportunities in automatic facial expression recognition, *Advances in Face Detection and Facial Image Analysis* (2016) 63–100.
- [125] K. Goyle, Q. Xie, V. Goyle, DataAssist: A machine learning approach to data cleaning and preparation, 2023. <https://arxiv.org/pdf/2307.07119v2.pdf>.
- [126] P. Li, Z. Chen, X. Chu, K. Rong, DiffPrep: Differentiable data preprocessing pipeline search for learning over tabular data, *Proceedings of the ACM on Management of Data* 1 (2) (2023) 1–26.
- [127] J.A. Valeri, L.R. Soenksen, K.M. Collins, P. Ramesh, G. Cai, R. Powers, N.M. Angenent-Mari, D.M. Camacho, F. Wong, T.K. Lu, et al., BioAutoMATED: An end-to-end automated machine learning tool for explanation and design of biological sequences, *Cell Systems* 14 (6) (2023) 525–542.
- [128] S. Krishnan, E. Wu, AlphaClean: Automatic generation of data cleaning pipelines, 2019. <https://arxiv.org/pdf/1904.11827v2.pdf>.
- [129] S. Chen, N. Tang, J. Fan, X. Yan, C. Chai, G. Li, X. Du, HAIPipe: Combining human-generated and machine-generated pipelines for data preparation, *Proceedings of the ACM on Management of Data* 1 (1) (2023) 1–26.
- [130] Y.-Y. Kim, K. Song, J. Jang, I.-C. Moon, LADA: Look-ahead data acquisition via augmentation for deep active learning, *Advances in Neural Information Processing Systems* 34 (2021) 22919–22930.
- [131] D. Sakkos, E.S.L. Ho, H.P.H. Shum, G. Elvin, Image editing-based data augmentation for illumination-insensitive background subtraction, *Journal of Enterprise Information Management* 36 (3) (2023) 818–838.
- [132] S. Schelter, J. Stoyanovich, Taming technical bias in machine learning pipelines, *Bulletin of the Technical Committee on Data Engineering* 43 (4) (2020) 39–50.
- [133] R. Jiang, S. Mei, Polar coordinate convolutional neural network: From rotation-invariance to translation-invariance, in: *Proceedings of the 2019 IEEE International Conference on Image Processing (ICIP)*, IEEE, Piscataway, 2019, pp. 355–359.
- [134] B. Hilprecht, C. Hammacher, E.S. Reis, M. Abdelaal, C. Binnig, DiffML: End-to-end differentiable ML pipelines, in: *Proceedings of the Seventh Workshop on Data Management for End-To-End Machine Learning*, ACM, New York, 2023, pp. 1–7.
- [135] A. Mumuni, F. Mumuni, Data augmentation: A comprehensive survey of modern approaches, *Array* 16 (2022) 100258.
- [136] T. Niu, M. Bansal, Automatically learning data augmentation policies for dialogue tasks, 2019. <https://arxiv.org/pdf/1909.12868v1.pdf>.
- [137] S. Ren, J. Zhang, L. Li, X. Sun, J. Zhou, Text AutoAugment: Learning compositional augmentation policy for text classification, 2021. <https://arxiv.org/pdf/2109.00523v1.pdf>.
- [138] E.D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, Q.V. Le, AutoAugment: Learning augmentation strategies from data, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, IEEE, Piscataway, 2019, pp. 113–123.
- [139] R. Hataya, J. Zdenek, K. Yoshizoe, H. Nakayama, Faster AutoAugment: Learning augmentation strategies using backpropagation, in: *Proceedings of the European Conference on Computer Vision*, Springer, 2020, pp. 1–16.
- [140] S. Lin, T. Yu, R. Feng, X. Li, X. Jin, Z. Chen, Local Patch AutoAugment with multi-agent collaboration, *IEEE Transactions on Multimedia* 26 (2023), 724–236.
- [141] E.D. Cubuk, B. Zoph, J. Shlens, Q.V. Le, RandAugment: Practical automated data augmentation with a reduced search space, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, IEEE, Piscataway, 2020, pp. 702–703.
- [142] S. Lim, I. Kim, T. Kim, C. Kim, S. Kim, Fast AutoAugment, in: *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, Curran Associates Inc., New York, 2019, pp. 6665–6675, 32 (2019).
- [143] A. Liu, Z. Huang, Z. Huang, N. Wang, Direct differentiable augmentation search, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, IEEE, Piscataway, 2021, pp. 12219–12228.
- [144] P. Chen, S. Liu, H. Zhao, J. Jia, GridMask data augmentation, 2020. <https://arxiv.org/pdf/2001.04086v2.pdf>.

- [145] Z. Tang, Y. Gao, L. Karlinsky, P. Sattigeri, R. Feris, D. Metaxas, OnlineAugment: Online data augmentation with less domain knowledge, in: *Proceedings of the European Conference on Computer Vision*, Springer, 2020, pp. 313–329.
- [146] Y. Gao, Z. Tang, M. Zhou, D. Metaxas, Enabling data diversity: Efficient automatic augmentation via regularized adversarial training, in: *Proceedings of the International Conference on Information Processing in Medical Imaging*, Springer, 2021, pp. 85–97.
- [147] H. Miao, L.T. Rahman, Multi-class traffic sign classification using autoaugment and spatial transformer, 2020. <https://api.semanticscholar.org/CorpusID:236445175>.
- [148] A. Zhao, G. Balakrishnan, F. Durand, J.V. Guttag, A.V. Dalca, Data augmentation using learned transformations for one-shot medical image segmentation, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, IEEE, Piscataway, 2019, pp. 8543–8553.
- [149] X. Zhang, Q. Wang, J. Zhang, Z. Zhong, Adversarial AutoAugment, 2019. <https://arxiv.org/pdf/1912.11188v1.pdf>.
- [150] V. Chinbat, S.-H. Bae, GA3N: Generative adversarial AutoAugment network, *Pattern Recognition* 127 (2022) 108637.
- [151] X. Peng, Z. Tang, F. Yang, R.S. Feris, D. Metaxas, Jointly optimize data augmentation and network training: Adversarial data augmentation in human pose estimation, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, Piscataway, 2018, pp. 2226–2234.
- [152] Y. Liu, W. Tian, S. Li, Meta-data augmentation based search strategy through generative adversarial network for AutoML model selection, in: *Proceedings of the Pacific-asia Conference on Knowledge Discovery and Data Mining*, Springer, 2021, pp. 312–324.
- [153] K. Tian, C. Lin, M. Sun, L. Zhou, J. Yan, W. Ouyang, Improving auto-augment via augmentation-wise weight sharing, *Advances in Neural Information Processing Systems* 33 (2020) 19088–19098.
- [154] C.-T. Chu, M. Rohmatillah, C.-H. Lee, J.-T. Chien, Augmentation strategy optimization for language understanding, in: *Proceedings of the ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, IEEE, Piscataway, 2022, pp. 7952–7956.
- [155] C. Zhang, X. Li, Z. Zhang, J. Cui, B. Yang, BO-Aug: Learning data augmentation policies via Bayesian optimization, *Applied Intelligence* 53 (8) (2023) 8978–8993.
- [156] T.-Y. Hu, A. Shrivastava, J.-H.R. Chang, H. Koppula, S. Braun, K. Hwang, O. Kalinli, O. Tuzel, SapAugment: Learning a sample adaptive policy for data augmentation, in: *Proceedings of the ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, IEEE, Piscataway, 2021, pp. 4040–4044.
- [157] A. Terauchi, N. Mori, Evolutionary approach for autoaugment using the thermodynamical genetic algorithm, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, AAAI, Melon Park, 2021, pp. 9851–9858.
- [158] S. Cheng, Z. Leng, E.D. Cubuk, B. Zoph, C. Bai, J. Ngiam, Y. Song, B. Caine, V. Vasudevan, C. Li, et al., Improving 3D object detection through progressive population based augmentation, in: *Proceedings of the European Conference on Computer Vision*, Springer, 2020, pp. 279–294.
- [159] D. Ho, E. Liang, X. Chen, I. Stoica, P. Abbeel, Population based augmentation: Efficient learning of augmentation policy schedules, in: *Proceedings of the International Conference on Machine Learning*, PMLR, 2019, pp. 2731–2741.
- [160] T.-H. Cheung, D.-Y. Yeung, MODALS: Modality-agnostic automated data augmentation in the latent space, in: *Proceedings of the International Conference on Learning Representations*, 2021. <https://openreview.net/forum?id=XjYgR6gbCEc>.
- [161] R.J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, *Machine Learning* 8 (3) (1992) 229–256.
- [162] C. Lin, M. Guo, C. Li, X. Yuan, W. Wu, J. Yan, D. Lin, W. Ouyang, Online hyper-parameter learning for auto-augmentation strategy, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, IEEE, Piscataway, 2019, pp. 6579–6588.
- [163] R. Hataya, J. Zdenek, K. Yoshizoe, H. Nakayama, Meta approach to data augmentation optimization, in: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, IEEE, Piscataway, 2022, pp. 2574–2583.
- [164] Y. Li, G. Hu, Y. Wang, T. Hospedales, N.M. Robertson, Y. Yang, DADA: Differentiable automatic data augmentation, 2020. <https://arxiv.org/pdf/2003.03780v3.pdf>.
- [165] M. Momeny, A.A. Neshat, A. Gholizadeh, A. Jafarnejad, E. Rahmzadeh, M. Marhamati, B. Moradi, A. Ghafoorifar, Y.-D. Zhang, Greedy Autoaugment for classification of mycobacterium tuberculosis image via generalized deep CNN using mixed pooling based on minimum square rough entropy, *Computers in Biology and Medicine* 141 (2022) 105175.
- [166] A. Naghizadeh, M. Abavisani, D.N. Metaxas, Greedy AutoAugment, *Pattern Recognition Letters* 138 (2020) 624–630.
- [167] T.C. LingChen, A. Khonsari, A. Lashkari, M.R. Nazari, J.S. Sambee, M.A. Nascimento, UniformAugment: A search-free probabilistic data augmentation approach, 2020. <https://arxiv.org/pdf/2003.14348v1.pdf>.
- [168] A. Caillon, P. Esling, Rave, A variational autoencoder for fast and high-quality neural audio synthesis, 2021. <https://arxiv.org/pdf/2111.05011v2.pdf>.
- [169] N. Park, M. Mohammadi, K. Gorde, S. Jajodia, H. Park, Y. Kim, Data synthesis based on generative adversarial networks, 2018. <https://arxiv.org/pdf/1806.03384.pdf>.
- [170] F.-A. Croitoru, V. Hondru, R.T. Ionescu, M. Shah, Diffusion models in vision: A survey, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45 (9) (2023) 10850–10869.
- [171] J. Yu, Y. Xu, J.Y. Koh, T. Luong, G. Baid, Z. Wang, V. Vasudevan, A. Ku, Y. Yang, B.K. Ayan, et al., Scaling autoregressive models for content-rich text-to-image generation, 2022. <https://arxiv.org/pdf/2206.10789v1.pdf>.
- [172] D. Lee, C. Kim, S. Kim, M. Cho, W.-S. Han, Autoregressive image generation using residual quantization, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, IEEE, Piscataway, 2022, pp. 11523–11532.
- [173] Y. Chang, X. Wang, J. Wang, Y. Wu, K. Zhu, H. Chen, L. Yang, X. Yi, C. Wang, Y. Wang, et al., A survey on evaluation of large language models, 2023. <https://arxiv.org/pdf/2307.03109v7.pdf>.
- [174] K.M. Yoo, D. Park, J. Kang, S.-W. Lee, W. Park, GPT3Mix: Leveraging large-scale language models for text augmentation, 2021. <https://arxiv.org/pdf/2104.08826v2.pdf>.
- [175] B. Peng, C. Zhu, M. Zeng, J. Gao, Data augmentation for spoken language understanding via pretrained language models, 2020. <https://arxiv.org/pdf/2004.13952v2.pdf>.
- [176] J. Kulhánek, V. Hudeček, T. Nekvinda, O. Dušek, AuGPT: Dialogue with pre-trained language models and data augmentation, 2022. <https://arxiv.org/pdf/2102.05126v3.pdf>.
- [177] T. Schick, H. Schutze, Generating datasets with pretrained language models, 2021. <https://arxiv.org/pdf/2104.07540v3.pdf>.
- [178] N. Carlini, F. Tramèr, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, Ú. Erlingsson, et al., Extracting training data from large language models, in: *Proceedings of the 30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2633–2650.
- [179] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: *Proceedings of the 31st International Conference on Neural Information Processing Systems*, Curran Associates Inc., New York, 2017, pp. 6000–6010.
- [180] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, S. Ganguli, Deep unsupervised learning using nonequilibrium thermodynamics, in: *Proceedings of the International Conference on Machine Learning*, PMLR, 2015, pp. 2256–2265.
- [181] L. Bonifacio, H. Abonizio, M. Fadaee, R. Nogueira, InPars: Data augmentation for information retrieval using large language models, 2022. <https://arxiv.org/pdf/2202.05144v1.pdf>.
- [182] H. Li, Y. Yang, M. Chang, S. Chen, H. Feng, Z. Xu, Q. Li, Y. Chen, SRDiff: Single image super-resolution with diffusion probabilistic models, *Neurocomputing* 479 (2022) 47–59.
- [183] S. Welker, H.N. Chapman, T. Gerkmann, DriftRec: Adapting diffusion models to blind image restoration tasks, 2022. <https://arxiv.org/pdf/2211.06757v2.pdf>.
- [184] G. Kim, T. Kwon, J.C. Ye, DiffusionCLIP: Text-guided diffusion models for robust image manipulation, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, IEEE, Piscataway, 2022, pp. 2426–2435.
- [185] Introducing ChatGPT. <https://openai.com/blog/chatgpt>.
- [186] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., LLaMA: Open and efficient foundation language models, 2023. <https://arxiv.org/pdf/2302.13971.pdf>.

- [187] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, 2018. <https://arxiv.org/pdf/1810.04805.pdf>.
- [188] S. Ubani, S.O. Polat, R. Nielsen, ZeroShotDataAug: Generating and augmenting training data with ChatGPT, 2023. <https://arxiv.org/pdf/2304.14334v1.pdf>.
- [189] A. Radford, J.W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al., Learning transferable visual models from natural language supervision, in: Proceedings of the International Conference on Machine Learning, PMLR, 2021, pp. 8748–8763.
- [190] O. Patashnik, Z. Wu, E. Shechtman, D. Cohen-Or, D. Lischinski, StyleCLIP: Text-driven manipulation of stylegan imagery, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, IEEE, Piscataway, 2021, pp. 2085–2094.
- [191] R. Gal, O. Patashnik, H. Maron, A.H. Bermano, G. Chechik, D. Cohen-Or, StyleGAN-NADA: CLIP-guided domain adaptation of image generators, ACM Transactions on Graphics 41 (4) (2022) 1–13.
- [192] H. Sahak, D. Watson, C. Saharia, D. Fleet, Denoising diffusion probabilistic models for robust image super-resolution in the wild, 2023. <https://arxiv.org/pdf/2302.07864v1.pdf>.
- [193] A. Nichol, P. Dhariwal, A. Ramesh, P. Shyam, P. Mishkin, B. McGrew, I. Sutskever, M. Chen, GLIDE: Towards photorealistic image generation and editing with text-guided diffusion models, 2021. <https://arxiv.org/pdf/2112.10741v3.pdf>.
- [194] C.V. Krishna, K. Venkatesh, S. Nibin, H. Vishakh, M.G. Nair, Segment-based, user-generated image styling with neural style transfer, in: Proceedings of the 2023 International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE), IEEE, Piscataway, 2023, pp. 1–6.
- [195] Q. Nguyen, T. Vu, A. Tran, K. Nguyen, Dataset Diffusion: Diffusion-based synthetic dataset generation for pixel-level semantic segmentation, 2023. <https://arxiv.org/pdf/2309.14303.pdf>.
- [196] Y. Yu, Y. Zhuang, J. Zhang, Y. Meng, A. Ratner, R. Krishna, J. Shen, C. Zhang, Large language model as attributed training data generator: A tale of diversity and bias, 2023. <https://arxiv.org/pdf/2306.15895v1.pdf>.
- [197] N. Liu, S. Li, Y. Du, A. Torralba, J.B. Tenenbaum, Compositional visual generation with composable diffusion models, in: Proceedings of the European Conference on Computer Vision, Springer, 2022, pp. 423–439.
- [198] S.-I. Cheng, Y.-J. Chen, W.-C. Chiu, H.-Y. Tseng, H.-Y. Lee, Adaptively-realistic image generation from stroke and sketch with diffusion model, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, IEEE, Piscataway, 2023, pp. 4054–4062.
- [199] J.Z. Wu, Y. Ge, X. Wang, S.W. Lei, Y. Gu, Y. Shi, W. Hsu, Y. Shan, X. Qie, M.Z. Shou, Tune-A-Video: One-shot tuning of image diffusion models for text-to-video generation, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, IEEE, Piscataway, 2023, pp. 7568–7599.
- [200] Z. Luo, D. Chen, Y. Zhang, Y. Huang, L. Wang, Y. Shen, D. Zhao, J. Zhou, T. Tan, VideoFusion: Decomposed diffusion models for high-quality video generation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, IEEE, Piscataway, 2023, pp. 10209–10218.
- [201] K. Mei, V. Patel, VIDM: Video implicit diffusion models, in: Proceedings of the AAAI Conference on Artificial Intelligence, AAAI, 2023, pp. 9117–9125.
- [202] S. Gong, M. Li, J. Feng, Z. Wu, L. Kong, DiffuSeq: Sequence to sequence text generation with diffusion models, 2022. <https://arxiv.org/pdf/2210.08933v3.pdf>.
- [203] Y. Leng, Z. Chen, J. Guo, H. Liu, J. Chen, X. Tan, D. Mandic, L. He, X. Li, T. Qin, S. Zhao, T.-Y. Liu, BinauralGrad: A two-stage conditional diffusion probabilistic model for binaural audio synthesis, Advances in Neural Information Processing Systems 35 (2022) 23689–23700.
- [204] Z. Kong, W. Ping, J. Huang, K. Zhao, B. Catanzaro, DiffWave: A versatile diffusion model for audio synthesis, 2020. <https://arxiv.org/pdf/2009.09761v3.pdf>.
- [205] Y. Tashiro, J. Song, Y. Song, S. Ermon, CSDI: Conditional score-based diffusion models for probabilistic time series imputation, Advances in Neural Information Processing Systems 34 (2021) 24804–24816.
- [206] N. Suh, X. Lin, D.-Y. Hsieh, M. Honarkhah, G. Cheng, AutoDiff: Combining auto-encoder and diffusion model for tabular data synthesizing, 2023. <https://arxiv.org/pdf/2310.15479.pdf>.
- [207] A. Kotelnikov, D. Baranchuk, I. Rubachev, A. Babenko, TabDDPM: Modelling tabular data with diffusion models, in: Proceedings of the International Conference on Machine Learning, PMLR, 2023, pp. 17564–17579.
- [208] T. Sattarov, M. Schreyer, D. Borth, FinDiff: Diffusion models for financial tabular data generation, in: Proceedings of the 4th ACM International Conference on AI in Finance, ACM, New York, 2023, pp. 64–72.
- [209] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, M. Chen, Hierarchical text-conditional image generation with CLIP latents, 2022. <https://arxiv.org/pdf/2204.06125v1.pdf>.
- [210] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer, High-resolution image synthesis with latent diffusion models, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, IEEE, Piscataway, 2022, pp. 10684–10695.
- [211] C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E.L. Denton, K. Ghasemipour, R. Gontijo Lopes, B. Karagol Ayan, T. Salimans, et al., Photorealistic text-to-image diffusion models with deep language understanding, Advances in Neural Information Processing Systems 35 (2022) 36479–36494.
- [212] O. Avrahami, O. Fried, D. Lischinski, Blended latent diffusion, ACM Transactions on Graphics 42 (4) (2023) 1–11.
- [213] O. Bar-Tal, L. Yariv, Y. Lipman, T. Dekel, MultiDiffusion: Fusing diffusion paths for controlled image generation, in: Proceedings of the 40th International Conference on Machine Learning, JMLR, 2023, pp. 1737–1752.
- [214] T. Wang, J.-Y. Zhu, A. Torralba, A.A. Efros, Dataset distillation, 2018. <https://arxiv.org/pdf/1811.10959v3.pdf>.
- [215] R. Yu, S. Liu, X. Wang, Dataset distillation: A comprehensive review, 2023. <https://arxiv.org/pdf/2301.07014v2.pdf>.
- [216] O. Bohdal, Y. Yang, T. Hospedales, Flexible dataset distillation: Learn labels instead of images, 2022. <https://arxiv.org/pdf/2006.08572v3.pdf>.
- [217] I. Sucholutsky, M. Schonlau, Soft-label dataset distillation and text dataset distillation, in: Proceedings of the 2021 International Joint Conference on Neural Networks (IJCNN), IEEE, Piscataway, 2021, pp. 1–8.
- [218] Y. Li, J. Yang, Y. Song, L. Cao, J. Luo, L.-J. Li, Learning from noisy labels with distillation, in: Proceedings of the IEEE International Conference on Computer Vision, IEEE, Piscataway, 2017, pp. 1910–1918.
- [219] L. Li, P. Dong, Z. Wei, Y. Yang, Automated knowledge distillation via Monte Carlo tree search, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, IEEE, Piscataway, 2023, pp. 17413–17424.
- [220] H. He, X. Shi, J. Mueller, S. Zha, M. Li, G. Karypis, Towards automated distillation: A systematic study of knowledge distillation in natural language processing, 2022. <https://www.amazon.science/publications/towards-automated-distillation-a-systematic-study-of-knowledge-distillation-in-natural-language-processing>.
- [221] A. Krizhevsky, G. Hinton, Learning multiple layers of features from tiny images, Communications of the ACM 60 (6) (2009) 84–90.
- [222] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, ImageNet: A large-scale hierarchical image database, in: Proceedings of the 2009 IEEE Conference on Computer Vision and Pattern Recognition, IEEE, Piscataway, 2009, pp. 248–255.
- [223] S. Zagoruyko, N. Komodakis, Wide residual networks, 2016. <https://arxiv.org/pdf/1605.07146.pdf>.
- [224] S.G. Muller, F. Hutter, TrivialAugment: Tuning-free yet state-of-the-art data augmentation, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, IEEE, Piscataway, 2021, pp. 774–782.
- [225] Z. Liu, H. Jin, T.-H. Wang, K. Zhou, X. Hu, DivAug: Plug-in automated data augmentation with explicit diversity maximization, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, IEEE, Piscataway, 2021, pp. 4762–4770.
- [226] F. Zhou, J. Li, C. Xie, F. Chen, L. Hong, R. Sun, Z. Li, MetaAugment: Sample-aware data augmentation policy learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, AAAI, 2021, pp. 11097–11105.
- [227] L. Li, A. Li, A2-Aug: Adaptive automated data augmentation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, IEEE, Piscataway, 2023, pp. 2266–2273.
- [228] C. Gong, D. Wang, M. Li, V. Chandra, Q. Liu, KeepAugment: A simple information-preserving data augmentation approach, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, IEEE, Piscataway, 2021, pp. 1055–1064.
- [229] Y. Zheng, Z. Zhang, S. Yan, M. Zhang, Deep AutoAugment, 2022. <https://arxiv.org/pdf/2203.06172v2.pdf>.
- [230] G. Huang, Y. Sun, Z. Liu, D. Sedra, K.Q. Weinberger, Deep networks with stochastic depth, in: Proceedings of the European Conference on Computer Vision, Springer, 2016, pp. 646–661.
- [231] Z. Zhong, L. Zheng, G. Kang, S. Li, Y. Yang, Random erasing data augmentation, in: Proceedings of the AAAI Conference on Artificial Intelligence, AAAI, 2020, pp. 13001–13008.

- [232] R. Takahashi, T. Matsubara, K. Uehara, RICAP: Random image cropping and patching data augmentation for deep CNNs, in: *Proceedings of the Asian Conference on Machine Learning*, PMLR, 2018, pp. 786–798.
- [233] A. Uddin, M. Monira, W. Shin, T. Chung, S.-H. Bae, SaliencyMix: A saliency guided data augmentation strategy for better regularization, 2020. <https://arxiv.org/pdf/2006.01791v2.pdf>.
- [234] J.-H. Lee, M.Z. Zaheer, M. Astrid, S.-I. Lee, SmoothMix: A simple yet effective data augmentation to train robust classifiers, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, IEEE, Piscataway, 2020, pp. 756–757.
- [235] V. Verma, A. Lamb, C. Beckham, A. Najafi, I. Mitliagkas, D. Lopez-Paz, Y. Bengio, Manifold Mixup: Better representations by interpolating hidden states, in: *Proceedings of the International Conference on Machine Learning*, PMLR, 2019, pp. 6438–6447.
- [236] T. DeVries, G.W. Taylor, Improved regularization of convolutional neural networks with cutout, 2017. <https://arxiv.org/pdf/1708.04552.pdf>.
- [237] H. Zhang, M. Cisse, Y.N. Dauphin, D. Lopez-Paz, Mixup: Beyond empirical risk minimization, 2017. <https://arxiv.org/pdf/1710.09412v2.pdf>.
- [238] E. Harris, A. Marcu, M. Painter, M. Niranjana, A. Prugel-Bennett, J. Hare, FMix: Enhancing mixed sample data augmentation, 2020. <https://arxiv.org/pdf/2002.12047v3.pdf>.
- [239] B.A. Pimentel, A.C. De Carvalho, A new data characterization for selecting clustering algorithms using meta-learning, *Information Sciences* 477 (2019) 203–219.
- [240] Y. Li, Y. Shen, W. Zhang, C. Zhang, B. Cui, VolcanoML: Speeding up end-to-end AutoML via scalable search space decomposition, *The VLDB Journal* 32 (2) (2023) 389–413.
- [241] S. Liu, P. Ram, D. Vijaykeerthy, D. Bouneffouf, G. Bramble, H. Samulowitz, D. Wang, A. Conn, A. Gray, An ADMM based framework for AutoML pipeline configuration, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, AAAI, Melon Park, 2020, pp. 4892–4899.
- [242] E. Bisong, Google AutoML, Cloud Vision, in: *Building Machine Learning and Deep Learning Models on Google Cloud Platform*, 2019, pp. 581–598.
- [243] C. Thornton, F. Hutter, H.H. Hoos, K. Leyton-Brown, Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms, in: *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, New York, 2013, pp. 847–855.
- [244] Featuretools. <https://www.featuretools.com/>.
- [245] F. Horn, R. Pack, M. Rieger, The autofeat Python library for automated feature engineering and selection, in: *Proceedings of the Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, Springer, 2019, pp. 111–120.
- [246] M. Christ, N. Braun, J. Neuffer, A.W. Kempa-Liehr, Time series feature extraction on basis of scalable hypothesis tests (tsfresh - a Python package), *Neurocomputing* 307 (2018) 72–77.
- [247] N.A. Roque, N. Ram, tsfeaturex: An R package for automating time series feature extraction, *Journal of Open Source Software* 4 (37) (2019).
- [248] M. Barandas, D. Folgado, L. Fernandes, S. Santos, M. Abreu, P. Bota, H. Liu, T. Schultz, H. Gamboa, TSFEL: Time series feature extraction library, *SoftwareX* 11 (2020) 100456.
- [249] B. Peng, S. Wan, Y. Bi, B. Xue, M. Zhang, Automatic feature extraction and construction using genetic programming for rotating machinery fault diagnosis, *IEEE Transactions on Cybernetics* 51 (10) (2020) 4909–4923.
- [250] A. De Brabandere, P. Robberechts, T. Op De Beeck, J. Davis, Automating feature construction for multi-view time series data, in: *Proceedings of the ECMLPKDD Workshop on Automating Data Science*, 2019, pp. 16–20.
- [251] K. Lillywhite, D.-J. Lee, B. Tippetts, J. Archibald, A feature construction method for general object recognition, *Pattern Recognition* 46 (12) (2013) 3300–3314.
- [252] U. Kamath, K. De Jong, A. Shehu, Effective automated feature construction and selection for classification of biological sequences, *PLoS One* 9 (7) (2014) e99982.
- [253] A. De Brabandere, T. Op De Beeck, K. Hendrickx, W. Meert, J. Davis, TSFuse: Automated feature construction for multiple time series data, *Machine Learning*, 2022, pp. 1–56.
- [254] I. Guyon, S. Gunn, M. Nikravesh, L.A. Zadeh, *Feature Extraction: Foundations and Applications*, Springer, 2008.
- [255] I. Mierswa, K. Morik, Automatic feature extraction for classifying audio data, *Machine Learning* 58 (2) (2005) 127–149.
- [256] N. Sumonja, B. Gemovic, N. Veljkovic, V. Perovic, Automated feature engineering improves prediction of protein-protein interactions, *Amino Acids* 51 (8) (2019) 1187–1200.
- [257] J. Zhang, Machine learning with feature selection using principal component analysis for malware detection: A case study, 2019. <https://arxiv.org/pdf/1902.03639v1.pdf>.
- [258] L. Pion-Tonachini, K. Kreutz-Delgado, S. Makeig, ICLabel: An automated electroencephalographic independent component classifier, dataset, and website, *Neuroimage* 198 (2019) 181–197.
- [259] W. Long, Z. Lu, L. Cui, Deep learning-based feature engineering for stock price movement prediction, *Knowledge-Based Systems* 164 (2019) 163–173.
- [260] V.P. Rathi, S. Palani, Brain tumor MRI image classification with feature selection and extraction using linear discriminant analysis, 2012. <https://arxiv.org/ftp/arxiv/papers/1208/1208.2128.pdf>.
- [261] M. Li, X. Luo, J. Yang, Y. Sun, Applying a locally linear embedding algorithm for feature extraction and visualization of MI-EEG, *Journal of Sensors* 2016 (2016) 7481946.
- [262] X. Wang, Y. Zheng, Z. Zhao, J. Wang, Bearing fault diagnosis based on statistical locally linear embedding, *Sensors* 15 (7) (2015) 16225–16247.
- [263] A. Agrapetidou, P. Charonyktakis, P. Gogas, T. Papadimitriou, I. Tsamardinos, An AutoML application to forecasting bank failures, *Applied Economics Letters* 28 (1) (2021) 5–9.
- [264] M.B. Kursu, A. Jankowski, W.R. Rudnicki, Boruta - A system for feature selection, *Fundamenta Informaticae* 101 (4) (2010) 271–285.
- [265] F. Yang, Y. Qiao, Y. Qi, J. Bo, X. Wang, BACS: Blockchain and AutoML-based technology for efficient credit scoring classification, *Annals of Operations Research* (2022) 1–21.
- [266] V. Lopes, A. Gaspar, L.A. Alexandre, J. Cordeiro, An AutoML-based approach to multimodal image sentiment analysis, in: *Proceedings of the 2021 International Joint Conference on Neural Networks (IJCNN)*, IEEE, Piscataway, 2021, pp. 1–9.
- [267] H. Rakotoarison, L. Milijaona, A. Rasoaivao, M. Sebag, M. Schoenauer, Learning meta-features for AutoML, in: *Proceedings of the International Conference on Learning Representations*, 2022. <https://api.semanticscholar.org/CorpusID:251648903>.
- [268] C. Xue, J. Yan, R. Yan, S.M. Chu, Y. Hu, Y. Lin, Transferable AutoML by model sharing over grouped datasets, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, IEEE, Piscataway, 2019, pp. 9002–9011.
- [269] G. Peyré, M. Cuturi, Computational optimal transport: With applications to data science, *Foundations and Trends® in Machine Learning* 11 (5-6) (2019) 355–607.
- [270] G. Katz, E.C.R. Shin, D. Song, ExploreKit: Automatic feature generation and selection, in: *Proceedings of the 2016 IEEE 16th International Conference on Data Mining (ICDM)*, IEEE, Piscataway, 2016, pp. 979–984.
- [271] A. Harari, G. Katz, Automatic features generation and selection from external sources: A DBpedia use case, *Information Sciences* 582 (2022) 398–414.
- [272] A. Fatima, F.A. Khan, A. Raza, A.B. Kamran, Automated feature synthesis from relational database for data science related problems, in: *Proceedings of the 2018 International Conference on Frontiers of Information Technology (FIT)*, IEEE, Piscataway, 2018, pp. 71–75.
- [273] U. Khurana, D. Turaga, H. Samulowitz, S. Parthasarathy, Cognito: Automated feature engineering for supervised learning, in: *Proceedings of the 2016 IEEE 16th International Conference on Data Mining Workshops (ICDMW)*, IEEE, Piscataway, 2016, pp. 1304–1307.
- [274] J.M. Kanter, K. Veeramachaneni, Deep feature synthesis: Towards automating data science endeavors, in: *Proceedings of the 2015 IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, IEEE, Piscataway, 2015, pp. 1–10.
- [275] H.T. Lam, J.-M. Thiebaut, M. Sinn, B. Chen, T. Mai, O. Alkan, One button machine for automating feature engineering in relational databases, 2017. <https://arxiv.org/pdf/1706.00327.pdf>.
- [276] U. Khurana, H. Samulowitz, D. Turaga, Feature engineering for predictive modeling using reinforcement learning, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, AAAI, Melon Park, 2018, pp. 3407–3414.
- [277] I. Reyes-Amezcu, D. Flores-Araiza, G. Ochoa-Ruiz, A. Mendez-Vazquez, E. Rodríguez-Tello, MACFE: A meta-learning and causality based feature engineering framework, in: *Proceedings of the Mexican International Conference on Artificial Intelligence*, Springer, Cham, 2022, pp. 52–65.

- [278] F. Nargesian, H. Samulowitz, U. Khurana, E.B. Khalil, D.S. Turaga, Learning feature engineering for classification, in: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI-17*, 2017, pp. 2529–2535.
- [279] J. Fan, R. Li, Statistical challenges with high dimensionality: Feature selection in knowledge discovery, 2006. <https://arxiv.org/pdf/math/0602133.pdf>.
- [280] H. Liu, H. Motoda, *Feature Selection for Knowledge Discovery and Data Mining*, Springer, New York, 2012.
- [281] G. Borboudakis, I. Tsamardinos, Extending greedy feature selection algorithms to multiple solutions, *Data Mining and Knowledge Discovery* 35 (4) (2021) 1393–1434.
- [282] A. Statnikov, Algorithms for discovery of multiple Markov boundaries: application to the molecular signature multiplicity problem, Ph.D. thesis, 2008. <https://api.semanticscholar.org/CorpusID:123942725>.
- [283] H. Pham, Q.V. Le, AutoDropout: Learning dropout patterns to regularize deep networks, 2021. <https://arxiv.org/abs/2101.01761>.
- [284] V. Dodballapur, R. Calisa, Y. Song, W. Cai, Automatic dropout for deep neural networks, in: *Proceedings of the International Conference on Neural Information Processing*, Springer, 2020, pp. 185–196.
- [285] G. Zhu, S. Jiang, X. Guo, C. Yuan, Y. Huang, Evolutionary automated feature engineering, in: *Proceedings of the Pacific Rim International Conference on Artificial Intelligence*, Springer, 2022, pp. 574–586.
- [286] R.P. Bonidia, A.P.A. Santos, B.L. de Almeida, P.F. Stadler, U.N. da Rocha, D.S. Sanches, A.C. de Carvalho, BioAutoML: Automated feature engineering and metalearning to predict noncoding RNAs in bacteria, *Briefings in Bioinformatics* 23 (4) (2022) bbac218.
- [287] G. Zhu, Z. Xu, C. Yuan, Y. Huang, DIFER: Differentiable automated feature engineering, in: *Proceedings of the International Conference on Automated Machine Learning*, PMLR, 2022, pp. 1–17.
- [288] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [289] S. Ahmed, K.K. Ghosh, P.K. Singh, Z.W. Geem, R. Sarkar, Hybrid of harmony search algorithm and ring theory-based evolutionary algorithm for feature selection, *IEEE Access* 8 (2020) 102629–102645.
- [290] Y. Chen, L. Wan, Z. Li, Q. Jing, Z. Sun, Neural feature search for RGB-infrared person re-identification, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, IEEE, Piscataway, 2021, pp. 587–597.
- [291] M. Sokolova, N. Japkowicz, S. Szpakowicz, Beyond accuracy, F-score and ROC: A family of discriminant measures for performance evaluation, in: *Proceedings of the 19th Australasian joint conference on Artificial Intelligence*, Springer, 2006, pp. 1015–1021.
- [292] T. Zhang, Z.A. Zhang, Z. Fan, H. Luo, F. Liu, Q. Liu, W. Cao, L. Jian, OpenFE: Automated feature generation with expert-level performance, in: *Proceedings of the International Conference on Machine Learning*, PMLR, 2023, pp. 41880–41901.
- [293] C. Aggarwal, J. Han, J. Wang, R. Agrawal, T. Imielinski, A. Asuncion, D. Newman, *UCI Machine Learning Repository*, 2008. <https://api.semanticscholar.org/CorpusID:14643686>.
- [294] UC Irvine Machine Learning Repository, 2023. <https://archive.ics.uci.edu/>.
- [295] J. Vanschoren, J.N. Van Rijn, B. Bischl, L. Torgo, OpenML: Networked science in machine learning, *ACM SIGKDD Explorations Newsletter* 15 (2) (2014) 49–60.
- [296] Open machine learning GitHub page, 2023. <https://github.com/openml/openml/>.
- [297] R. Wang, R. Shivanna, D. Cheng, S. Jain, D. Lin, L. Hong, E. Chi, DCN V2: Improved deep & cross network and practical lessons for web-scale learning to rank systems, *Proceedings of the Web Conference 2021* (2021) 1785–1797.
- [298] W. Fan, E. Zhong, J. Peng, O. Verscheure, K. Zhang, J. Ren, R. Yan, Q. Yang, Generalized and heuristic-free feature construction for improved accuracy, in: *Proceedings of the 2010 SIAM International Conference on Data Mining*, ACM, New York, 2010, pp. 629–640.
- [299] L. Li, H. Wang, L. Zha, Q. Huang, S. Wu, G. Chen, J. Zhao, Learning a data-driven policy network for pretraining automated feature engineering, 2023. <https://openreview.net/forum?id=688hNNMigVX>.
- [300] Y. Luo, M. Wang, H. Zhou, Q. Yao, W.-W. Tu, Y. Chen, W. Dai, Q. Yang, AutoCross: Automatic feature crossing for tabular data in real-world applications, in: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ACM, New York, 2019, pp. 1936–1945.
- [301] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, LightGBM: A highly efficient gradient boosting decision tree, in: *Proceedings of the 31th International Conference on Neural Information Processing Systems*, 2017, pp. 3149–3157.
- [302] Y. Gorishniy, I. Rubachev, A. Babenko, On embeddings for numerical features in tabular deep learning, *Advances in Neural Information Processing Systems* 35 (2022) 24991–25004.
- [303] L. Grinsztajn, E. Oyallon, G. Varoquaux, Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems* 35 (2022) 507–520.
- [304] T. Swearingen, W. Drevo, B. Cyphers, A. Cuesta-Infante, A. Ross, K. Veeramachaneni, ATM: A distributed, collaborative, scalable system for automated machine learning, in: *Proceedings of the 2017 IEEE International Conference on Big Data (Big Data 2017)*, IEEE, Piscataway, 2017, pp. 151–162.
- [305] F. Mohr, M. Wever, E. Hüllermeier, ML-Plan: Automated machine learning via hierarchical planning, *Machine Learning* 107 (8) (2018) 1495–1515.
- [306] B. Komer, J. Bergstra, C. Eliasmith, Hyperopt-Sklearn: Automatic hyperparameter configuration for Scikit-learn, in: *Proceedings of the 13th Python in Science Conference*, 2014. <https://api.semanticscholar.org/CorpusID:6083252>.
- [307] Y. Zhao, R. Zhang, X. Li, AutoDESS: AutoML pipeline generation of classification with dynamic ensemble strategy selection, 2022. <https://arxiv.org/pdf/2201.00207v2.pdf>.
- [308] C. Wang, Q. Wu, M. Weimer, E. Zhu, FLAML: A fast and lightweight AutoML library, *Proceedings of Machine Learning and Systems* 3 (2021) 434–447.
- [309] A.G.C. de Sá, W.J.G.S. Pinto, L.O.V.B. Oliveira, G.L. Pappa, RECIPE: A grammar-based framework for automatically evolving classification pipelines, in: *Proceedings of the European Conference on Genetic Programming*, Springer, 2017, pp. 246–261.
- [310] A. Alaa, M. Schaar, AutoPrognosis: Automated clinical prognostic modeling via Bayesian optimization with structured kernel learning, in: *Proceedings of the International Conference on Machine Learning*, PMLR, 2018, pp. 139–148.
- [311] Z. Luo, Z. He, J. Wang, M. Dong, J. Huang, M. Chen, B. Zheng, AutoSmart: An efficient and automatic machine learning framework for temporal relational data, in: *Proceedings of the 27th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ACM, New York, 2021, pp. 3976–3984.
- [312] P. Das, N. Ivkin, T. Bansal, L. Rouesnel, P. Gautier, Z. Karnin, L. Dirac, L. Ramakrishnan, A. Perunicic, I. Shcherbatyi, et al., Amazon SageMaker Autopilot: A white box AutoML solution at scale, in: *Proceedings of the Fourth International Workshop on Data Management for End-To-End Machine Learning*, ACM, New York, 2020, pp. 1–7.
- [313] DataRobot API resources, [25-01-2024]. <https://docs.datarobot.com/en/api>.
- [314] W. Qi, C. Xu, X. Xu, AutoGluon: A revolutionary framework for landslide hazard analysis, *Natural Hazards Research* 1 (3) (2021) 103–108.
- [315] P. Gijssbers, J. Vanschoren, GAMA: Genetic automated machine learning assistant, *Journal of Open Source Software* 4 (33) (2019) 1132.
- [316] What is Custom Vision?, 2023, [25-01-2024]. <https://learn.microsoft.com/en-us/azure/ai-services/custom-vision-service/overview>.
- [317] E. Korot, Z. Guan, D. Ferraz, S.K. Wagner, G. Zhang, X. Liu, L. Faes, N. Pontikos, S.G. Finlayson, H. Khalid, et al., Code-free deep learning for multi-modality medical image classification, *Nature Machine Intelligence* 3 (4) (2021) 288–298.
- [318] A.I. Google Vertex, 2022. <https://cloud.google.com/vertex-ai>.
- [319] A. Sajeeth, H. Lee, K. Brown, T. Rompf, H. Chafi, M. Wu, A. Atreya, M. Odersky, K. Olukotun, OptiML: An implicitly parallel domain-specific language for machine learning, in: *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, 2011, pp. 609–616.
- [320] M. Ali, Announcing PyCaret 2.0: An open source, low-code machine learning library in Python, 2020. <https://pycaret.gitbook.io/docs/learn-pycaret/official-blog/announcing-pycaret-2.0>.
- [321] L. Etaati, *Machine Learning with Microsoft Technologies: Selecting the Right Architecture and Tools for Your Project*, Azure Databricks, Apress, New York, 2019, pp. 159–171.
- [322] M. Feurer, A. Klein, K. Eggenberger, J. Springenberg, M. Blum, F. Hutter, Efficient and robust automated machine learning, *Advances in Neural Information Processing Systems* 28 (2016) 2944–2952.
- [323] R. Barga, V. Fontana, W.H. Tok, *Predictive Analytics with Microsoft Azure Machine Learning*, 2nd Edition, Springer, 2015.

- [324] P. Hall, N. Gill, M. Kurka, W. Phan, Machine Learning Interpretability with H2O Driverless AI, H2O.ai, Inc., 2017.
- [325] SparkCognition's release of Darwin™ 2.0 brings increased speed and scalability to automated model building. <https://www.sparkcognition.com/sparkcognition-release-of-darwin-2-0-brings-increased-speed-and-scalability-to-automated-model-building/>.
- [326] D. Wang, P. Ram, D.K.I. Weidele, S. Liu, M. Muller, J.D. Weisz, A. Valente, A. Chaudhary, D. Torres, H. Samulowitz, et al., AutoAI: Automating the end-to-end ai lifecycle with humans-in-the-loop, in: Proceedings of the 25th International Conference on Intelligent User Interfaces Companion, ACM, New York, 2020, pp. 77–78.
- [327] N.O. Nikitin, P. Vychuzhanin, M. Sarafanov, I.S. Polonskaia, I. Revin, I.V. Barabanova, G. Maximov, A.V. Kalyuzhnaya, A. Boukhanovsky, Automated evolutionary approach for the design of composite machine learning pipelines, *Future Generation Computer Systems* 127 (2022) 109–125.
- [328] MLBox, Machine Learning Box. <https://mlbox.readthedocs.io/en/latest/>.
- [329] Microsoft neural network intelligence. <https://mlbox.readthedocs.io/en/latest/>.
- [330] Ludwig. <https://ludwig.ai/latest/>.
- [331] Predictive analytics with AutoML. <https://www.pecan.ai/content/predictive-analytics-automl/>.
- [332] M.-A. Zoller, W. Titov, T. Schlegel, M.F. Huber, XAutoML: A visual analytics tool for establishing trust in automated machine learning, 2022. <https://arxiv.org/pdf/2202.11954v2.pdf>.
- [333] D. Wang, J.D. Weisz, M. Muller, P. Ram, W. Geyer, C. Dugan, Y. Tausczik, H. Samulowitz, A. Gray, Human-AI collaboration in data science: Exploring data scientists' perceptions of automated AI, in: Proceedings of the ACM on Human-Computer Interaction 3 (CSCW), 2019, pp. 1–24.
- [334] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., Pytorch: An imperative style, high-performance deep learning library, *Advance in Neural Information Processing Systems* 32 (2019) 8026–8037.
- [335] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., Scikit-learn: Machine learning in Python, *Journal of Machine Learning Research* 12 (2011) 2825–2830.
- [336] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G.S. Corrado, A. Davis, J. Dean, M. Devin, et al., TensorFlow: Large-scale machine learning on heterogeneous distributed systems, 2016. <https://arxiv.org/pdf/1603.04467.pdf>.
- [337] A. Sestino, A. De Mauro, Leveraging artificial intelligence in business: Implications, applications and methods, *Technology Analysis & Strategic Management* 34 (1) (2022) 16–29.
- [338] C. Pompa, T. Burke, Data Science and Analytics Skills Shortage: Equipping the APEC Workforce with the Competencies Demanded by Employers, *Asia-Pacific Economic Cooperation Secretariat*, 2017.
- [339] D. Dolezel, A. McLeod, Big-data skills: Bridging the data science theory-practice gap in healthcare, *Perspectives in Health Information Management* 18 (2021) 1j. Winter.
- [340] K. Singhal, T. Tu, J. Gottweis, R. Sayres, E. Wulczyn, L. Hou, K. Clark, S. Pföhl, H. Cole-Lewis, D. Neal, M. Schaekermann, et al., Towards expert-level medical question answering with large models, 2023. <https://arxiv.org/abs/2305.09617>.
- [341] T. Han, L.C. Adams, J.-M. Papaioannou, P. Grundmann, T. Oberhauser, A. Loser, D. Truhn, K.K. Bressem, MedAlpaca - An open-source collection of medical conversational AI models and training data, 2023. <https://arxiv.org/pdf/2304.08247v1.pdf>.
- [342] K. Singhal, T. Tu, J. Gottweis, R. Sayres, E. Wulczyn, L. Hou, K. Clark, S. Pföhl, H. Cole-Lewis, D. Neal, et al., Towards expert-level medical question answering with large language models, 2023. <https://arxiv.org/pdf/2305.09617v1.pdf>.
- [343] Y. Zhou, C. Cui, J. Yoon, L. Zhang, Z. Deng, C. Finn, M. Bansal, H. Yao, Analyzing and mitigating object hallucination in large vision-language models, 2023. <https://arxiv.org/pdf/2310.00754.pdf>.
- [344] J. Hendler, Avoiding another AI winter, *IEEE Intelligent Systems* 23 (2) (2008) 2–4.
- [345] J. Huang, S.S. Gu, L. Hou, Y. Wu, X. Wang, H. Yu, J. Han, Large language models can self-improve, 2022. <https://arxiv.org/pdf/2210.11610.pdf>.



Alhassan Mumuni is currently a researcher at the Department of Electrical and Electronics Engineering, Cape Coast Technical University (CCTU), Ghana. His areas of interest are knowledge and data engineering, machine learning, robotics, virtual and augmented reality, automatic control, and embedded systems.



Fuseini Mumuni specialized in Robotics and Automated Systems at the Department of Robotics Technology and Automation of Technological Processes, Saint Petersburg State Electrotechnical University, Russia. He is currently at the University of Mines and Technology (UMaT), Tarkwa, Ghana. His research interests include control systems, robotics and artificial intelligence, cognitive systems and machine vision.